

Algoritmi in podatkovne strukture v Pythonu

Skripta po predavanjih in vajah

Teodor Jovanovski
2. oktober 2026



Predgovor

Pričujoča skripta je nastala v okviru predmeta **Računalništvo**, ki ga na Fakulteti za matematiko in fiziko v Ljubljani (FMF) izvajata **Bojan Žunkovič** in **Aljaž Osojnik**. Predmet obravnava osnove programiranja v jeziku Python ter temeljne algoritme in podatkovne strukture: rekurzijo in sestopanje, iskanje in urejanje, deli in vladaj, dinamično programiranje, objektno programiranje, iteratorje in generatorje, regularne izraze, drevesa ter grafe. Predmet obsega predavanja in vaje, pri čemer se ta skripta osredotoča na naloge z vaj.

Čeprav za to področje obstaja kakovostna tuja literatura, študenti pogosto pogrešamo gradivo, ki bi na enem mestu sistematično združevalo naloge z vaj, specifičnih za ta predmet. Ta skripta je moj poskus zapolnitve te vrzeli z namenom, da služi kot zemljevid skozi kompleksno snov algoritmov in podatkovnih struktur. Brez ustreznih gradiv ostaja znanje za študenta pogosto težko dostopno, kajti:

"For the lost traveler, the unmapped city does not exist."

Gradivo temelji na zbirki nalog iz sistema *Projekt Tomo*, ki ga razvija FMF Univerze v Ljubljani. Njen cilj ni le pokazati *končno* rešitev, ampak pokazati, *kako* do nje pridemo: kako o nalogi razmišljamo, kako jo razbijemo na manjše korake in kako vsak korak zapišemo v kodi. Skripta je zasnovana tako, da jo bralec bere z računalnikom pred seboj.

Besedilo, struktura poglavij in rešitve nalog so bili generirani s pomočjo umetne inteligence (jezikovnega modela DeepSeek) na podlagi nalog sistema Projekt Tomo. **Ker vsebina in izračuni niso bili v celoti ročno preverjeni, skripta verjetno vsebuje napake ali netočnosti.** Gradivo uporabljajte s potrebno mero kritičnosti. Vse opažene napake lahko sporočite na elektronski naslov: jovanovskiot@yahoo.com.

Ljubljana, 2. oktober 2026

Teodor Jovanovski

Kako uporabljati to skripto

Postopna gradnja kode

V tej skripti programov ne pokažemo takoj v celoti. Namesto tega vsako funkcijo zgradimo korak za korakom: vsako vrstico (ali manjšo skupino vrstic) najprej razložimo in šele nato pokažemo. V kodi so **obarvane vrstice** tiste, ki so se pravkar dodale.

Vsaka podnaloge se konča s čistim izpisom **končne kode**, ki jo lahko prepišeš in zaženeš.

Okvirčki

Skozi skripto se pojavljajo naslednji okvirčki:

Novo v tem razdelku predstavi nov pojem (sintakso ali tehniko) z minimalnim, nedvoumnim primerom.

Razmislek razloži pot od naloge do rešitve.

Zakaj deluje utemelji, zakaj je rešitev pravilna.

Pogoste napake opozori na tipične spodrslejaje.

Poskusi sam zastavi kratko vprašanje, na katerega odgovoriš sam.

Konzola prikaže primer izvajanja.

Preden začneš

Potrebuješ nameščen Python 3 in urejevalnik kode (priporočamo Visual Studio Code). Kodo iz skripte prepiši v datoteko s končnico `.py` in jo zaženi. Če se rezultat razlikuje od pričakovanega, poskusi ugotoviti, zakaj — prav odpravljanje napak je del učenja.

Zahvala in licenca

Naloge izvirajo iz sistema *Projekt Tomo*, ki ga razvija Fakulteta za matematiko in fiziko Univerze v Ljubljani. Projekt je objavljen pod licenco AGPL-3.0. Tudi ta skripta je objavljena pod isto licenco, z ustrežno navedbo vira.

Poglavje 1

Ponavljalne vaje

To poglavje je sklop ponavljalnih vaj. Skoznje bomo obnovili in utrdili osnovne prijeme programiranja v Pythonu: pisanje funkcij, zanke in pogoje, delo s seznamami in nizi, uporabo slovarjev in množic ter branje in pisanje datotek. Predpostavljamo, da že poznaš osnove (kako definirati funkcijo z `def`, kako napisati zanko `for` in stavek `if`); vse ostalo bomo sproti predstavili.

Kot v prejšnjih poglavjih bomo funkcije *gradili* po korakih: vsako vrstico posebej predstavimo in pojasnimo, *zakaj* je tam, šele nato pa jo pokažemo. **Obarvane vrstice** so tiste, ki so se pravkar dodale.

1.1 Matrike

Matriko predstavimo s **seznamom seznamov**: zunanji seznam so vrstice, vsak njegov element pa je ena vrstica. Matrika $\begin{bmatrix} 1 & 1/2 \\ 1/2 & 1/3 \end{bmatrix}$ je torej `[[1, 1/2], [1/2, 1/3]]`.

Novo v tem razdelku

Dostop do elementa matrike. Do elementa v vrstici `i` in stolpcu `j` pridemo z `mat[i][j]`: najprej izberemo vrstico `mat[i]`, nato njen `j`-ti element.

```
1 mat = [[1, 2], [3, 4]]
2 mat[0]      # [1, 2]      (prva vrstica)
3 mat[0][1]   # 2           (vrstica 0, stolpec 1)
4 len(mat)    # 2           (stevilo vrstic)
5 len(mat[0]) # 2           (stevilo stolpcev)
```

1.1.1 1. podnalog: identiteta

Cilj. `identiteta(n)` naj vrne identično matriko velikosti $n \times n$: na diagonali so enice, druge ničle.

Najprej sestavimo *eno* vrstico. Začnimo s praznim seznamom, v katerega bomo postopoma dodajali elemente:

```
1 vrstica = []
```

Zdaj se moramo sprehoditi po vseh stolpcih. Stolpce štejemo od 0 do $n - 1$, zato uporabimo `for j in range(n)`; spremenljivka `j` je indeks trenutnega stolpca:

```
1 vrstica = []
2 for j in range(n):
```

Za vsak stolpec nas zanima, ali je polje na diagonalni. Na diagonalni so natanko polja, kjer se indeks stolpca j ujema z indeksom vrstice i , torej ko velja $j == i$:

```
1 vrstica = []
2 for j in range(n):
3     if j == i:
```

Če smo na diagonalni, mora biti v polju enica. Ker seznam šele gradimo in elementa na tem mestu še ni, ga dodamo na konec z `.append`:

```
1 vrstica = []
2 for j in range(n):
3     if j == i:
4         vrstica.append(1)
```

Če polje ni na diagonalni, vanj zapišemo ničlo:

```
1 vrstica = []
2 for j in range(n):
3     if j == i:
4         vrstica.append(1)
5     else:
6         vrstica.append(0)
```

Novo v tem razdelku

Seznam: dodajanje in branje.

```
1 s = []
2 s.append(3)      # s je [3]
3 s.append(7)      # s je [3, 7]
4 s[0]             # 3   (branje po indeksu)
5 len(s)           # 2
```

`s.append(x)` doda `x` na konec seznama, `s[i]` pa prebere element na indeksu `i`.

Zdaj imamo postopek za eno vrstico. Ponovimo ga za vsako vrstico i in vrstice zberemo v matriko. Najprej pripravimo prazen seznam za celotno matriko:

```
1 def identiteta(n):
2     matrika = []
```

Nato se sprehodimo po vrsticah; vrstice štejemo od 0 do $n - 1$:

```
1 def identiteta(n):
2     matrika = []
3     for i in range(n):
```

Za vsako vrstico zgradimo njeno vsebino natanko tako kot prej (ta del že znamo, zato ga pokažemo naenkrat):

```
1 def identiteta(n):
2     matrika = []
3     for i in range(n):
4         vrstica = []
5         for j in range(n):
6             if j == i:
7                 vrstica.append(1)
8             else:
9                 vrstica.append(0)
```

Zgrajeno vrstico dodamo v matriko:

```

1 def identiteta(n):
2     matrika = []
3     for i in range(n):
4         vrstica = []
5         for j in range(n):
6             if j == i:
7                 vrstica.append(1)
8             else:
9                 vrstica.append(0)
10        matrika.append(vrstica)

```

Na koncu vrnemo zgrajeno matriko:

```

1 def identiteta(n):
2     matrika = []
3     for i in range(n):
4         vrstica = []
5         for j in range(n):
6             if j == i:
7                 vrstica.append(1)
8             else:
9                 vrstica.append(0)
10        matrika.append(vrstica)
11    return matrika

```

Novo v tem razdelku

Izpeljan seznam (*list comprehension*) zgradi seznam v eni vrstici. Namesto

```

1 kvadrati = []
2 for x in range(5):
3     kvadrati.append(x * x)

```

lahko zapišemo

```

1 kvadrati = [x * x for x in range(5)]      # [0, 1, 4, 9, 16]

```

Dodamo lahko tudi pogoj: `[x for x in range(10) if x % 2 == 0]` da `[0, 2, 4, 6, 8]`. **Pogojni izraz** `a if pogoj else b` vrne `a`, kadar pogoj velja, sicer `b`.

Vsako vrstico lahko zapišemo tudi z izpeljanim seznamom in pogojnim izrazom `1 if j == i else 0` (enica na diagonali, drugje ničla):

```

1 [1 if j == i else 0 for j in range(n)]

```

Za matriko to ponovimo za vsako vrstico `i`:

```

1 def identiteta(n):
2     return [[1 if i == j else 0 for j in range(n)] for i in range(n)]

```

Konzola

```

>>> identiteta(3)
[[1, 0, 0], [0, 1, 0], [0, 0, 1]]

```

Končna koda.

```

1 def identiteta(n):
2     return [[1 if i == j else 0 for j in range(n)] for i in range(n)]

```

1.1.2 2. podnalog: transponiraj

Cilj. `transponiraj(mat)` naj vrne transponirko: vrstice in stolpce zamenjamo.

Transponirka ima toliko vrstic, kolikor ima izvirnik stolpcev, zato si najprej zabeležimo dimenziji. `len(mat)` je število vrstic, `len(mat[0])` pa dolžina prve vrstice, torej število stolpcev:

```

1 def transponiraj(mat):
2     m = len(mat)           # stevilo vrstic
3     n = len(mat[0])        # stevilo stolpcev

```

Pripravimo prazen seznam za rezultat:

```

1 def transponiraj(mat):
2     m = len(mat)
3     n = len(mat[0])
4     nova = []

```

Vrstice transponirke ustrezajo stolpcem izvirnika, zato se zunanja zanka sprehodi po stolpcih `j`:

```

1 def transponiraj(mat):
2     m = len(mat)
3     n = len(mat[0])
4     nova = []
5     for j in range(n):

```

Za vsako vrstico rezultata pripravimo nov prazen seznam:

```

1 def transponiraj(mat):
2     m = len(mat)
3     n = len(mat[0])
4     nova = []
5     for j in range(n):
6         vrstica = []

```

Vrstico `j` transponirke sestavljajo elementi `j`-tega stolpca izvirnika, torej `mat[i][j]` za vsako vrstico `i`. Zato se notranja zanka sprehodi po vrsticah:

```

1 def transponiraj(mat):
2     m = len(mat)
3     n = len(mat[0])
4     nova = []
5     for j in range(n):
6         vrstica = []
7         for i in range(m):

```

Element iz izvirnikove vrstice `i` in stolpca `j` dodamo v novo vrstico:

```

1 def transponiraj(mat):
2     m = len(mat)
3     n = len(mat[0])
4     nova = []
5     for j in range(n):
6         vrstica = []
7         for i in range(m):
8             vrstica.append(mat[i][j])

```

Končano vrstico dodamo v rezultat in na koncu vrnemo celotno matriko:

```

1 def transponiraj(mat):
2     m = len(mat)
3     n = len(mat[0])
4     nova = []
5     for j in range(n):
6         vrstica = []
7         for i in range(m):
8             vrstica.append(mat[i][j])
9         nova.append(vrstica)
10    return nova

```

Zakaj deluje

Vrstica j transponirke je natanko j -ti stolpec izvirnika. Ker zunanja zanka teče po j , notranja pa po i , v vsako vrstico zberemo $\text{mat}[0][j]$, $\text{mat}[1][j]$, $\dots$.

Z izpeljanim seznamom je ista ideja krajša:

```

1 def transponiraj(mat):
2     m = len(mat)
3     n = len(mat[0])
4     return [[mat[i][j] for i in range(m)] for j in range(n)]

```

Končna koda.

```

1 def transponiraj(mat):
2     m = len(mat)
3     n = len(mat[0])
4     return [[mat[i][j] for i in range(m)] for j in range(n)]

```

1.1.3 3. podnalog: uporabi

Cilj. `uporabi(mat, v)` naj matriko `mat` uporabi na vektorju `v`.

Rezultat je vektor, ki ima toliko komponent, kolikor ima matrika vrstic. Zabeležimo dimenziji:

```

1 def uporabi(mat, v):
2     m = len(mat)
3     n = len(mat[0])

```

Pripravimo prazen seznam za rezultat:

```

1 def uporabi(mat, v):
2     m = len(mat)
3     n = len(mat[0])
4     rezultat = []

```

Za vsako vrstico matrike izračunamo eno komponento, zato zunanja zanka teče po vrsticah `i`:

```

1 def uporabi(mat, v):
2     m = len(mat)
3     n = len(mat[0])
4     rezultat = []
5     for i in range(m):

```

Komponenta je vsota produktov $\text{mat}[i][j] * v[j]$. Vsoto začnemo pri nič:

```

1 def uporabi(mat, v):
2     m = len(mat)
3     n = len(mat[0])
4     rezultat = []
5     for i in range(m):
6         vsota = 0

```

Po stolpcih j prištevamo produkte:

```

1 def uporabi(mat, v):
2     m = len(mat)
3     n = len(mat[0])
4     rezultat = []
5     for i in range(m):
6         vsota = 0
7         for j in range(n):
8             vsota += mat[i][j] * v[j]

```

Izračunano komponento dodamo v rezultat in vrnemo vektor:

```

1 def uporabi(mat, v):
2     m = len(mat)
3     n = len(mat[0])
4     rezultat = []
5     for i in range(m):
6         vsota = 0
7         for j in range(n):
8             vsota += mat[i][j] * v[j]
9         rezultat.append(vsota)
10    return rezultat

```

Zakaj deluje

Za vsako vrstico i seštejemo produkte njenih elementov s pripadajočimi komponentami vektorja: $\sum_j \text{mat}[i][j] \cdot v[j]$. To je natanko i -ta komponenta produkta matrike z vektorjem.

Končna koda.

```

1 def uporabi(mat, v):
2     m = len(mat)
3     n = len(mat[0])
4     rezultat = []
5     for i in range(m):
6         vsota = 0
7         for j in range(n):
8             vsota += mat[i][j] * v[j]
9         rezultat.append(vsota)
10    return rezultat

```

1.1.4 4. podnalog: seštej

Cilj. `sestej(mat1, mat2)` naj vrne vsoto matrik: istoležne elemente seštejemo.

Rezultat ima enake dimenzije kot vsota; zabeležimo jih in pripravimo prazen seznam:

```

1 def sestej(mat1, mat2):
2     m = len(mat1)

```

```

3     n = len(mat1[0])
4     rezultat = []

```

Za vsako vrstico i zgradimo novo vrstico:

```

1 def sestej(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     rezultat = []
5     for i in range(m):
6         vrstica = []

```

V vsaki vrstici po stolpcih j seštejemo istoležna elementa obeh matrik:

```

1 def sestej(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     rezultat = []
5     for i in range(m):
6         vrstica = []
7         for j in range(n):
8             vrstica.append(mat1[i][j] + mat2[i][j])

```

Vrstico dodamo v rezultat in vrnemo matriko:

```

1 def sestej(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     rezultat = []
5     for i in range(m):
6         vrstica = []
7         for j in range(n):
8             vrstica.append(mat1[i][j] + mat2[i][j])
9     rezultat.append(vrstica)
10    return rezultat

```

Zakaj deluje

Zanko po i in j uporabimo kot pri transponiranju, le da tokrat na istem mestu seštejemo elementa obeh matrik. Zato morata biti matriki enakih dimenzij.

Končna koda.

```

1 def sestej(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     rezultat = []
5     for i in range(m):
6         vrstica = []
7         for j in range(n):
8             vrstica.append(mat1[i][j] + mat2[i][j])
9     rezultat.append(vrstica)
10    return rezultat

```

1.1.5 5. podnaloge: zmnoži

Cilj. `zmnozi(mat1, mat2)` naj vrne produkt matrik.

Produkt matrike dimenzij $m \times n$ in matrike dimenzij $n \times k$ je matrika dimenzij $m \times k$. Zabeležimo vse tri dimenzije:

```

1 def zmnozi(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     k = len(mat2[0])

```

Pripravimo prazen seznam za rezultat:

```

1 def zmnozi(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     k = len(mat2[0])
5     produkt = []

```

Za vsako vrstico i rezultata pripravimo novo vrstico:

```

1 def zmnozi(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     k = len(mat2[0])
5     produkt = []
6     for i in range(m):
7         vrstica = []

```

Element (i, j) je skalarni produkt i-te vrstice prve matrike in j-tega stolpca druge, zato se srednja zanka sprehodi po stolpcih j rezultata:

```

1 def zmnozi(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     k = len(mat2[0])
5     produkt = []
6     for i in range(m):
7         vrstica = []
8         for j in range(k):

```

Skalarni produkt začnemo pri nič:

```

1 def zmnozi(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     k = len(mat2[0])
5     produkt = []
6     for i in range(m):
7         vrstica = []
8         for j in range(k):
9             vsota = 0

```

Notranja zanka c se sprehodi po skupni dimenziji n in prišteje produkte `mat1[i][c] * mat2[c][j]`:

```

1 def zmnozi(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     k = len(mat2[0])
5     produkt = []
6     for i in range(m):
7         vrstica = []
8         for j in range(k):
9             vsota = 0
10            for c in range(n):

```

```
11 vsota += mat1[i][c] * mat2[c][j]
```

Izračunani element dodamo v vrstico, vrstico v rezultat in vrnemo matriko:

```
1 def zmnozi(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     k = len(mat2[0])
5     produkt = []
6     for i in range(m):
7         vrstica = []
8         for j in range(k):
9             vsota = 0
10            for c in range(n):
11                vsota += mat1[i][c] * mat2[c][j]
12            vrstica.append(vsota)
13        produkt.append(vrstica)
14    return produkt
```

Zakaj deluje

Zunanji dve zanki določita polje (i, j) rezultata, notranja zanka c pa sešteje produkte po skupni dimenziji n. Da je produkt definiran, mora veljati `len(mat1[0]) == len(mat2)`.

Poskusi sam

Preveri z `zmnozi([[1, 2], [3, 4]], [[0, 1], [0, 0]])`. Dobiš `[[0, 1], [0, 3]]`.

Končna koda.

```
1 def zmnozi(mat1, mat2):
2     m = len(mat1)
3     n = len(mat1[0])
4     k = len(mat2[0])
5     produkt = []
6     for i in range(m):
7         vrstica = []
8         for j in range(k):
9             vsota = 0
10            for c in range(n):
11                vsota += mat1[i][c] * mat2[c][j]
12            vrstica.append(vsota)
13        produkt.append(vrstica)
14    return produkt
```

1.2 ROT13

Vsakemu znaku pripada številski koda. Metoda, ki ji tu rečemo ROT13, zamenja *k*-ti znak abecede s *k*-tim znakom s konca: 'A' postane 'Z', 'B' postane 'Y', 'C' postane 'X' in tako naprej.

Novo v tem razdelku

`ord` in `chr`. `ord` vrne kodo znaka, `chr` pa je njena inverzija.

```
1 ord('A')    # 65
2 ord('a')    # 97
3 chr(73)     # 'I'
```

Vezano primerjanje. `'A' <= znak <= 'Z'` je okrajšava za `'A' <= znak and znak <= 'Z'`.

1.2.1 1. podnalog: en znakec

Cilj. `en_znakec_rot13(znak)` naj zašifrira en znak.

Znak na k -tem mestu od začetka zamenjamo z k -tim od konca. Vsota kod nasprotnih črk je vedno enaka, zato velja nova = `ord('A') + ord('Z') - ord(znak)`. Najprej obravnavajmo velike črke:

```
1 def en_znakec_rot13(znak):
2     if 'A' <= znak <= 'Z':
3         return chr(ord('A') + ord('Z') - ord(znak))
```

Male črke preslikamo enako, le z mejama `'a'` in `'z'`:

```
1 def en_znakec_rot13(znak):
2     if 'A' <= znak <= 'Z':
3         return chr(ord('A') + ord('Z') - ord(znak))
4     elif 'a' <= znak <= 'z':
5         return chr(ord('a') + ord('z') - ord(znak))
```

Vsi drugi znaki (števila, presledki, ločila) ostanejo nespremenjeni:

```
1 def en_znakec_rot13(znak):
2     if 'A' <= znak <= 'Z':
3         return chr(ord('A') + ord('Z') - ord(znak))
4     elif 'a' <= znak <= 'z':
5         return chr(ord('a') + ord('z') - ord(znak))
6     else:
7         return znak
```

Konzola

```
>>> en_znakec_rot13('A')
'Z'
>>> en_znakec_rot13('b')
'y'
```

Končna koda.

```
1 def en_znakec_rot13(znak):
2     if 'A' <= znak <= 'Z':
3         return chr(ord('A') + ord('Z') - ord(znak))
4     elif 'a' <= znak <= 'z':
5         return chr(ord('a') + ord('z') - ord(znak))
6     else:
7         return znak
```

1.2.2 2. podnaloga: rot13

Cilj. `rot13(besedilo)` naj zašifrira celo besedilo.

Rezultat bomo gradili znak za znakom, zato pripravimo prazen niz:

```
1 def rot13(besedilo):
2     rezultat = ''
```

Sprehodimo se po vseh znakih besedila. Zanka `for znak in besedilo` vrne vsak znak posebej:

```
1 def rot13(besedilo):
2     rezultat = ''
3     for znak in besedilo:
```

Vsak znak zašifriramo z že napisano funkcijo in ga pripnemo na konec rezultata (operator `+=` pri nizih doda na konec):

```
1 def rot13(besedilo):
2     rezultat = ''
3     for znak in besedilo:
4         rezultat += en_znahec_rot13(znak)
```

Vrnemo sestavljeni niz:

```
1 def rot13(besedilo):
2     rezultat = ''
3     for znak in besedilo:
4         rezultat += en_znahec_rot13(znak)
5     return rezultat
```

Končna koda.

```
1 def rot13(besedilo):
2     rezultat = ''
3     for znak in besedilo:
4         rezultat += en_znahec_rot13(znak)
5     return rezultat
```

1.2.3 3. podnaloga: na datoteki

Cilj. `na_datoteki_rot13(vhodna, izhodna)` naj prebere vhodno datoteko in njeno šifrirano vsebino zapiše v izhodno.

Novo v tem razdelku

Datoteke. Datoteko odpremo z `open`; stavek `with` poskrbi, da se samodejno zapre.

```
1 with open('vhod.txt') as f:           # branje
2     vsebina = f.read()
3 with open('izhod.txt', 'w') as g:     # pisanje ('w' = write)
4     g.write(vsebina)
```

`f.read()` prebere celotno vsebino, `g.write(x)` pa jo zapiše.

Odpremo vhodno datoteko in preberemo vso njeno vsebino:

```
1 def na_datoteki_rot13(vhodna, izhodna):
2     with open(vhodna) as f:
3         vsebina = f.read()
```

Odpremo izhodno datoteko za pisanje in vanjo zapišemo šifrirano vsebino:

```
1 def na_datoteki_rot13(vhodna, izhodna):
2     with open(vhodna) as f:
3         vsebina = f.read()
4     with open(izhodna, 'w') as g:
5         g.write(rot13(vsebina))
```

Zakaj deluje

Ker se pri šifriranju spremenijo le črke, se presledki in nove vrstice ohranijo; zato lahko obdelamo celotno vsebino naenkrat.

Končna koda.

```
1 def na_datoteki_rot13(vhodna, izhodna):
2     with open(vhodna) as f:
3         vsebina = f.read()
4     with open(izhodna, 'w') as g:
5         g.write(rot13(vsebina))
```

1.2.4 4. podnalog: desifriraj

Cilj. `desifriraj_rot13(sifrirano)` naj dešifrira besedilo.

Če črko zašifriramo dvakrat, skoči na nasprotno stran abecede in nazaj — torej v izhodišče. Dešifriranje je zato isto kot šifriranje, zato funkciji lahko preprosto damo novo ime:

```
1 desifriraj_rot13 = rot13
```

Novo v tem razdelku

Funkcija je vrednost. Ime funkcije lahko priredimo drugemu imenu; obe imeni kažeta na isto funkcijo.

```
1 d = desifriraj_rot13    # d je zdaj drugo ime za rot13
2 d('Fklizyr')           # 'Uporab'
```

Konzola

```
>>> desifriraj_rot13('Fklizyr Krgztlile raivp!')
'Uporabi Pitagorov izrek!'
```

Končna koda.

```
1 desifriraj_rot13 = rot13
```

1.3 Praštevila

1.3.1 1. podnalog: je prastevilo

Cilj. `je_prastevilo(n)` naj vrne `True`, če je `n` praštevilo.

Najprej obravnavajmo poseben primer: število 1 ni praštevilo, zato takoj vrnemo `False`:

```

1 def je_prastevilo(n):
2     if n == 1:
3         return False

```

Praštevilu ni deljivo z nobenim številom med 2 in $n - 1$. Zato se sprehodimo po vseh kandidatih i od 2 do $n - 1$:

```

1 def je_prastevilo(n):
2     if n == 1:
3         return False
4     for i in range(2, n):

```

Če je ostanek pri deljenju $n \% i$ enak nič, je n deljivo z i in torej ni praštevilu:

```

1 def je_prastevilo(n):
2     if n == 1:
3         return False
4     for i in range(2, n):
5         if n % i == 0:
6             return False

```

Če se zanka izteče, delitelja nismo našli in n je praštevilu:

```

1 def je_prastevilo(n):
2     if n == 1:
3         return False
4     for i in range(2, n):
5         if n % i == 0:
6             return False
7     return True

```

To deluje, a je počasno. Če n ni praštevilu, ima delitelj, ki ni večji od $\sqrt{n}$; zadošča torej preverjati do $i * i \leq n$. Ker je edino sodo praštevilu 2, najprej obravnavamo sode, nato pa preizkušamo le lihe delitelje:

```

1 def je_prastevilo(n):
2     if n == 1:
3         return False
4     if n % 2 == 0:
5         return n == 2
6     i = 3
7     while i * i <= n:
8         if n % i == 0:
9             return False
10        i += 2
11    return True

```

Zakaj deluje

Če je n sodo in ni 2, ni praštevilu. Sicer preizkusimo lihe delitelje 3, 5, 7, ... do $\sqrt{n}$. Ker ima vsako sestavljeno število delitelj $\leq \sqrt{n}$, je odgovor pravilen, hkrati pa je delo bistveno manjše.

Končna koda.

```

1 def je_prastevilo(n):
2     if n == 1:
3         return False

```

```
4     if n % 2 == 0:
5         return n == 2
6     i = 3
7     while i * i <= n:
8         if n % i == 0:
9             return False
10        i += 2
11    return True
```

1.3.2 2. podnaloga: prastevilo

Cilj. `prastevilo(n)` naj vrne n -to praštevilo.

Kandidate bomo pregledovali po vrsti in šteli, koliko pravih praštevil smo že našli. Števec začnemo pri nič:

```
1 def prastevilo(n):
2     najdenih = 0
```

Prvi kandidat, ki ga bomo preverili, je 2; zato nastavimo `kandidat = 1`, saj v zanki najprej naredimo `kandidat += 1`:

```
1 def prastevilo(n):
2     najdenih = 0
3     kandidat = 1
```

Ponavljamo, dokler nismo našli n praštevil:

```
1 def prastevilo(n):
2     najdenih = 0
3     kandidat = 1
4     while najdenih < n:
```

Premaknemo se na naslednjega kandidata:

```
1 def prastevilo(n):
2     najdenih = 0
3     kandidat = 1
4     while najdenih < n:
5         kandidat += 1
```

Če je kandidat praštevilo, povečamo števec:

```
1 def prastevilo(n):
2     najdenih = 0
3     kandidat = 1
4     while najdenih < n:
5         kandidat += 1
6         if je_prastevilo(kandidat):
7             najdenih += 1
```

Ko se zanka ustavi, je `kandidat` ravno n -to praštevilo:

```
1 def prastevilo(n):
2     najdenih = 0
3     kandidat = 1
4     while najdenih < n:
5         kandidat += 1
6         if je_prastevilo(kandidat):
7             najdenih += 1
8     return kandidat
```

Konzola

```
>>> prastevilo(10)
29
```

Končna koda.

```
1 def prastevilo(n):
2     najdenih = 0
3     kandidat = 1
4     while najdenih < n:
5         kandidat += 1
6         if je_prastevilo(kandidat):
7             najdenih += 1
8     return kandidat
```

1.3.3 3. podnaloge: naslednje prastevilo

Cilj. `naslednje_prastevilo(n)` naj vrne prvo praštevílo, strogo večje od `n`.

Ker mora biti praštevílo strogo večje od `n`, začnemo s kandidatom `n + 1`:

```
1 def naslednje_prastevilo(n):
2     kandidat = n + 1
```

Dokler kandidat ni praštevílo, skačemo na naslednjega:

```
1 def naslednje_prastevilo(n):
2     kandidat = n + 1
3     while not je_prastevilo(kandidat):
4         kandidat += 1
```

Ko se zanka ustavi, vrnemo kandidata:

```
1 def naslednje_prastevilo(n):
2     kandidat = n + 1
3     while not je_prastevilo(kandidat):
4         kandidat += 1
5     return kandidat
```

Konzola

```
>>> naslednje_prastevilo(100)
101
```

Končna koda.

```
1 def naslednje_prastevilo(n):
2     kandidat = n + 1
3     while not je_prastevilo(kandidat):
4         kandidat += 1
5     return kandidat
```

1.4 Ljubezen nam je vsem v pogubo

Socialno omrežje zaljubljenosti podamo s slovarjem, ki ime osebe preslika v **množico** vseh, v katere je ta oseba zaljubljena.

Novo v tem razdelku

Slovarji in množice. `d[oseba]` vrne množico ljubcev, `d.items()` vrne pare (ključ, vrednost), `.values()` pa vse množice. `.add` doda element v množico, `.update` pa doda vse elemente druge množice.

```
1 s = {'Ana': {'Bine', 'Cene'}, 'Bine': set()}
2 'Ana' in s           # True
3 s['Ana']             # {'Bine', 'Cene'}
4 s.items()            # dict_items([('Ana', {'Bine', 'Cene'}), ...])
```

1.4.1 1. podnalog: narcisoidi

Cilj. `narcisoidi(d)` naj vrne množico tistih, ki ljubijo sami sebe.

Pripravimo prazno množico za rezultat:

```
1 def narcisoidi(d):
2     rezultat = set()
```

Sprehodimo se po vseh osebah; zanka `for oseba in d` vrne ključne slovarja, torej imena:

```
1 def narcisoidi(d):
2     rezultat = set()
3     for oseba in d:
```

Oseba je narcisoidna, če je sama v svoji množici ljubcev, torej če velja `oseba in d[oseba]`:

```
1 def narcisoidi(d):
2     rezultat = set()
3     for oseba in d:
4         if oseba in d[oseba]:
```

Tako osebo dodamo v rezultat in na koncu rezultat vrnemo:

```
1 def narcisoidi(d):
2     rezultat = set()
3     for oseba in d:
4         if oseba in d[oseba]:
5             rezultat.add(oseba)
6     return rezultat
```

Konzola

```
>>> s = {'Ana': {'Bine', 'Cene'}, 'Bine': set(),
...      'Cene': {'Bine'}, 'Davorka': {'Davorka'}, 'Eva': {'Bine'}}
>>> narcisoidi(s)
{'Davorka'}
```

Končna koda.

```
1 def narcisoidi(d):
2     rezultat = set()
```

```

3     for oseba in d:
4         if oseba in d[oseba]:
5             rezultat.add(oseba)
6     return rezultat

```

1.4.2 2. podnalog: ljubljani

Cilj. `ljubljeni(d)` naj vrne množico vseh, ki so ljubljani.

Pripravimo prazno množico:

```

1 def ljubljeni(d):
2     rezultat = set()

```

Ljubljen je vsak, ki se pojavi v kateri koli množici ljubcev. Zato se sprehodimo po vseh množicah z `d.values()`:

```

1 def ljubljeni(d):
2     rezultat = set()
3     for ljubcki in d.values():

```

Vse elemente te množice dodamo v rezultat (z `.update` dodamo celotno množico):

```

1 def ljubljeni(d):
2     rezultat = set()
3     for ljubcki in d.values():
4         rezultat.update(ljubcki)

```

Vrnemo rezultat:

```

1 def ljubljeni(d):
2     rezultat = set()
3     for ljubcki in d.values():
4         rezultat.update(ljubcki)
5     return rezultat

```

Konzola

```

>>> ljubljeni(s)
{'Bine', 'Cene', 'Davorka'}

```

Končna koda.

```

1 def ljubljeni(d):
2     rezultat = set()
3     for ljubcki in d.values():
4         rezultat.update(ljubcki)
5     return rezultat

```

1.4.3 3. podnalog: pari

Cilj. `pari(d)` naj vrne množico srečno zaljubljenih parov.

Pripravimo prazno množico parov:

```

1 def pari(d):
2     rezultat = set()

```

Sprehodimo se po vseh osebah in njihovih ljubcih; `d.items()` vrne pare (oseba, ljubcki):

```
1 def pari(d):
2     rezultat = set()
3     for oseba, ljubcki in d.items():
```

Za vsako osebo se sprehodimo po njenih ljubcih:

```
1 def pari(d):
2     rezultat = set()
3     for oseba, ljubcki in d.items():
4         for ljubcek in ljubcki:
```

Par je srečen, če se imata oba rada. Da se vsak par pojavi samo enkrat in urejen po abecedi, ga dodamo le, kadar je prva oseba abecedno pred drugo (`oseba < ljubcek`) in druga ljubi prvo (`oseba in d[ljubcek]`):

```
1 def pari(d):
2     rezultat = set()
3     for oseba, ljubcki in d.items():
4         for ljubcek in ljubcki:
5             if oseba < ljubcek and oseba in d[ljubcek]:
```

Par shranimo kot terko in na koncu vrnemo rezultat:

```
1 def pari(d):
2     rezultat = set()
3     for oseba, ljubcki in d.items():
4         for ljubcek in ljubcki:
5             if oseba < ljubcek and oseba in d[ljubcek]:
6                 rezultat.add((oseba, ljubcek))
7     return rezultat
```

Zakaj deluje

Pogoj `oseba < ljubcek` poskrbi, da par dodamo le enkrat in v abecednem vrstnem redu, `oseba in d[ljubcek]` pa preveri, da je ljubezen vzajemna. Par shranimo kot *terko* (`oseba, ljubcek`).

Konzola

```
>>> d = {'Ana': {'Bine'}, 'Bine': {'Ana'}}
>>> pari(d)
{('Ana', 'Bine')}
```

Končna koda.

```
1 def pari(d):
2     rezultat = set()
3     for oseba, ljubcki in d.items():
4         for ljubcek in ljubcki:
5             if oseba < ljubcek and oseba in d[ljubcek]:
6                 rezultat.add((oseba, ljubcek))
7     return rezultat
```

1.4.4 4. podnalog: ustrežljivi

Cilj. `ustrezljivi(oseba, d)` naj vrne vse, ki so do dane osebe posebej ustrežljivi: bodisi so zaljubljeni vanjo, bodisi v nekoga, ki je do nje ustrežljiv, in tako naprej.

Pripravimo prazno množico za rezultat:

```
1 def ustreznjivi(oseba, d):
2     rezultat = set()
```

Neposredno ustrežljivi so vsi, ki ljubijo dano osebo. To so tisti `o`, za katere velja `oseba in d[o]`; zberemo jih v množico dodani:

```
1 def ustreznjivi(oseba, d):
2     rezultat = set()
3     dodani = {o for o in d if oseba in d[o]}
```

Zdaj moramo dodati še tiste, ki ljubijo pravkar dodane, in to ponavljati, dokler dodajamo nove. Zato ponavljamo, dokler je `dodani` neprazna:

```
1 def ustreznjivi(oseba, d):
2     rezultat = set()
3     dodani = {o for o in d if oseba in d[o]}
4     while dodani:
```

V vsakem krogu najprej vse iz `dodani` dodamo v rezultat:

```
1 def ustreznjivi(oseba, d):
2     rezultat = set()
3     dodani = {o for o in d if oseba in d[o]}
4     while dodani:
5         rezultat.update(dodani)
```

Nato poiščemo novi krog: vse `o`, ki ljubijo koga iz `dodani` (`cilj in d[o]`) in jih še nismo šteli (`o not in rezultat`):

```
1 def ustreznjivi(oseba, d):
2     rezultat = set()
3     dodani = {o for o in d if oseba in d[o]}
4     while dodani:
5         rezultat.update(dodani)
6         dodani = {o for o in d
7                     for cilj in dodani
8                     if cilj in d[o] and o not in rezultat}
```

Na koncu vrnemo rezultat:

```
1 def ustreznjivi(oseba, d):
2     rezultat = set()
3     dodani = {o for o in d if oseba in d[o]}
4     while dodani:
5         rezultat.update(dodani)
6         dodani = {o for o in d
7                     for cilj in dodani
8                     if cilj in d[o] and o not in rezultat}
9     return rezultat
```

Zakaj deluje

Množica **dodani** vsebuje zadnji krog novo odkritih oseb. V vsakem krogu jih dodamo v **rezultat**, nato pa poiščemo vse, ki ljubijo koga iz tega kroga in jih še nismo šteli. Ko v krogu ni nikogar novega, se zanka ustavi. Vsak človek je obiskan največ enkrat, zato se postopek vedno ustavi.

Konzola

```
>>> s2 = {'Ana': {'Bine', 'Cene'}, 'Bine': {'Ana'}, 'Cene': {'Bine'},
...       'Davorka': {'Davorka'}, 'Eva': {'Bine'}}
>>> ustrezljivi('Cene', s2)
{'Ana', 'Bine', 'Cene', 'Eva'}
```

Končna koda.

```
1 def ustrezljivi(oseba, d):
2     rezultat = set()
3     dodani = {o for o in d if oseba in d[o]}
4     while dodani:
5         rezultat.update(dodani)
6         dodani = {o for o in d
7                   for cilj in dodani
8                   if cilj in d[o] and o not in rezultat}
9     return rezultat
```

1.5 Povzetek

- **matrika** je seznam seznamov; do elementa dostopamo z `mat[i][j]`;
- **izpeljan seznam** zgradi seznam v eni vrstici, tudi gnezden;
- `ord` in `chr` pretvarjata med znakom in njegovo kodo;
- **datoteke** odpiramo z `open` in `with`;
- **slovarji** in **množice** so priročni za preverjanje pripadnosti in združevanje;
- **transitivno zaprtje** (pri `ustrezljivi`) dobimo s ponavljanjem, dokler dodajamo nove elemente.

Preveri se

1. Zakaj pri `je_prastevilo` zadošča preverjati delitelje do $\sqrt{n}$?
2. Kako bi iz `transponiraj` dobili `zmnozi` z dvema klicema?
3. Zakaj se `ustrezljivi` ne more zagnati v neskončno zanko?
4. Kaj vrne `narcisoidi({})`?

Poglavje 2

Preproste funkcije in aritmetika

To poglavje nadaljuje ponavljanje osnov: pisali bomo preproste funkcije, uporabljali pogojne stavke, ponavljali zanke in računali. Sklop *Matrike* iz tega poglavja je enak tistemu iz 1. poglavja, zato ga ne ponavljamo; če ga želiš obnoviti, poglej razdelek *Matrike* v 1. poglavju.

2.1 Preproste funkcije

2.1.1 1. podnalog: linearna

Cilj. `linearna(a, b)` naj vrne rešitev enačbe $ax + b = 0$.

Enačbo rešimo tako, da b prenesemo na desno stran in delimo z a . Rešitev je torej $-b / a$:

```
1 def linearna(a, b):  
2     return -b / a
```

Konzola

```
>>> linearna(2, 3)  
-1.5
```

Končna koda.

```
1 def linearna(a, b):  
2     return -b / a
```

2.1.2 2. podnalog: ploscina

Cilj. `ploscina(n, a)` naj vrne ploščino pravilnega n -kotnika s stranico a .

Novo v tem razdelku

Modul `math`. V njem so matematične funkcije in konstante:

```
1 import math  
2 math.pi      # 3.141592653589793  
3 math.tan(1)   # 1.5574077246549023
```

`import math` naredimo enkrat, navadno na vrhu datoteke.

Pravilni n -kotnik razdelimo na n skladnih enakokrakih trikotnikov. Ploščina enega je $a^2 / (4 \tan(\pi/n))$, zato je ploščina n -kotnika n krat toliko:

```

1 def ploscina(n, a):
2     return a**2 * n / (4 * math.tan(math.pi / n))

```

Konzola

```

>>> ploscina(4, 2)
4.0000000000000001

```

Končna koda.

```

1 def ploscina(n, a):
2     return a**2 * n / (4 * math.tan(math.pi / n))

```

2.1.3 3. podnaloga: prestopno

Cilj. `prestopno(n)` naj vrne `True`, če je leto prestopno.

Leto je prestopno, če je deljivo s 4, a ne s 100 — razen če je deljivo s 400. Najprej preverimo deljivost s 400:

```

1 def prestopno(n):
2     return n % 400 == 0

```

Dodamo še pogoj za leta, deljiva s 4 in ne s 100. Povezava `or` poskrbi, da zadošča eden od obeh pogojev:

```

1 def prestopno(n):
2     return (n % 400 == 0) or (n % 4 == 0 and n % 100 != 0)

```

Zakaj deluje

Leta, deljiva s 400, so prestopna ne glede na ostalo, zato stojijo prva. Za druga velja: deljiva s 4 in ne s 100. Povezava `and` zahteva oba pogoja, `or` pa zadošča eden.

Končna koda.

```

1 def prestopno(n):
2     return (n % 400 == 0) or (n % 4 == 0 and n % 100 != 0)

```

2.2 Pogojni stavek

2.2.1 1. podnaloga: predznak

Cilj. `predznak(n)` naj vrne 1 za pozitivno, `-1` za negativno in 0 za ničlo.

Najprej obravnavamo pozitivna števila:

```

1 def predznak(n):
2     if n > 0:
3         return 1

```

Nato negativna; ker prvi pogoj že vrača, uporabimo `elif`:

```

1 def predznak(n):
2     if n > 0:
3         return 1

```

```

4     elif n < 0:
5         return -1

```

Če ni ne pozitivno ne negativno, je ničla:

```

1 def predznak(n):
2     if n > 0:
3         return 1
4     elif n < 0:
5         return -1
6     return 0

```

Končna koda.

```

1 def predznak(n):
2     if n > 0:
3         return 1
4     elif n < 0:
5         return -1
6     return 0

```

2.2.2 2. podnalog: limone

Cilj. `limone(n)` naj vrne pravilno sklanjan niz, npr. 'Imaš 2 limoni.'.

Najprej obravnavamo nesmiselni vnos (negativno število):

```

1 def limone(n):
2     if n < 0:
3         return 'Vnos ni smiseln.'

```

Poseben primer je nič, kjer je oblika čisto drugačna:

```

1 def limone(n):
2     if n < 0:
3         return 'Vnos ni smiseln.'
4     if n == 0:
5         return 'Nimaš nobene limone.'

```

Za vse ostale začnemo z osnovno končnico, ki jo bomo po potrebi zamenjali:

```

1 def limone(n):
2     if n < 0:
3         return 'Vnos ni smiseln.'
4     if n == 0:
5         return 'Nimaš nobene limone.'
6     koncnica = '.'

```

Za končnico šteje zadnji dve števki, torej ostanek pri deljenju s 100:

```

1 def limone(n):
2     if n < 0:
3         return 'Vnos ni smiseln.'
4     if n == 0:
5         return 'Nimaš nobene limone.'
6     koncnica = '.'
7     ostanek = n % 100

```

Zdaj glede na ostanek izberemo končnico: $1 \rightarrow \text{'o.'}$, $2 \rightarrow \text{'i.'}$, 3 ali 4 $\rightarrow \text{'e.'}$, sicer '.' :

```
1 def limone(n):
2     if n < 0:
3         return 'Vnos ni smiseln.'
4     if n == 0:
5         return 'Nimaš nobene limone.'
6     koncnica = '.'
7     ostanek = n % 100
8     if ostanek == 1:
9         koncnica = 'o.'
10    elif ostanek == 2:
11        koncnica = 'i.'
12    elif ostanek == 3 or ostanek == 4:
13        koncnica = 'e.'
```

Število pretvorimo v niz s `str` in sestavimo celoten stavek:

```
1 def limone(n):
2     if n < 0:
3         return 'Vnos ni smiseln.'
4     if n == 0:
5         return 'Nimaš nobene limone.'
6     koncnica = '.'
7     ostanek = n % 100
8     if ostanek == 1:
9         koncnica = 'o.'
10    elif ostanek == 2:
11        koncnica = 'i.'
12    elif ostanek == 3 or ostanek == 4:
13        koncnica = 'e.'
14    return 'Imaš ' + str(n) + ' limon' + koncnica
```

Konzola

```
>>> limone(2)
'Imaš 2 limoni.'
>>> limone(5)
'Imaš 5 limon.'
>>> limone(101)
'Imaš 101 limono.'
```

Končna koda.

```
1 def limone(n):
2     if n < 0:
3         return 'Vnos ni smiseln.'
4     if n == 0:
5         return 'Nimaš nobene limone.'
6     koncnica = '.'
7     ostanek = n % 100
8     if ostanek == 1:
9         koncnica = 'o.'
10    elif ostanek == 2:
11        koncnica = 'i.'
12    elif ostanek == 3 or ostanek == 4:
13        koncnica = 'e.'
14    return 'Imaš ' + str(n) + ' limon' + koncnica
```

2.3 Aritmetika

2.3.1 1. podnalog: vsota potenc

Cilj. `vsota_potenc(k, n)` naj vrne vsoto k -tih potenc prvih n naravnih števil.

Vsoto začnemo pri nič:

```
1 def vsota_potenc(k, n):
2     vsota = 0
```

Sprehodimo se po številih od 1 do n (zgornja meja $n + 1$, ker `range` ni vključujoč):

```
1 def vsota_potenc(k, n):
2     vsota = 0
3     for i in range(1, n + 1):
```

Vsakemu prištejemo njegovo k -to potenco `i**k`:

```
1 def vsota_potenc(k, n):
2     vsota = 0
3     for i in range(1, n + 1):
4         vsota += i**k
```

Vrnemo vsoto:

```
1 def vsota_potenc(k, n):
2     vsota = 0
3     for i in range(1, n + 1):
4         vsota += i**k
5     return vsota
```

Konzola

```
>>> vsota_potenc(10, 2)
1025
```

Končna koda.

```
1 def vsota_potenc(k, n):
2     vsota = 0
3     for i in range(1, n + 1):
4         vsota += i**k
5     return vsota
```

2.3.2 2. podnalog: inverzna harmonična

Cilj. `inverzna_harmonicna(x)` naj vrne, koliko členov harmonične vrste $1 + 1/2 + 1/3 + \dots$ moramo sešteti, da presežemo x .

Pripravimo vsoto in števec členov:

```
1 def inverzna_harmonicna(x):
2     vsota = 0
3     k = 0
```

Dokler vsota ne preseže x , dodajamo naslednji člen:

```

1 def inverzna_harmonicna(x):
2     vsota = 0
3     k = 0
4     while vsota <= x:

```

Povečamo števec in prištejemo člen $1/k$:

```

1 def inverzna_harmonicna(x):
2     vsota = 0
3     k = 0
4     while vsota <= x:
5         k += 1
6         vsota += 1 / k

```

Ko se zanka ustavi, je k iskano število členov:

```

1 def inverzna_harmonicna(x):
2     vsota = 0
3     k = 0
4     while vsota <= x:
5         k += 1
6         vsota += 1 / k
7     return k

```

Konzola

```

>>> inverzna_harmonicna(2)
4

```

Končna koda.

```

1 def inverzna_harmonicna(x):
2     vsota = 0
3     k = 0
4     while vsota <= x:
5         k += 1
6         vsota += 1 / k
7     return k

```

2.3.3 3. podnalog: vsota števk

Cilj. `vsota_stevk(n)` naj vrne vsoto števk naravnega števila n .

Pripravimo vsoto:

```

1 def vsota_stevk(n):
2     vsota = 0

```

Ponavljamo, dokler ima n še kakšno števko:

```

1 def vsota_stevk(n):
2     vsota = 0
3     while n > 0:

```

Zadnjo števko dobimo z ostankom pri deljenju z 10 in jo prištejemo:

```

1 def vsota_stevk(n):
2     vsota = 0

```

```

3 while n > 0:
4     vsota += n % 10

```

Nato zadnjo števk odstranimo s celoštevilskim deljenjem z 10:

```

1 def vsota_stevk(n):
2     vsota = 0
3     while n > 0:
4         vsota += n % 10
5         n //= 10

```

Vrnemo vsoto:

```

1 def vsota_stevk(n):
2     vsota = 0
3     while n > 0:
4         vsota += n % 10
5         n //= 10
6     return vsota

```

Konzola

```

>>> vsota_stevk(2014)
7

```

Končna koda.

```

1 def vsota_stevk(n):
2     vsota = 0
3     while n > 0:
4         vsota += n % 10
5         n //= 10
6     return vsota

```

2.3.4 4. podnaloga: vsote

Cilj. $vsote(n)$ naj vrne, na koliko načinov lahko n zapišemo kot vsoto strogo naraščajočih naravnih števil. Na primer, 7 lahko zapišemo na 5 načinov.

Novo v tem razdelku

Prvi stik z rekurzijo. Rekurzivna funkcija pokliče *samo sebe* z manjšim problemom. Potrebuje osnovni primer (ko se ustavi) in rekurzivni korak. Podrobno jo bomo spoznali v 10. poglavju; tu jo le uporabimo.

Razmišljamo takole: naj bo $stej(m, j)$ število načinov, da m zapišemo kot strogo naraščajočo vsoto, katere prvi člen je vsaj j . Če je m enak nič, smo našli en način (prazno vsoto):

```

1 def stej(m, j):
2     if m == 0:
3         return 1

```

Če je j večji od m , vsote ni mogoče sestaviti:

```

1 def stej(m, j):
2     if m == 0:
3         return 1

```

```

4     elif j > m:
5         return 0

```

Sicer za prvi člen poskusimo vsako število i od j do m in seštejemo načine za preostanek $m - i$, pri čemer mora biti naslednji člen strogo večji, torej vsaj $i + 1$:

```

1 def stej(m, j):
2     if m == 0:
3         return 1
4     elif j > m:
5         return 0
6     else:
7         skupaj = 0
8         for i in range(j, m + 1):
9             skupaj += stej(m - i, i + 1)
10    return skupaj

```

Iskani odgovor dobimo tako, da začnemo s prvim členom vsaj 1:

```

1 def vsote(n):
2     return stej(n, 1)

```

Konzola

```

>>> vsote(7)
5

```

Končna koda.

```

1 def stej(m, j):
2     if m == 0:
3         return 1
4     elif j > m:
5         return 0
6     else:
7         skupaj = 0
8         for i in range(j, m + 1):
9             skupaj += stej(m - i, i + 1)
10    return skupaj
11
12 def vsote(n):
13    return stej(n, 1)

```

2.4 FizzBuzz

2.4.1 1. podnaloga: fizzbuzz

Cilj. `fizzbuzz(n)` naj vrne seznam od 1 do n , kjer večkratnike 3 zamenja 'Fizz', večkratnike 5 'Buzz' in večkratnike 15 'FizzBuzz'.

Najprej napišimo pomožno funkcijo `fb(n)`, ki vrne pravilen element za eno število. Najprej preverimo deljivost s 15, ker mora imeti prednost pred 3 in 5:

```

1 def fb(n):
2     if n % 15 == 0:
3         return 'FizzBuzz'

```

Nato deljivost s 3:

```
1 def fb(n):
2     if n % 15 == 0:
3         return 'FizzBuzz'
4     elif n % 3 == 0:
5         return 'Fizz'
```

In deljivost s 5:

```
1 def fb(n):
2     if n % 15 == 0:
3         return 'FizzBuzz'
4     elif n % 3 == 0:
5         return 'Fizz'
6     elif n % 5 == 0:
7         return 'Buzz'
```

Če ni deljivo z nobenim, vrnemo kar število:

```
1 def fb(n):
2     if n % 15 == 0:
3         return 'FizzBuzz'
4     elif n % 3 == 0:
5         return 'Fizz'
6     elif n % 5 == 0:
7         return 'Buzz'
8     return n
```

Celoten seznam dobimo z izpeljanim seznamom, ki `fb` uporabi na vsakem številu od 1 do `n`:

```
1 def fizzbuzz(n):
2     return [fb(i) for i in range(1, n + 1)]
```

Konzola

```
>>> fizzbuzz(15)
[1, 2, 'Fizz', 4, 'Buzz', 'Fizz', 7, 8, 'Fizz', 'Buzz', 11, 'Fizz', 13, 14, 'FizzBuzz']
```

Končna koda.

```
1 def fb(n):
2     if n % 15 == 0:
3         return 'FizzBuzz'
4     elif n % 3 == 0:
5         return 'Fizz'
6     elif n % 5 == 0:
7         return 'Buzz'
8     return n
9
10 def fizzbuzz(n):
11     return [fb(i) for i in range(1, n + 1)]
```

2.4.2 2. podnalog: zapisi

Cilj. `zapisi(ime_datoteke, n)` naj v datoteko zapiše elemente seznama `fizzbuzz(n)`, vsakega v svojo vrstico.

Odpremo datoteko za pisanje in se sprehodimo po elementih seznama:

```
1 def zapisi(ime_datoteke, n):
2     with open(ime_datoteke, 'w') as f:
3         for x in fizzbuzz(n):
```

Vsak element izpišemo v datoteko; `print` sam doda novo vrstico:

```
1 def zapisi(ime_datoteke, n):
2     with open(ime_datoteke, 'w') as f:
3         for x in fizzbuzz(n):
4             print(x, file=f)
```

Novo v tem razdelku

Pisanje v datoteko. `print(x, file=f)` namesto na zaslon izpiše v datoteko `f`. Samodejno doda novo vrstico, zato dobimo vsak element v svoji vrstici.

Končna koda.

```
1 def zapisi(ime_datoteke, n):
2     with open(ime_datoteke, 'w') as f:
3         for x in fizzbuzz(n):
4             print(x, file=f)
```

2.4.3 3. podnalog: poisci napake

Cilj. `poisci_napake(l)` naj vrne indekse, kjer se seznam `l` razlikuje od pravega FizzBuzz zaporedja.

Pripravimo seznam za indekse napak:

```
1 def poisci_napake(l):
2     napake = []
```

Sestavimo pravilno zaporedje enake dolžine:

```
1 def poisci_napake(l):
2     napake = []
3     pravilno = fizzbuzz(len(l))
```

Sprehodimo se po indeksih in vsak element primerjamo s pravilnim:

```
1 def poisci_napake(l):
2     napake = []
3     pravilno = fizzbuzz(len(l))
4     for i in range(len(l)):
5         if l[i] != pravilno[i]:
```

Kjer se razlikujeta, zabeležimo indeks; na koncu vrnemo seznam:

```
1 def poisci_napake(l):
2     napake = []
3     pravilno = fizzbuzz(len(l))
4     for i in range(len(l)):
5         if l[i] != pravilno[i]:
```

```

6         napake.append(i)
7     return napake

```

Konzola

```

>>> poisci_napake([1, 2, 3, 'Fizz', 'Buzz', 7, 'Fizz', 8])
[2, 3, 5, 6]

```

Končna koda.

```

1 def poisci_napake(l):
2     napake = []
3     pravilno = fizzbuzz(len(l))
4     for i in range(len(l)):
5         if l[i] != pravilno[i]:
6             napake.append(i)
7     return napake

```

2.4.4 4. podnalog: buzzfizz

Cilj. buzzfizz(n) naj vrne enak seznam kot fizzbuzz, a naj bo definirana *rekurzivno*.

Osnovni primer: za $n == 0$ ni elementov, zato vrnemo prazen seznam:

```

1 def buzzfizz(n):
2     if n == 0:
3         return []

```

Sicer rekurzivno sestavimo seznam za $n - 1$ in mu pripnemo element za n :

```

1 def buzzfizz(n):
2     if n == 0:
3         return []
4     return buzzfizz(n - 1) + [fb(n)]

```

Zakaj deluje

Rekurzivni klic sestavi celoten seznam do $n - 1$, nato pa mu dodamo še zadnji element. Tako dobimo natanko seznam od 1 do n .

Končna koda.

```

1 def buzzfizz(n):
2     if n == 0:
3         return []
4     return buzzfizz(n - 1) + [fb(n)]

```

2.5 Povzetek

- **modul math** ponuja matematične funkcije in konstante;
- **pogojni stavek** if / elif / else izbira med možnostmi;
- **zanke** nabirajo vsote in štejejo člene;
- **deljenje** // in ostanek % razkrojita število na števke;
- **rekurzija** funkcijo pokliče samo sebe (podrobneje v 10. poglavju).

Preveri se

1. Zakaj pri `fb` najprej preverimo deljivost s 15?
2. Kako `vsota_stevk` iz števila pobere posamezne številke?
3. Kaj vrne `prestopno(2100)`? Zakaj?

Poglavje 3

Aritmetika in trigonometrija

V tem poglavju naloge niso vedno funkcije: nekatere zahtevajo *program*, ki izračuna rezultat in ga shrani v spremenljivko z danim imenom ali izpiše na zaslon. Take spremenljivke so navadno že podane (npr. `starost`, `margarina`), mi pa dodamo izračun.

3.1 Razpolovna doba

Cilj. Pri podani `starost` fosila izračunaj, koliko procentov ogljikovega izotopa še ni razkrojenega, in rezultat (zaokrožen navzdol) shrani v `c`. Razpolovna doba je 5730 let, delež pa je $100/2^{\text{starost}/5730}$.

Najprej shranimo razpolovno dobo v spremenljivko, da je ne ponavljamo:

```
1 h1 = 5730
```

Delež izračunamo po formuli. Potenciranje zapišemo z operatorjem `**`:

```
1 h1 = 5730
2 c = 100 / 2 ** (starost / h1)
```

Rezultat mora biti zaokrožen navzdol. Funkcija `int` odreže decimalni del, kar pri pozitivnem številu pomeni zaokrožitev navzdol:

```
1 h1 = 5730
2 c = 100 / 2 ** (starost / h1)
3 c = int(c)
```

Novo v tem razdelku

int. `int(x)` pretvori `x` v celo število tako, da *odreže* decimalni del: `int(3.9)` vrne 3, `int(3.1)` pa 3.

Konzola

```
>>> starost = 2014
>>> h1 = 5730
>>> c = 100 / 2 ** (starost / h1)
>>> c = int(c)
>>> c
78
```

Končna koda.

```

1 h1 = 5730
2 c = 100 / 2 ** (starost / h1)
3 c = int(c)

```

3.2 Min in max

3.2.1 1. podnalog: koliko tort

Cilj. Za eno torto potrebujemo 0,8 kg margarine, 2 kg moka in 1,5 kg sladkorja. Iz količin margarina, moka in sladkor izračunaj največje število tort in ga shrani v `koliko_tort`.

Za margarino: iz vsakega kilograma dobimo $1/0,8 = 10/8$ torte. Zato lahko naredimo $10 * \text{margarina} // 8$ tort:

```

1 koliko_tort = 10 * margarina // 8

```

Za moko dobimo $\text{moka} // 2$ tort, za sladkor pa $10 * \text{sladkor} // 15$:

```

1 koliko_tort = min(10 * margarina // 8, moka // 2, 10 * sladkor // 15)

```

Novo v tem razdelku

min in max. `min(a, b, c)` vrne najmanjšo, `max(...)` pa največjo vrednost med argumenti. Primer: `min(5, 2, 7)` vrne 2.

Zakaj deluje

Tort ne moremo narediti več, kot nam dopušča najbolj omejena sestavina, zato vzamemo najmanjše izmed treh števil. Ker `//` zaokroži navzdol, dobimo celo število tort.

Končna koda.

```

1 koliko_tort = min(10 * margarina // 8, moka // 2, 10 * sladkor // 15)

```

3.2.2 2. podnalog: točke sloga

Cilj. Pri smučarskih skokih je slog vsota ocen petih sodnikov (`sodnik_1 ... sodnik_5`), pri čemer izpustimo najboljšo in najslabšo. Rezultat shrani v `tocke_sloga`.

Najprej seštejemo vse ocene:

```

1 vsota = sodnik_1 + sodnik_2 + sodnik_3 + sodnik_4 + sodnik_5

```

Poiščemo najslabšo in najboljšo oceno:

```

1 vsota = sodnik_1 + sodnik_2 + sodnik_3 + sodnik_4 + sodnik_5
2 najslabsa = min(sodnik_1, sodnik_2, sodnik_3, sodnik_4, sodnik_5)
3 najboljsa = max(sodnik_1, sodnik_2, sodnik_3, sodnik_4, sodnik_5)

```

Od vsote odštejemo obe in dobimo točke sloga:

```

1 vsota = sodnik_1 + sodnik_2 + sodnik_3 + sodnik_4 + sodnik_5
2 najslabsa = min(sodnik_1, sodnik_2, sodnik_3, sodnik_4, sodnik_5)
3 najboljsa = max(sodnik_1, sodnik_2, sodnik_3, sodnik_4, sodnik_5)
4 tocke_sloga = vsota - najslabsa - najboljsa

```

Končna koda.

```
1 vsota = sodnik_1 + sodnik_2 + sodnik_3 + sodnik_4 + sodnik_5
2 najslabsa = min(sodnik_1, sodnik_2, sodnik_3, sodnik_4, sodnik_5)
3 najboljsa = max(sodnik_1, sodnik_2, sodnik_3, sodnik_4, sodnik_5)
4 tocke_sloga = vsota - najslabsa - najboljsa
```

3.3 Koti

3.3.1 1. podnalog: vsota kotov

Cilj. Seštej dva kota, podana v stopinjah, minutah in sekundah, in izpiši rezultat tako, da sta minute in sekunde manjši od 60.

Kote seštejemo po delih:

```
1 stopinje = stopinje1 + stopinje2
2 minute = minute1 + minute2
3 sekunde = sekunde1 + sekunde2
```

Če je sekund 60 ali več, jih pretvorimo v minute. Celoštevilsko deljenje s 60 pove, koliko polnih minut je v sekundah, ostanek pa so preostale sekunde:

```
1 stopinje = stopinje1 + stopinje2
2 minute = minute1 + minute2
3 sekunde = sekunde1 + sekunde2
4 minute = minute + sekunde // 60
5 sekunde = sekunde % 60
```

Enako naredimo za minute, ki jih pretvorimo v stopinje:

```
1 stopinje = stopinje1 + stopinje2
2 minute = minute1 + minute2
3 sekunde = sekunde1 + sekunde2
4 minute = minute + sekunde // 60
5 sekunde = sekunde % 60
6 stopinje = stopinje + minute // 60
7 minute = minute % 60
```

Na koncu izpišemo rezultat. `print` lahko prejme več argumentov, ki jih loči s presledki:

```
1 print('Vsota kotov je', stopinje, 'stopinj,', minute, 'minut in', sekunde, 'sekund.')
```

Konzola

```
Vsota kotov je 70 stopinj, 5 minut in 12 sekund.
```

Končna koda.

```
1 stopinje = stopinje1 + stopinje2
2 minute = minute1 + minute2
3 sekunde = sekunde1 + sekunde2
4 minute = minute + sekunde // 60
5 sekunde = sekunde % 60
6 stopinje = stopinje + minute // 60
7 minute = minute % 60
8 print('Vsota kotov je', stopinje, 'stopinj,', minute, 'minut in', sekunde, 'sekund.')
```

3.3.2 2. podnalog: radiani v stopinje

Cilj. Kot kot v radianih pretvori v stopinje in minute (decimalke odreži) ter izpiši.

Poln krog je 2π radianov ali 360 stopinj, zato stopinje dobimo tako, da pomnožimo s $360/(2\pi)$:

```
1 stopinje = kot * 360 / (2 * math.pi)
```

Celoštevilski del so stopinje, decimalni del pa pretvorimo v minute (pomnožimo s 60 in odrežemo):

```
1 stopinje = kot * 360 / (2 * math.pi)
2 minute = int(60 * (stopinje - int(stopinje)))
3 stopinje = int(stopinje)
```

Izpišemo rezultat:

```
1 print(kot, 'radianov je', stopinje, 'stopinj in', minute, 'minut.')
```

Končna koda.

```
1 stopinje = kot * 360 / (2 * math.pi)
2 minute = int(60 * (stopinje - int(stopinje)))
3 stopinje = int(stopinje)
4 print(kot, 'radianov je', stopinje, 'stopinj in', minute, 'minut.')
```

3.3.3 3. podnalog: kot med kazalcema

Cilj. Izračunaj manjši kot med urnim in minutnim kazalcem ob času `ura:minuta` ter ga izpiši v stopinjah in minutah.

Uro najprej preslikamo na 12-urni cikel, da je izračun pravilen tudi za popoldanske ure:

```
1 ura = ura % 12
```

Minutni kazalec se v eni minuti premakne za 6 stopinj, zato je njegov kot $6 * minuta$. Da se izognemo decimalnim številom, računamo v *kotnih minutah* (1 stopinja = 60 minut), torej pomnožimo s 60:

```
1 ura = ura % 12
2 alfa = 60 * (6 * minuta)
```

Urn timer se v eni uri premakne za 30 stopinj, v eni minuti pa dodatno za 0,5 stopinje:

```
1 ura = ura % 12
2 alfa = 60 * (6 * minuta)
3 beta = 60 * (30 * ura + 0.5 * minuta)
```

Vzamemo absolutno razliko in nato manjšega od obeh kotov (drugi je $360^\circ - \text{razlika}$):

```
1 ura = ura % 12
2 alfa = 60 * (6 * minuta)
3 beta = 60 * (30 * ura + 0.5 * minuta)
4 minute = abs(alfa - beta)
5 minute = min(minute, 360 * 60 - minute)
```

Decimalni del odrežemo in minute pretvorimo v stopinje in minute:

```
1 ura = ura % 12
2 alfa = 60 * (6 * minuta)
3 beta = 60 * (30 * ura + 0.5 * minuta)
4 minute = abs(alfa - beta)
```

```

5 minute = min(minute, 360 * 60 - minute)
6 minute = int(minute)
7 stopinje = minute // 60
8 minute = minute % 60

```

Izpišemo rezultat:

```

1 print('Kot med urnim in minutnim kazalcem je', stopinje, 'stopinj in', minute, 'minut
  .')

```

Konzola

```
Kot med urnim in minutnim kazalcem je 108 stopinj in 30 minut.
```

Končna koda.

```

1 ura = ura % 12
2 alfa = 60 * (6 * minuta)
3 beta = 60 * (30 * ura + 0.5 * minuta)
4 minute = abs(alfa - beta)
5 minute = min(minute, 360 * 60 - minute)
6 minute = int(minute)
7 stopinje = minute // 60
8 minute = minute % 60
9 print('Kot med urnim in minutnim kazalcem je', stopinje, 'stopinj in', minute, 'minut
  .')

```

3.4 Trigonomanija

3.4.1 1. podnalog: polarne v kartezične

Cilj. Točko, podano s polarnima koordinatama r in ϕ (kot v stopinjah), pretvori v kartezični koordinati x in y .

Kot pretvorimo iz stopinj v radiane tako, da pomnožimo s $\pi/180$. Nato uporabimo kosinus za x :

```
1 x = r * math.cos(math.pi * fi / 180)
```

In sinus za y :

```

1 x = r * math.cos(math.pi * fi / 180)
2 y = r * math.sin(math.pi * fi / 180)

```

Novo v tem razdelku

math.cos, math.sin, math.acos. Trigonometrične funkcije pričakujejo kot v *radianih*. Zveza je $\text{radiani} = \text{stopinje} \cdot \pi/180$.

Končna koda.

```

1 x = r * math.cos(math.pi * fi / 180)
2 y = r * math.sin(math.pi * fi / 180)

```

3.4.2 2. podnalog: ploščina in polmera

Cilj. Iz stranic a , b , c izračunaj ploščino trikotnika ter polmer včrtanega in očrtanega kroga. Najprej izračunamo polobseg s :

```
1 s = (a + b + c) / 2
```

Ploščino dobimo s Heronovo formulo:

```
1 s = (a + b + c) / 2
2 ploscina = (s * (s - a) * (s - b) * (s - c)) ** 0.5
```

Polmer včrtanega kroga je ploščina deljena s polobsegom:

```
1 s = (a + b + c) / 2
2 ploscina = (s * (s - a) * (s - b) * (s - c)) ** 0.5
3 r_vcrtani = ploscina / s
```

Polmer očrtanega kroga je $abc/(4 \cdot \text{ploščina})$:

```
1 s = (a + b + c) / 2
2 ploscina = (s * (s - a) * (s - b) * (s - c)) ** 0.5
3 r_vcrtani = ploscina / s
4 r_ocrtani = (a * b * c) / (4 * ploscina)
```

Novo v tem razdelku

Kvadratni koren. $x ** 0.5$ je kvadratni koren števila x .

Končna koda.

```
1 s = (a + b + c) / 2
2 ploscina = (s * (s - a) * (s - b) * (s - c)) ** 0.5
3 r_vcrtani = ploscina / s
4 r_ocrtani = (a * b * c) / (4 * ploscina)
```

3.4.3 3. podnalog: notranji koti

Cilj. Iz stranic a , b , c izračunaj notranje kote trikotnika (v radianih) in jih shrani urejene v α , β , γ .

Vsak kot dobimo iz kosinusnega izreka. Prvi kot leži nasproti stranici a :

```
1 kot_1 = math.acos((b**2 + c**2 - a**2) / (2 * b * c))
```

Drugi nasproti b :

```
1 kot_1 = math.acos((b**2 + c**2 - a**2) / (2 * b * c))
2 kot_2 = math.acos((a**2 + c**2 - b**2) / (2 * a * c))
```

Tretji nasproti c :

```
1 kot_1 = math.acos((b**2 + c**2 - a**2) / (2 * b * c))
2 kot_2 = math.acos((a**2 + c**2 - b**2) / (2 * a * c))
3 kot_3 = math.acos((a**2 + b**2 - c**2) / (2 * a * b))
```

Najmanjši kot je α , največji γ :

```
1 kot_1 = math.acos((b**2 + c**2 - a**2) / (2 * b * c))
2 kot_2 = math.acos((a**2 + c**2 - b**2) / (2 * a * c))
3 kot_3 = math.acos((a**2 + b**2 - c**2) / (2 * a * b))
4 alfa = min(kot_1, kot_2, kot_3)
5 gama = max(kot_1, kot_2, kot_3)
```

Srednji kot **beta** dobimo tako, da od vsote vseh treh odštejemo **alfa** in **gama** (vsota kotov v trikotniku je π):

```
1 alfa = min(kot_1, kot_2, kot_3)
2 gama = max(kot_1, kot_2, kot_3)
3 beta = (kot_1 + kot_2 + kot_3) - (alfa + gama)
```

Novo v tem razdelku

math.acos. `math.acos(x)` vrne kot (v radianih), katerega kosinus je `x`. Uporabimo ga pri kosinusnem izreku.

Končna koda.

```
1 kot_1 = math.acos((b**2 + c**2 - a**2) / (2 * b * c))
2 kot_2 = math.acos((a**2 + c**2 - b**2) / (2 * a * c))
3 kot_3 = math.acos((a**2 + b**2 - c**2) / (2 * a * b))
4 alfa = min(kot_1, kot_2, kot_3)
5 gama = max(kot_1, kot_2, kot_3)
6 beta = (kot_1 + kot_2 + kot_3) - (alfa + gama)
```

3.5 Povzetek

- `min` in `max` vrneta najmanjšo oz. največjo vrednost;
- `int` odreže decimalni del, `//` in `%` pa razdelita število na cele dele in ostanek;
- pretvorba med stopinjami in radiani: $\text{rad} = \text{stopinje} \cdot \pi / 180$;
- trigonometrija: `math.cos`, `math.sin`, `math.acos`.

Preveri se

1. Zakaj pri `koliko_tort` uporabimo `min` in ne `max`?
2. Kako iz sekund dobimo minute in preostale sekunde?
3. Zakaj pri kotu med kazalcema vzamemo `min(minute, 360 * 60 - minute)`?

Poglavje 4

Nizi in znaki

V tem poglavju bomo obdelovali nize: preverjali gnezdenje oklepajev, se sprehajali po znakih, iskali palindrome in premikali samoglasnike. Sklop *ROT13* iz tega poglavja je enak tistemu iz 1. poglavja, zato ga ne ponavljamo.

4.1 Gnezdenje oklepajev

Cilj. `oklepaji(niz)` naj vrne `True`, če so oklepaji v nizu pravilno gnezdeni. Na ostale znake se ne oziramo.

Ideja: štejemo odprte oklepaje. Začnemo pri nič:

```
1 def oklepaji(niz):  
2     oklepaji = 0
```

Sprehodimo se po vseh znakih niza:

```
1 def oklepaji(niz):  
2     oklepaji = 0  
3     for z in niz:
```

Ob uglepaju števec povečamo:

```
1 def oklepaji(niz):  
2     oklepaji = 0  
3     for z in niz:  
4         if z == '(':  
5             oklepaji += 1
```

Ob zaklepaju ga zmanjšamo:

```
1 def oklepaji(niz):  
2     oklepaji = 0  
3     for z in niz:  
4         if z == '(':  
5             oklepaji += 1  
6         elif z == ')':  
7             oklepaji -= 1
```

Če števec kdaj pade pod nič, je zaklepajev preveč — niz ni pravilno gnezden:

```
1 def oklepaji(niz):  
2     oklepaji = 0  
3     for z in niz:  
4         if z == '(':  
5             oklepaji += 1
```

```

6     elif z == ')':
7         oklepaji -= 1
8     if oklepaji < 0:
9         return False

```

Na koncu mora biti števec nič (vsi oklepaji so zaprti):

```

1 def oklepaji(niz):
2     oklepaji = 0
3     for z in niz:
4         if z == '(':
5             oklepaji += 1
6         elif z == ')':
7             oklepaji -= 1
8         if oklepaji < 0:
9             return False
10    return oklepaji == 0

```

Zakaj deluje

Števec je število trenutno odprtih oklepajev. Ne sme nikoli pasti pod nič (preveč zaklepajev) in mora na koncu biti nič (vsi oklepaji so zaprti).

Končna koda.

```

1 def oklepaji(niz):
2     oklepaji = 0
3     for z in niz:
4         if z == '(':
5             oklepaji += 1
6         elif z == ')':
7             oklepaji -= 1
8         if oklepaji < 0:
9             return False
10    return oklepaji == 0

```

4.2 Sprehodi

4.2.1 1. podnalog: celoštevilski sprehod

Cilj. celostevilski(sprehod) naj vrne končno število sprehoda, ki se začne pri 0 in se premika za + ali -.

Začnemo pri nič:

```

1 def celostevilski(sprehod):
2     x = 0

```

Sprehodimo se po znakih:

```

1 def celostevilski(sprehod):
2     x = 0
3     for korak in sprehod:

```

Ob + se premaknemo naprej:

```

1 def celostevilski(sprehod):
2     x = 0

```

```

3     for korak in sprehod:
4         if korak == '+':
5             x += 1

```

Ob – pa nazaj:

```

1 def celostevilski(sprehod):
2     x = 0
3     for korak in sprehod:
4         if korak == '+':
5             x += 1
6         elif korak == '-':
7             x -= 1

```

Vrnemo končni položaj:

```

1 def celostevilski(sprehod):
2     x = 0
3     for korak in sprehod:
4         if korak == '+':
5             x += 1
6         elif korak == '-':
7             x -= 1
8     return x

```

Konzola

```

>>> celostevilski('++--#---@++---++')
-1

```

Končna koda.

```

1 def celostevilski(sprehod):
2     x = 0
3     for korak in sprehod:
4         if korak == '+':
5             x += 1
6         elif korak == '-':
7             x -= 1
8     return x

```

4.2.2 2. podnalog: ravninski sprehod

Cilj. ravninski(sprehod) naj vrne končno točko sprehoda po ravnini, kjer S, J, V, Z pomenijo sever, jug, vzhod, zahod.

Začnemo v izhodišču; $x = y = 0$ naenkrat nastavi obe koordinati:

```

1 def ravninski(sprehod):
2     x = y = 0

```

Sprehodimo se po znakih:

```

1 def ravninski(sprehod):
2     x = y = 0
3     for korak in sprehod:

```

Sever poveča y, jug ga zmanjša:

```
1 def ravninski(sprehod):
2     x = y = 0
3     for korak in sprehod:
4         if korak == 'S':
5             y += 1
6         elif korak == 'J':
7             y -= 1
```

Vzhod poveča x, zahod ga zmanjša:

```
1 def ravninski(sprehod):
2     x = y = 0
3     for korak in sprehod:
4         if korak == 'S':
5             y += 1
6         elif korak == 'J':
7             y -= 1
8         elif korak == 'V':
9             x += 1
10        elif korak == 'Z':
11            x -= 1
```

Vrnemo obe koordinati kot par:

```
1 def ravninski(sprehod):
2     x = y = 0
3     for korak in sprehod:
4         if korak == 'S':
5             y += 1
6         elif korak == 'J':
7             y -= 1
8         elif korak == 'V':
9             x += 1
10        elif korak == 'Z':
11            x -= 1
12    return x, y
```

Novo v tem razdelku

Vračanje več vrednosti. `return x, y` vrne *terko* dveh vrednosti; klic jo lahko razpakiramo: `a, b = f()`.

Končna koda.

```
1 def ravninski(sprehod):
2     x = y = 0
3     for korak in sprehod:
4         if korak == 'S':
5             y += 1
6         elif korak == 'J':
7             y -= 1
8         elif korak == 'V':
9             x += 1
10        elif korak == 'Z':
11            x -= 1
12    return x, y
```

4.2.3 3. podnalog: hitri tek

Cilj. `hitri(tek)` je kot `ravninski`, le da lahko vsebuje številke od 1 do 9, ki povedo dolžino naslednjega koraka.

Začnemo v izhodišču. Dodamo še `dolzina`, privzeto 1 (navaden korak):

```
1 def hitri(tek):
2     x = y = 0
3     dolzina = 1
```

Sprehodimo se po znakih in se premaknemo za `dolzina`:

```
1 def hitri(tek):
2     x = y = 0
3     dolzina = 1
4     for korak in tek:
5         if korak == 'S':
6             y += dolzina
7         elif korak == 'J':
8             y -= dolzina
9         elif korak == 'V':
10            x += dolzina
11        elif korak == 'Z':
12            x -= dolzina
```

Če je znak številka, nastavimo novo dolžino (`int` jo pretvori v število), sicer dolžino ponastavimo na 1:

```
1 def hitri(tek):
2     x = y = 0
3     dolzina = 1
4     for korak in tek:
5         if korak == 'S':
6             y += dolzina
7         elif korak == 'J':
8             y -= dolzina
9         elif korak == 'V':
10            x += dolzina
11        elif korak == 'Z':
12            x -= dolzina
13        if korak in '123456789':
14            dolzina = int(korak)
15        else:
16            dolzina = 1
```

Vrnemo končno točko:

```
1 def hitri(tek):
2     x = y = 0
3     dolzina = 1
4     for korak in tek:
5         if korak == 'S':
6             y += dolzina
7         elif korak == 'J':
8             y -= dolzina
9         elif korak == 'V':
10            x += dolzina
11        elif korak == 'Z':
12            x -= dolzina
13        if korak in '123456789':
```

```

14         dolzina = int(korak)
15     else:
16         dolzina = 1
17     return x, y

```

Zakaj deluje

Številka velja za *naslednji* korak, zato jo obdelamo šele po premiku (ali pred njim — važno je le, da se dolžina uporabi pri naslednji smeri). Ker zaporednih števk ni, zadošča ena spremenljivka *dolzina*.

Končna koda.

```

1 def hitri(tek):
2     x = y = 0
3     dolzina = 1
4     for korak in tek:
5         if korak == 'S':
6             y += dolzina
7         elif korak == 'J':
8             y -= dolzina
9         elif korak == 'V':
10            x += dolzina
11        elif korak == 'Z':
12            x -= dolzina
13        if korak in '123456789':
14            dolzina = int(korak)
15        else:
16            dolzina = 1
17    return x, y

```

4.3 Palindromi

4.3.1 1. podnalog: palindrom

Cilj. `palindrom(niz)` naj vrne `True`, če je niz palindrom (beremo ga enako naprej in nazaj).

Postavimo se na oba konca niza: *i* na začetek, *j* na konec:

```

1 def palindrom(niz):
2     i, j = 0, len(niz) - 1

```

Primerjamo znaka na obeh koncih, dokler se ne srečata:

```

1 def palindrom(niz):
2     i, j = 0, len(niz) - 1
3     while i < j:

```

Če se znaka razlikujeta, palindrom ni:

```

1 def palindrom(niz):
2     i, j = 0, len(niz) - 1
3     while i < j:
4         if niz[i] != niz[j]:
5             return False

```

Sicer se pomaknemo za en znak navznoter:

```

1 def palindrom(niz):
2     i, j = 0, len(niz) - 1
3     while i < j:
4         if niz[i] != niz[j]:
5             return False
6         i += 1
7         j -= 1

```

Če se kazalca srečata, je niz palindrom:

```

1 def palindrom(niz):
2     i, j = 0, len(niz) - 1
3     while i < j:
4         if niz[i] != niz[j]:
5             return False
6         i += 1
7         j -= 1
8     return True

```

Novo v tem razdelku

Obrnjeni niz. Rezina `niz[::-1]` vrne niz, obrnjen naokrog. Zato je kratek test palindroma `niz == niz[::-1]`.

Končna koda.

```

1 def palindrom(niz):
2     i, j = 0, len(niz) - 1
3     while i < j:
4         if niz[i] != niz[j]:
5             return False
6         i += 1
7         j -= 1
8     return True

```

4.3.2 2. podnalog: praktično palindrom

Cilj. `prakticno_palindrom(niz)` naj vrne `True`, če je treba za palindrom zbrisati natanko en znak.

Poskusimo izbrisati vsak znak posebej in preverimo, ali preostanek je palindrom. Indeks `i` teče po vseh mestih:

```

1 def prakticno_palindrom(niz):
2     for i in range(len(niz)):

```

Znak na mestu `i` izpustimo tako, da združimo del pred njim (`niz[:i]`) in del za njim (`niz[i+1:]`). Če je to palindrom, smo uspeli:

```

1 def prakticno_palindrom(niz):
2     for i in range(len(niz)):
3         if palindrom(niz[:i] + niz[i+1:]):
4             return True

```

Če noben izbris ne pomaga, vrnemo `False`:

```

1 def prakticno_palindrom(niz):
2     for i in range(len(niz)):

```

```

3     if palindrom(niz[:i] + niz[i+1:]):
4         return True
5     return False

```

Končna koda.

```

1 def prakticno_palindrom(niz):
2     for i in range(len(niz)):
3         if palindrom(niz[:i] + niz[i+1:]):
4             return True
5     return False

```

4.3.3 3. podnalog: skoraj palindrom

Cilj. `skoraj_palindrom(niz)` naj vrne `True`, če je treba dodati, izbrisati ali zamenjati natančno en znak.

Dodajanje ali brisanje enega znaka pokrije že `prakticno_palindrom` (dodati znak palindromu je isto kot izbrisati znak iz niza s tem znakom več):

```

1 def skoraj_palindrom(niz):
2     if prakticno_palindrom(niz):
3         return True

```

Preveriti moramo še zamenjavo enega znaka. Za vsak `i` zamenjamo znak s tistim, ki bi moral stati na tem mestu (to je zrcalni znak `niz[-i-1]`):

```

1 def skoraj_palindrom(niz):
2     if prakticno_palindrom(niz):
3         return True
4     for i in range(len(niz)):
5         if palindrom(niz[:i] + niz[-i-1] + niz[i+1:]):
6             return True

```

Če nobena zamenjava ne pomaga, vrnemo `False`:

```

1 def skoraj_palindrom(niz):
2     if prakticno_palindrom(niz):
3         return True
4     for i in range(len(niz)):
5         if palindrom(niz[:i] + niz[-i-1] + niz[i+1:]):
6             return True
7     return False

```

Zakaj deluje

Znak na mestu `i` zamenjamo z `niz[-i-1]`, ki je njegov zrcalni partner. Če s tem dobimo palindrom, je bila potrebna ena zamenjava. Hkrati `prakticno_palindrom` pokrije dodajanje in brisanje, zato so obravnavani vsi trije primeri.

Končna koda.

```

1 def skoraj_palindrom(niz):
2     if prakticno_palindrom(niz):
3         return True
4     for i in range(len(niz)):
5         if palindrom(niz[:i] + niz[-i-1] + niz[i+1:]):
6             return True
7     return False

```

4.4 Disemvoweling

4.4.1 1. podnalog: prvi samoglasnik

Cilj. `prvi_samoglasnik(niz)` naj vrne položaj prvega samoglasnika.

Samoglasnike shranimo v niz; velikim črkam se izognemo tako, da dodamo še njihove velike različice z `.upper()`:

```
1 def prvi_samoglasnik(niz):
2     samoglasniki = 'aeiou'
3     samoglasniki += samoglasniki.upper()
```

Sprehodimo se po znakih skupaj z njihovimi indeksi. Funkcija `enumerate` vrne par (indeks, znak):

```
1 def prvi_samoglasnik(niz):
2     samoglasniki = 'aeiou'
3     samoglasniki += samoglasniki.upper()
4     for i, c in enumerate(niz):
```

Ko naletimo na samoglasnik, vrnemo njegov indeks:

```
1 def prvi_samoglasnik(niz):
2     samoglasniki = 'aeiou'
3     samoglasniki += samoglasniki.upper()
4     for i, c in enumerate(niz):
5         if c in samoglasniki:
6             return i
```

Novo v tem razdelku

enumerate. `enumerate(niz)` daje pare `(i, c)`: indeks in znak. Uporabno, kadar potrebujemo oboje hkrati.

```
1 for i, c in enumerate('abc'):
2     print(i, c)    # 0 a, 1 b, 2 c
```

Končna koda.

```
1 def prvi_samoglasnik(niz):
2     samoglasniki = 'aeiou'
3     samoglasniki += samoglasniki.upper()
4     for i, c in enumerate(niz):
5         if c in samoglasniki:
6             return i
```

4.4.2 2. podnalog: disemvowel

Cilj. `disemvowel(sporocilo)` naj prestavi vse samoglasnike na konec niza.

Pripravimo dva prazna niza: enega za soglasnike in ostale znake, drugega za samoglasnike:

```
1 def disemvowel(sporocilo):
2     brez = ''
3     samoglasniki = ''
```

Sprehodimo se po znakih:

```

1 def disemvowel(sporocilo):
2     brez = ''
3     samoglasniki = ''
4     for c in sporocilo:

```

Samoglasnike damo v en niz, vse ostalo v drugega:

```

1 def disemvowel(sporocilo):
2     brez = ''
3     samoglasniki = ''
4     for c in sporocilo:
5         if c in 'aeiouAEIOU':
6             samoglasniki += c
7         else:
8             brez += c

```

Na koncu ju združimo: najprej brez samoglasnikov, nato samoglasniki:

```

1 def disemvowel(sporocilo):
2     brez = ''
3     samoglasniki = ''
4     for c in sporocilo:
5         if c in 'aeiouAEIOU':
6             samoglasniki += c
7         else:
8             brez += c
9     return brez + samoglasniki

```

Konzola

```

>>> disemvowel('Banana je dobra!')
'Bnn j dbr!aaaeoa'

```

Končna koda.

```

1 def disemvowel(sporocilo):
2     brez = ''
3     samoglasniki = ''
4     for c in sporocilo:
5         if c in 'aeiouAEIOU':
6             samoglasniki += c
7         else:
8             brez += c
9     return brez + samoglasniki

```

4.4.3 3. podnalog: disemvowel na datoteki

Cilj. `disemvowel_datoteko(vhodna, izhodna)` naj na vsaki vrstici posebej izvede `disemvowel`.

Odpremo izhodno datoteko in se sprehodimo po vrsticah vhodne:

```

1 def disemvowel_datoteko(vhodna, izhodna):
2     with open(izhodna, 'w', encoding='utf-8') as g:
3         for vrstica in open(vhodna, encoding='utf-8'):

```

Vsako vrstico obdelamo. `.rstrip()` odstrani znak za novo vrstico, ki bi sicer postal del niza; `print` doda svojo:

```

1 def disemvowel_datoteko(vhodna, izhodna):
2     with open(izhodna, 'w', encoding='utf-8') as g:
3         for vrstica in open(vhodna, encoding='utf-8'):
4             print(disemvowel(vrstica.rstrip()), file=g)

```

Novo v tem razdelku

`rstrip`. `vrstica.rstrip()` odstrani presledke in nove vrstice z desnega konca niza.

Končna koda.

```

1 def disemvowel_datoteko(vhodna, izhodna):
2     with open(izhodna, 'w', encoding='utf-8') as g:
3         for vrstica in open(vhodna, encoding='utf-8'):
4             print(disemvowel(vrstica.rstrip()), file=g)

```

4.4.4 4. podnalog: razveljavi disemvowel

Cilj. `razveljavi_disemvowel(niz)` naj zvezdice `*` zamenja s samoglasniki, ki so na koncu niza.

Najprej preštejemo zvezdice; to je tudi število samoglasnikov na koncu:

```

1 def razveljavi_disemvowel(niz):
2     st_zvezdic = niz.count('*')

```

Če zvezdic ni, ni kaj početi:

```

1 def razveljavi_disemvowel(niz):
2     st_zvezdic = niz.count('*')
3     if st_zvezdic == 0:
4         return niz

```

Zadnjih `st_zvezdic` znakov so samoglasniki, ki jih bomo vračali nazaj:

```

1 def razveljavi_disemvowel(niz):
2     st_zvezdic = niz.count('*')
3     if st_zvezdic == 0:
4         return niz
5     samoglasniki = niz[-st_zvezdic:]

```

Pripravimo rezultat in števec, ki pove, kateri samoglasnik je na vrsti:

```

1 def razveljavi_disemvowel(niz):
2     st_zvezdic = niz.count('*')
3     if st_zvezdic == 0:
4         return niz
5     samoglasniki = niz[-st_zvezdic:]
6     sporočilo = ''
7     stevec = 0

```

Sprehodimo se po delu niza brez končnih samoglasnikov. Vsako zvezdico zamenjamo z naslednjim samoglasnikom:

```

1 def razveljavi_disemvowel(niz):
2     st_zvezdic = niz.count('*')
3     if st_zvezdic == 0:
4         return niz
5     samoglasniki = niz[-st_zvezdic:]

```

```

6   sporocilo = ''
7   stevec = 0
8   for c in niz[:-st_zvezdic]:
9       if c == '*':
10          sporocilo += samoglasniki[stevec]
11          stevec += 1

```

Ostale znake prepíšemo nespremenjene:

```

1  def razveljavi_disemvowel(niz):
2      st_zvezdic = niz.count('*')
3      if st_zvezdic == 0:
4          return niz
5      samoglasniki = niz[:-st_zvezdic:]
6      sporocilo = ''
7      stevec = 0
8      for c in niz[:-st_zvezdic]:
9          if c == '*':
10             sporocilo += samoglasniki[stevec]
11             stevec += 1
12         else:
13             sporocilo += c
14     return sporocilo

```

Konzola

```

>>> razveljavi_disemvowel('B*n*n* j* d*br*!aaaeoa')
'Banana je dobra!'

```

Končna koda.

```

1  def razveljavi_disemvowel(niz):
2      st_zvezdic = niz.count('*')
3      if st_zvezdic == 0:
4          return niz
5      samoglasniki = niz[:-st_zvezdic:]
6      sporocilo = ''
7      stevec = 0
8      for c in niz[:-st_zvezdic]:
9          if c == '*':
10             sporocilo += samoglasniki[stevec]
11             stevec += 1
12         else:
13             sporocilo += c
14     return sporocilo

```

4.5 Povzetek

- niz lahko obravnavamo znak za znakom z zanko for;
- enumerate da indeks in znak hkrati;
- niz[::-1] obrne niz, niz[a:b] pa vrne rezino;
- in preveri pripadnost znaka nizu;
- str.count, str.upper, str.rstrip so priročne metode.

Preveri se

1. Zakaj pri oklepaji zadošča en sam števec?
2. Kako iz dveh nizov sestavimo disemvowelan niz?
3. Zakaj je `niz == niz[::-1]` dovolj za preverjanje palindroma?

Poglavje 5

Seznami in skladi

V tem poglavju bomo obdelovali sezname: računali vsote, uporabljali sklad, primerjali zaporedja in iskali vzorce. Sklop *Praštevila* iz tega poglavja je enak tistemu iz 1. poglavja, zato ga ne ponavljamo.

5.1 Ruleta

5.1.1 1. podnaloga: število žetonov

Cilj. `koliko_zetonov(zetoni_na_zacetku, spremembe)` naj vrne končno število žetonov, kjer so v `spremembe` zapisani dobitki (+1) in izgube (−1).

Vse spremembe seštejemo in prištejemo začetnemu stanju. Funkcija `sum` sešteje vse elemente seznama:

```
1 def koliko_zetonov(zetoni_na_zacetku, spremembe):  
2     return zetoni_na_zacetku + sum(spremembe)
```

Novo v tem razdelku

`sum.sum([1, 2, 3])` vrne 6; sešteje vse elemente seznama.

Končna koda.

```
1 def koliko_zetonov(zetoni_na_zacetku, spremembe):  
2     return zetoni_na_zacetku + sum(spremembe)
```

5.1.2 2. podnaloga: največji zaslužek

Cilj. `koliko_max(zasluzki)` naj vrne največjo vsoto, ki jo Janez doseže, če igra kadar koli prekine.

Največji zaslužek začnemo pri nič:

```
1 def koliko_max(zasluzki):  
2     max_zasluzek = 0
```

Za vsak možen konec igre `j` izračunamo vsoto od začetka do `j`:

```
1 def koliko_max(zasluzki):  
2     max_zasluzek = 0  
3     for j in range(len(zasluzki)):  
4         if sum(zasluzki[0:j+1]) > max_zasluzek:
```

Če je ta vsota večja od dosedanje najboljše, si jo zapomnimo:

```

1 def koliko_max(zasluzki):
2     max_zasluzek = 0
3     for j in range(len(zasluzki)):
4         if sum(zasluzki[0:j+1]) > max_zasluzek:
5             max_zasluzek = sum(zasluzki[0:j+1])

```

Vrnemo najboljšo vsoto:

```

1 def koliko_max(zasluzki):
2     max_zasluzek = 0
3     for j in range(len(zasluzki)):
4         if sum(zasluzki[0:j+1]) > max_zasluzek:
5             max_zasluzek = sum(zasluzki[0:j+1])
6     return max_zasluzek

```

Končna koda.

```

1 def koliko_max(zasluzki):
2     max_zasluzek = 0
3     for j in range(len(zasluzki)):
4         if sum(zasluzki[0:j+1]) > max_zasluzek:
5             max_zasluzek = sum(zasluzki[0:j+1])
6     return max_zasluzek

```

5.1.3 3. podnalog: zaslužek

Cilj. `zasluzek(janez, pepe)` naj iz dveh nizov stav in rezultatov sestavi seznam zaslužkov: 1, če se ujemata, sicer `-1`.

Pripravimo prazen seznam:

```

1 def zasluzek(janez, pepe):
2     zasluzek = []

```

Sprehodimo se po vseh indeksih (niza sta enako dolga):

```

1 def zasluzek(janez, pepe):
2     zasluzek = []
3     for i in range(len(janez)):

```

Če se znaka ujemata, dodamo 1, sicer `-1`:

```

1 def zasluzek(janez, pepe):
2     zasluzek = []
3     for i in range(len(janez)):
4         if janez[i] == pepe[i]:
5             zasluzek.append(1)
6         else:
7             zasluzek.append(-1)

```

Vrnemo seznam:

```

1 def zasluzek(janez, pepe):
2     zasluzek = []
3     for i in range(len(janez)):
4         if janez[i] == pepe[i]:
5             zasluzek.append(1)
6         else:
7             zasluzek.append(-1)
8     return zasluzek

```

Konzola

```
>>> zasluzek('rčč', 'rrč')
[1, -1, 1]
```

Končna koda.

```
1 def zasluzek(janez, pepe):
2     zasluzek = []
3     for i in range(len(janez)):
4         if janez[i] == pepe[i]:
5             zasluzek.append(1)
6         else:
7             zasluzek.append(-1)
8     return zasluzek
```

5.1.4 4. podnalog: največji dobiček

Cilj. `max_dobicek(stave)` naj vrne največjo vsoto poljubnega zaporednega odseka seznama (Janez lahko začne in konča kadar koli).

Največji dobiček začnemo pri nič:

```
1 def max_dobicek(stave):
2     max_zasluzek = 0
```

Za vsak začetek *i* poskusimo vsak konec *j*:

```
1 def max_dobicek(stave):
2     max_zasluzek = 0
3     for i in range(len(stave)):
4         for j in range(i, len(stave)):
```

Vsoto odseka `stave[i:j+1]` primerjamo z najboljšo:

```
1 def max_dobicek(stave):
2     max_zasluzek = 0
3     for i in range(len(stave)):
4         for j in range(i, len(stave)):
5             zasluzim = sum(stave[i:j+1])
6             if zasluzim > max_zasluzek:
```

Če je boljša, si jo zapomnimo:

```
1 def max_dobicek(stave):
2     max_zasluzek = 0
3     for i in range(len(stave)):
4         for j in range(i, len(stave)):
5             zasluzim = sum(stave[i:j+1])
6             if zasluzim > max_zasluzek:
7                 max_zasluzek = zasluzim
8     return max_zasluzek
```

Konzola

```
>>> max_dobicek([1, 1, -1, 1, -1, 1, 1, -1])
3
```

Končna koda.

```

1 def max_dobicek(stave):
2     max_zasluzek = 0
3     for i in range(len(stave)):
4         for j in range(i, len(stave)):
5             zasluzim = sum(stave[i:j+1])
6             if zasluzim > max_zasluzek:
7                 max_zasluzek = zasluzim
8     return max_zasluzek

```

5.2 RPN kalkulator

V obrnjenem poljskem zapisu (RPN) operacijo pišemo *za* argumentoma: namesto $4 + 5$ pišemo $4\ 5\ +$. Števila dajemo na **sklad**, ob operaciji pa s sklada vzamemo dve vrhnji števili in vrnemo rezultat.

5.2.1 1. podnalog: izracunaj

Cilj. `izracunaj(a, b, op)` naj na nizih `a` in `b` izvede operacijo `op` in vrne rezultat kot niz.

Vhodna niza pretvorimo v celi števili:

```

1 def izracunaj(a, b, op):
2     a = int(a)
3     b = int(b)

```

Če je operacija `'+'`, seštejemo, sicer (gre za `'*'`) zmnožimo:

```

1 def izracunaj(a, b, op):
2     a = int(a)
3     b = int(b)
4     if op == '+':
5         return str(a + b)
6     else:
7         return str(a * b)

```

Novo v tem razdelku

`int` in `str`. `int('10')` pretvori niz v število, `str(15)` pa število v niz. Potrebujemo oboje, ker so vhod in izhod niza.

Končna koda.

```

1 def izracunaj(a, b, op):
2     a = int(a)
3     b = int(b)
4     if op == '+':
5         return str(a + b)
6     else:
7         return str(a * b)

```

5.2.2 2. podnalog: rpn

Cilj. `rpn(niz)` naj izračuna vrednost RPN izraza.

Pripravimo prazen sklad:

```
1 def rpn(niz):
2     stack = []
```

Niz razbijemo po presledkih z `.split(' ')` in pregledamo vse dele:

```
1 def rpn(niz):
2     stack = []
3     for token in niz.strip().split(' '):
```

Če je del operacija, s sklada vzamemo dve števili, izračunamo in rezultat vrnemo na sklad:

```
1 def rpn(niz):
2     stack = []
3     for token in niz.strip().split(' '):
4         if token in operacije:
5             stack.append(izracunaj(stack.pop(), stack.pop(), token))
```

Sicer je del število, zato ga damo na sklad:

```
1 def rpn(niz):
2     stack = []
3     for token in niz.strip().split(' '):
4         if token in operacije:
5             stack.append(izracunaj(stack.pop(), stack.pop(), token))
6         else:
7             stack.append(token)
```

Na koncu je na skladu samo še rezultat:

```
1 def rpn(niz):
2     stack = []
3     for token in niz.strip().split(' '):
4         if token in operacije:
5             stack.append(izracunaj(stack.pop(), stack.pop(), token))
6         else:
7             stack.append(token)
8     return int(stack[0])
```

Novo v tem razdelku

Sklad z `append` in `pop`. `stack.append(x)` doda na vrh, `stack.pop()` pa vzame in odstrani vrhnji element. To je *sklad* (LIFO): zadnji noter, prvi ven.

Končna koda.

```
1 def rpn(niz):
2     stack = []
3     for token in niz.strip().split(' '):
4         if token in operacije:
5             stack.append(izracunaj(stack.pop(), stack.pop(), token))
6         else:
7             stack.append(token)
8     return int(stack[0])
```

5.2.3 3. podnalog: običajni zapis

Cilj. `obicajni_zapis(niz)` naj iz RPN izraza sestavi običajni zapis, pri čemer ovijemo vsako operacijo v oklepaje.

Pomožna funkcija združi iz dveh operandov in operacije sestavi niz z oklepaji:

```

1 def zdruzi(a, b, op):
2     return '(' + b + ' ' + op + ' ' + a + ' ' + ')'

```

Postopek je enak kot pri rpn, le da namesto računanja sestavljamo niz:

```

1 def obicajni_zapis(niz):
2     stack = []
3     for token in niz.strip().split(' '):
4         if token in operacije:
5             stack.append(zdruzi(stack.pop(), stack.pop(), token))
6         else:
7             stack.append(token)
8     return stack[0]

```

Konzola

```

>>> obicajni_zapis('2 4 + 3 *')
'((2 + 4) * 3)'

```

Končna koda.

```

1 def zdruzi(a, b, op):
2     return '(' + b + ' ' + op + ' ' + a + ' ' + ')'
3
4 def obicajni_zapis(niz):
5     stack = []
6     for token in niz.strip().split(' '):
7         if token in operacije:
8             stack.append(zdruzi(stack.pop(), stack.pop(), token))
9         else:
10            stack.append(token)
11    return stack[0]

```

5.3 Tovornjakarji

5.3.1 1. podnalog: počitek in povprečje

Cilj. `pocitek_in_povprecje(voznja)` naj vrne par: število dni počitka in povprečno prevoženo razdaljo na delovni dan.

Pripravimo tri števec: počitke, delovne dni in skupno vsoto:

```

1 def pocitek_in_povprecje(voznja):
2     st_pocitkov = 0
3     delovnih_dni = 0
4     vsota = 0

```

Sprehodimo se po dnevih:

```

1 def pocitek_in_povprecje(voznja):
2     st_pocitkov = 0
3     delovnih_dni = 0
4     vsota = 0
5     for d in voznja:

```

Nič pomeni počitek, drugače je bil delovni dan:

```

1 def pocitek_in_povprecje(voznja):
2     st_pocitkov = 0
3     delovnih_dni = 0
4     vsota = 0
5     for d in voznja:
6         if d == 0:
7             st_pocitkov += 1
8         else:
9             delovnih_dni += 1

```

Vsak dan prištejemo k vsoti in na koncu vrnemo povprečje:

```

1 def pocitek_in_povprecje(voznja):
2     st_pocitkov = 0
3     delovnih_dni = 0
4     vsota = 0
5     for d in voznja:
6         if d == 0:
7             st_pocitkov += 1
8         else:
9             delovnih_dni += 1
10    vsota += d
11    return st_pocitkov, vsota / delovnih_dni

```

Končna koda.

```

1 def pocitek_in_povprecje(voznja):
2     st_pocitkov = 0
3     delovnih_dni = 0
4     vsota = 0
5     for d in voznja:
6         if d == 0:
7             st_pocitkov += 1
8         else:
9             delovnih_dni += 1
10    vsota += d
11    return st_pocitkov, vsota / delovnih_dni

```

5.3.2 2. podnalog: primerjaj

Cilj. primerjaj(prvi, drugi) naj za vsak dan vrne, kdo je do tedaj prevozil več: 1, 2 ali 'enako'.

Pripravimo seznam rezultatov in dve tekoči vsoti:

```

1 def primerjaj(prvi, drugi):
2     vsota_1 = 0
3     vsota_2 = 0
4     primerjava = []

```

Za vsak dan prištejemo razdaljo obema in primerjamo tekoči vsoti:

```

1 def primerjaj(prvi, drugi):
2     vsota_1 = 0
3     vsota_2 = 0
4     primerjava = []
5     for i in range(len(prvi)):
6         vsota_1 += prvi[i]
7         vsota_2 += drugi[i]
8         if vsota_1 > vsota_2:

```

Glede na primerjavo dodamo 1, 2 ali 'enako':

```
1 def primerjaj(prvi, drugi):
2     vsota_1 = 0
3     vsota_2 = 0
4     primerjava = []
5     for i in range(len(prvi)):
6         vsota_1 += prvi[i]
7         vsota_2 += drugi[i]
8         if vsota_1 > vsota_2:
9             primerjava.append(1)
10        elif vsota_1 < vsota_2:
11            primerjava.append(2)
12        else:
13            primerjava.append('enako')
14    return primerjava
```

Končna koda.

```
1 def primerjaj(prvi, drugi):
2     vsota_1 = 0
3     vsota_2 = 0
4     primerjava = []
5     for i in range(len(prvi)):
6         vsota_1 += prvi[i]
7         vsota_2 += drugi[i]
8         if vsota_1 > vsota_2:
9             primerjava.append(1)
10        elif vsota_1 < vsota_2:
11            primerjava.append(2)
12        else:
13            primerjava.append('enako')
14    return primerjava
```

5.3.3 3. podnaloga: po pravilih

Cilj. `po_pravilih(voznja)` naj preveri dve pravili: povprečje treh zaporednih dni ne sme preseči 500 km in brez počitka ne sme biti več kot 5 zaporednih dni.

Najprej preverimo dolžino niza delovnih dni. Števec **zaporedno** ob počitku ponastavimo:

```
1 def po_pravilih(voznja):
2     zaporedno = 0
3     for d in voznja:
4         if d == 0:
5             zaporedno = 0
6         else:
7             zaporedno += 1
8         if zaporedno > 5:
9             return False
```

Nato preverimo povprečje vsakih treh zaporednih dni:

```
1 def po_pravilih(voznja):
2     zaporedno = 0
3     for d in voznja:
4         if d == 0:
5             zaporedno = 0
6         else:
7             zaporedno += 1
```

```

8     if zaporedno > 5:
9         return False
10    for i in range(len(voznja) - 2):
11        povprecje = (voznja[i] + voznja[i+1] + voznja[i+2]) / 3
12        if povprecje > 500:
13            return False
14    return True

```

Zakaj deluje

Zanka `range(len(voznja) - 2)` poskrbi, da `i+2` ne sega čez seznam. Če katero koli pravilo ni izpolnjeno, takoj vrnemo `False`; če preživimo oba preizkusa, vrnemo `True`.

Končna koda.

```

1 def po_pravilih(voznja):
2     zaporedno = 0
3     for d in voznja:
4         if d == 0:
5             zaporedno = 0
6         else:
7             zaporedno += 1
8         if zaporedno > 5:
9             return False
10    for i in range(len(voznja) - 2):
11        povprecje = (voznja[i] + voznja[i+1] + voznja[i+2]) / 3
12        if povprecje > 500:
13            return False
14    return True

```

5.3.4 4. podnalog: preveri vse

Cilj. `preveri_vse(seznam_vozenj)` naj za vsakega tovornjaka vrne, ali je vozil po pravilih.

Pripravimo seznam rezultatov in se sprehodimo po vožnjah:

```

1 def preveri_vse(seznam_vozenj):
2     rezultat = []
3     for k in seznam_vozenj:

```

Za vsako vožnjo dodamo rezultat funkcije `po_pravilih`:

```

1 def preveri_vse(seznam_vozenj):
2     rezultat = []
3     for k in seznam_vozenj:
4         rezultat.append(po_pravilih(k))
5     return rezultat

```

Končna koda.

```

1 def preveri_vse(seznam_vozenj):
2     rezultat = []
3     for k in seznam_vozenj:
4         rezultat.append(po_pravilih(k))
5     return rezultat

```

5.4 Malica

5.4.1 1. podnalog: brez ponovitev

Cilj. `brez_ponovitev(l)` naj vrne seznam, v katerem se vsak element pojavi samo enkrat, v vrstnem redu prvih pojavitev.

Pripravimo prazen seznam in se sprehodimo po elementih:

```
1 def brez_ponovitev(l):
2     nov = []
3     for x in l:
```

Element dodamo le, če ga še ni v novem seznamu:

```
1 def brez_ponovitev(l):
2     nov = []
3     for x in l:
4         if x not in nov:
5             nov.append(x)
6     return nov
```

Novo v tem razdelku

`in` in `not in`. `x in seznam` preveri, ali se `x` pojavi v seznamu; `x not in seznam` je nasprotno.

Končna koda.

```
1 def brez_ponovitev(l):
2     nov = []
3     for x in l:
4         if x not in nov:
5             nov.append(x)
6     return nov
```

5.4.2 2. podnalog: kdaj prej

Cilj. `kdaj_prej(l)` naj za vsak dan vrne, koliko dni je minilo, odkar je bila ista jed nazadnje na jedilniku (jedilnik je periodičen).

Pripravimo rezultat in si zapomnimo dolžino jedilnika:

```
1 def kdaj_prej(l):
2     ans = []
3     n = len(l)
```

Za vsak dan `i` iščemo nazaj, koliko dni je minilo. Začnemo z 1 dan nazaj:

```
1 def kdaj_prej(l):
2     ans = []
3     n = len(l)
4     for i in range(n):
5         p = 1
6         j = i - 1
```

Premikamo se nazaj, dokler ne naletimo na isto jed. Ker je jedilnik periodičen, uporabimo indeks `j % n`:

```

1 def kdaj_prej(l):
2     ans = []
3     n = len(l)
4     for i in range(n):
5         p = 1
6         j = i - 1
7         while l[j % n] != l[i]:
8             j -= 1
9             p += 1

```

Najdeno število dni dodamo v rezultat in ga vrnemo:

```

1 def kdaj_prej(l):
2     ans = []
3     n = len(l)
4     for i in range(n):
5         p = 1
6         j = i - 1
7         while l[j % n] != l[i]:
8             j -= 1
9             p += 1
10    ans.append(p)
11    return ans

```

Konzola

```

>>> kdaj_prej(['burek', 'jogurt', 'burek'])
[1, 3, 2]

```

Končna koda.

```

1 def kdaj_prej(l):
2     ans = []
3     n = len(l)
4     for i in range(n):
5         p = 1
6         j = i - 1
7         while l[j % n] != l[i]:
8             j -= 1
9             p += 1
10    ans.append(p)
11    return ans

```

5.4.3 3. podnalog: raznolikost in kakovost

Cilj. `raznolikost_in_kakovost(l)` naj vrne par: raznolikost (število različnih jedi) in kakovost (povprečje kvadratov vrednosti iz `kdaj_prej`).

Raznolikost dobimo kar z dolžino seznama brez ponovitev:

```

1 def raznolikost_in_kakovost(l):
2     bp = brez_ponovitev(l)

```

Vrednosti `kdaj_prej` kvadriramo in izračunamo povprečje:

```

1 def raznolikost_in_kakovost(l):
2     bp = brez_ponovitev(l)

```

```

3  kp = [x**2 for x in kdaj_prej(1)]
4  povp = sum(kp) / len(kp)

```

Vrnemo obe števili:

```

1  def raznolikost_in_kakovost(l):
2      bp = brez_ponovitev(l)
3      kp = [x**2 for x in kdaj_prej(l)]
4      povp = sum(kp) / len(kp)
5      return len(bp), povp

```

Končna koda.

```

1  def raznolikost_in_kakovost(l):
2      bp = brez_ponovitev(l)
3      kp = [x**2 for x in kdaj_prej(l)]
4      povp = sum(kp) / len(kp)
5      return len(bp), povp

```

5.4.4 4. podnalog: naj jedilnik

Cilj. naj_jedilnik(ll) naj vrne najboljši jedilnik: najprej gledamo raznolikost, nato kakovost.

Najboljšega še nimamo:

```

1  def naj_jedilnik(ll):
2      naj = None

```

Sprehodimo se po vseh jedilnikih:

```

1  def naj_jedilnik(ll):
2      naj = None
3      for l in ll:

```

Prvega vzamemo za začetnega:

```

1  def naj_jedilnik(ll):
2      naj = None
3      for l in ll:
4          if naj is None:
5              naj = l

```

Sicer primerjamo par (raznolikost, kakovost). Python para primerja najprej po prvem elementu, nato po drugem — natanko tako, kot želimo:

```

1  def naj_jedilnik(ll):
2      naj = None
3      for l in ll:
4          if naj is None:
5              naj = l
6          elif raznolikost_in_kakovost(l) > raznolikost_in_kakovost(naj):
7              naj = l
8      return naj

```

Zakaj deluje

Terki primerjamo *leksikografsko*: najprej po raznolikosti, ob izenačenju pa po kakovosti. Zato ena sama primerjava > izbere pravi jedilnik.

Končna koda.

```
1 def naj_jedilnik(ll):
2     naj = None
3     for l in ll:
4         if naj is None:
5             naj = l
6         elif raznolikost_in_kakovost(l) > raznolikost_in_kakovost(naj):
7             naj = l
8     return naj
```

5.5 Povzetek

- `sum` sešteje elemente seznama;
- `sklad` (`append` / `pop`) uporabimo pri RPN;
- `.split(' ')` razbije niz na dele, `.strip()` odstrani presledke;
- `in` / `not in` preverita pripadnost;
- terke primerjamo leksikografsko (najprej prvi element, nato drugi).

Preveri se

1. Zakaj pri `rpn` uporabimo sklad in ne seznama, ki bi mu brali z začetka?
2. Kaj vrne `max_dobicek([-1, -2, -3])`? Zakaj?
3. Zakaj kdaj prej uporabi `j % n`?

Poglavje 6

Slovarji in permutacije

V tem poglavju bomo delali s slovarji in množicami: sestavljali recepte, šifrirali besedila, obravnavali permutacije in analizirali kalorije. Sklop *Ljubezen nam je vsem v pogubo* je enak tistemu iz 1. poglavja, zato ga ne ponavljamo.

6.1 Kuhamo in pečemo

6.1.1 1. podnalog: pomnoži

Cilj. `pomnozi(recept, faktor)` naj vrne nov recept z vsemi količinami, pomnoženimi s faktor.

Pripravimo prazen slovar:

```
1 def pomnozi(recept, faktor):  
2     nov_recept = {}
```

Sprehodimo se po sestavinah in količinah; `.items()` da pare:

```
1 def pomnozi(recept, faktor):  
2     nov_recept = {}  
3     for sestavina, kolicina in recept.items():
```

V nov slovar shranimo pomnoženo količino:

```
1 def pomnozi(recept, faktor):  
2     nov_recept = {}  
3     for sestavina, kolicina in recept.items():  
4         nov_recept[sestavina] = kolicina * faktor  
5     return nov_recept
```

Novo v tem razdelku

Nov par v slovar. `d[kljuc] = vrednost` doda nov par (ali posodobi obstoječega). **Izpeljan slovar** zapiše isto v eni vrstici:

```
1 {k: v * 2 for k, v in d.items()}
```

Končna koda.

```
1 def pomnozi(recept, faktor):  
2     nov_recept = {}  
3     for sestavina, kolicina in recept.items():  
4         nov_recept[sestavina] = kolicina * faktor  
5     return nov_recept
```

6.1.2 2. podnalog: imamo sestavine

Cilj. `imamo_sestavine(recept, shramba)` naj preveri, ali imamo v shrambi dovolj vsake sestavine.

Sprehodimo se po sestavinah recepta:

```
1 def imamo_sestavine(recept, shramba):  
2     for sestavina, kolicina in recept.items():
```

Če imamo v shrambi premalo, takoj vrnemo `False`. Če sestavine v shrambi sploh ni, jo obravnavamo kot nič — to omogoča `.get`:

```
1 def imamo_sestavine(recept, shramba):  
2     for sestavina, kolicina in recept.items():  
3         if shramba.get(sestavina, 0) < kolicina:  
4             return False
```

Če preživimo vse sestavine, imamo vse:

```
1 def imamo_sestavine(recept, shramba):  
2     for sestavina, kolicina in recept.items():  
3         if shramba.get(sestavina, 0) < kolicina:  
4             return False  
5     return True
```

Novo v tem razdelku

`d.get(kljuc, privzeto)`. Vrne vrednost za `kljuc`, če ključ obstaja, sicer pa `privzeto`. Tako se izognemo napaki, ko ključa ni: `shramba.get('jajca', 0)` vrne 0, če jajc ni v shrambi.

Končna koda.

```
1 def imamo_sestavine(recept, shramba):  
2     for sestavina, kolicina in recept.items():  
3         if shramba.get(sestavina, 0) < kolicina:  
4             return False  
5     return True
```

6.1.3 3. podnalog: potrebno kupiti

Cilj. `potrebno_kupiti(recept, shramba)` naj vrne slovar sestavin, ki jih moramo še dokupiti.

Pripravimo prazen slovar in se sprehodimo po receptu:

```
1 def potrebno_kupiti(recept, shramba):  
2     kupiti = {}  
3     for sestavina, kolicina in recept.items():
```

Izračunamo razliko med potrebno in razpoložljivo količino:

```
1 def potrebno_kupiti(recept, shramba):  
2     kupiti = {}  
3     for sestavina, kolicina in recept.items():  
4         razlika = kolicina - shramba.get(sestavina, 0)
```

Če je razlika pozitivna, sestavino dodamo med tiste, ki jih je treba kupiti:

```

1 def potrebno_kupiti(recept, shramba):
2     kupiti = {}
3     for sestavina, kolicina in recept.items():
4         razlika = kolicina - shramba.get(sestavina, 0)
5         if razlika > 0:
6             kupiti[sestavina] = razlika
7     return kupiti

```

Končna koda.

```

1 def potrebno_kupiti(recept, shramba):
2     kupiti = {}
3     for sestavina, kolicina in recept.items():
4         razlika = kolicina - shramba.get(sestavina, 0)
5         if razlika > 0:
6             kupiti[sestavina] = razlika
7     return kupiti

```

6.2 Šifriranje

6.2.1 1. podnalog: sifriraj

Cilj. `sifriraj(sifra, beseda)` naj zašifrira besedo s substitucijsko šifro `sifra` (slovar črka → črka).

Za vsako črko besede poiščemo njeno zamenjavo `sifra[crka]` in jih združimo v niz. To naredi `".join(...)"` nad izpeljanim seznamom:

```

1 def sifriraj(sifra, beseda):
2     return ''.join([sifra[crka] for crka in beseda])

```

Novo v tem razdelku

`str.join. ".join(seznam)"` združi elemente seznama v en niz, mednje pa vstavi niz, na katerem je metoda. Ker je ta niz prazen (`"`), elemente preprosto staknemo.

Končna koda.

```

1 def sifriraj(sifra, beseda):
2     return ''.join([sifra[crka] for crka in beseda])

```

6.2.2 2. podnalog: je sifra

Cilj. `je_sifra(slovar)` naj vrne `True`, če je slovar bijekcija na abecedi (vsaka črka ima natanko eno zamenjavo in vsaka se uporabi natanko enkrat).

Ključni slovarja so po definiciji enolični. Če je tudi množica vrednosti enaka množici ključev, je vsaka črka uporabljena natanko enkrat in slovar je bijekcija:

```

1 def je_sifra(slovar):
2     return set(slovar.keys()) == set(slovar.values())

```

Zakaj deluje

Če se množica vrednosti ujema z množico ključev, je preslikava surjektivna. Ker ima vsaka vrednost natanko en izvir (ključji so enolični), je tudi injektivna, torej bijektivna.

Končna koda.

```

1 def je_sifra(slovar):
2     return set(slovar.keys()) == set(slovar.values())

```

6.2.3 3. podnalog: inverz

Cilj. `inverz(sifra)` naj vrne inverzno šifro ali `None`, če inverz ne obstaja.

Inverz obstaja le, če je šifra bijekcija:

```

1 def inverz(sifra):
2     if not je_sifra(sifra):
3         return None

```

Pripravimo prazen slovar:

```

1 def inverz(sifra):
2     if not je_sifra(sifra):
3         return None
4     inv = {}

```

Inverz dobimo tako, da zamenjamo ključe in vrednosti:

```

1 def inverz(sifra):
2     if not je_sifra(sifra):
3         return None
4     inv = {}
5     for k, v in sifra.items():
6         inv[v] = k
7     return inv

```

Končna koda.

```

1 def inverz(sifra):
2     if not je_sifra(sifra):
3         return None
4     inv = {}
5     for k, v in sifra.items():
6         inv[v] = k
7     return inv

```

6.2.4 4. podnalog: odsifriraj

Cilj. `odsifriraj(sifra, beseda)` naj odšifrira besedo. Če šifra ni bijekcija, naj vrne `None`.

Odšifriranje je šifriranje z inverzno šifro. Najprej jo dobimo:

```

1 def odsifriraj(sifra, beseda):
2     inv = inverz(sifra)

```

Če inverz obstaja, z njim zašifriramo besedo, sicer vrnemo `None`:

```

1 def odsifriraj(sifra, beseda):
2     inv = inverz(sifra)
3     if inv is not None:
4         return sifriraj(inv, beseda)
5     else:
6         return None

```

Končna koda.

```

1 def odsifriraj(sifra, beseda):
2     inv = inverz(sifra)
3     if inv is not None:
4         return sifriraj(inv, beseda)
5     else:
6         return None

```

6.3 Permutacije

Permutacijo naravnih števil $1, \dots, n$ predstavimo s slovarjem, npr. $\{1: 3, 2: 2, 3: 1\}$.

6.3.1 1. podnalog: slika

Cilj. `slika(permutacija, x)` naj vrne sliko števila x .

Slika je preprosto vrednost, shranjena pod ključem x :

```

1 def slika(permutacija, x):
2     return permutacija[x]

```

Končna koda.

```

1 def slika(permutacija, x):
2     return permutacija[x]

```

6.3.2 2. podnalog: slike

Cilj. `slike(permutacija, x, n)` naj vrne zaporedje $n + 1$ slik, če začnemo s x in n -krat uporabimo permutacijo.

Pripravimo prazen seznam:

```

1 def slike(permutacija, x, n):
2     r = []

```

Zaporedje ima $n + 1$ členov (začetni x in n slik):

```

1 def slike(permutacija, x, n):
2     r = []
3     for i in range(n + 1):

```

Vsak člen dodamo in nato x nadomestimo z njegovo sliko:

```

1 def slike(permutacija, x, n):
2     r = []
3     for i in range(n + 1):
4         r.append(x)
5         x = permutacija[x]
6     return r

```

Končna koda.

```

1 def slike(permutacija, x, n):
2     r = []
3     for i in range(n + 1):
4         r.append(x)
5         x = permutacija[x]
6     return r

```

6.3.3 3. podnalog: cikel

Cilj. `cikel(permutacija, x)` naj vrne celoten cikel, ki se začne s `x`.

Cikel začnemo z `x`:

```
1 def cikel(permutacija, x):
2     cikel = [x]
```

Nato sledimo slikam, dokler se ne vrnemo na začetek:

```
1 def cikel(permutacija, x):
2     cikel = [x]
3     y = permutacija[x]
```

Dokler `y` ni spet `x`, ga dodajamo in sledimo naprej:

```
1 def cikel(permutacija, x):
2     cikel = [x]
3     y = permutacija[x]
4     while y != x:
5         cikel.append(y)
6         y = permutacija[y]
7     return cikel
```

Končna koda.

```
1 def cikel(permutacija, x):
2     cikel = [x]
3     y = permutacija[x]
4     while y != x:
5         cikel.append(y)
6         y = permutacija[y]
7     return cikel
```

6.3.4 4. podnalog: cikli

Cilj. `cikli(permutacija)` naj vrne vse disjunktne cikle, urejene po začetnem elementu.

Pripravimo seznam ciklov in množico že obdelanih števil:

```
1 def cikli(permutacija):
2     cikli = []
3     ugotovljena = set()
```

Pojdimo po vseh številih od 1 do `n`:

```
1 def cikli(permutacija):
2     cikli = []
3     ugotovljena = set()
4     for i in range(1, len(permutacija) + 1):
```

Če `i` še ni v nobenem ciklu, je najmanjši v svojem ciklu (ker gremo po vrsti). Zato izračunamo njegov cikel in ga dodamo:

```
1 def cikli(permutacija):
2     cikli = []
3     ugotovljena = set()
4     for i in range(1, len(permutacija) + 1):
5         if i not in ugotovljena:
6             c = cikel(permutacija, i)
7             cikli.append(c)
```

Vsa števila iz cikla označimo kot obdelana in vrnemo seznam:

```

1 def cikli(permutacija):
2     cikli = []
3     ugotovljena = set()
4     for i in range(1, len(permutacija) + 1):
5         if i not in ugotovljena:
6             c = cikel(permutacija, i)
7             cikli.append(c)
8             ugotovljena.update(c)
9     return cikli

```

Končna koda.

```

1 def cikli(permutacija):
2     cikli = []
3     ugotovljena = set()
4     for i in range(1, len(permutacija) + 1):
5         if i not in ugotovljena:
6             c = cikel(permutacija, i)
7             cikli.append(c)
8             ugotovljena.update(c)
9     return cikli

```

6.3.5 5. podnalog: je permutacija

Cilj. `je_permutacija(slovar)` naj vrne `True`, če slovar predstavlja permutacijo števil $1, \dots, n$. Pripravimo dva seznama oznak dolžine n , za domeno in za sliko:

```

1 def je_permutacija(slovar):
2     n = len(slovar)
3     je_v_domeni = [False for i in range(n)]
4     je_v_sliki = [False for i in range(n)]

```

Preglejmo vse pare. Če je kateri ključ ali vrednost zunaj obsega $1, \dots, n$, to ni permutacija:

```

1 def je_permutacija(slovar):
2     n = len(slovar)
3     je_v_domeni = [False for i in range(n)]
4     je_v_sliki = [False for i in range(n)]
5     for k, v in slovar.items():
6         if 1 <= k <= n and 1 <= v <= n:
7             je_v_domeni[k - 1] = True
8             je_v_sliki[v - 1] = True
9     else:
10        return False

```

Označimo prisotnost in na koncu preverimo, da sta oba seznama sama `True` (funkcija `all`):

```

1 def je_permutacija(slovar):
2     n = len(slovar)
3     je_v_domeni = [False for i in range(n)]
4     je_v_sliki = [False for i in range(n)]
5     for k, v in slovar.items():
6         if 1 <= k <= n and 1 <= v <= n:
7             je_v_domeni[k - 1] = True
8             je_v_sliki[v - 1] = True
9     else:
10        return False
11    return all(je_v_domeni) and all(je_v_sliki)

```

Zakaj deluje

Slovar predstavlja permutacijo natanko tedaj, ko je vsako število od 1 do n hkrati v domeni in v sliki. Zato preverimo, da so vse oznake resnične.

Končna koda.

```

1 def je_permutacija(slovar):
2     n = len(slovar)
3     je_v_domeni = [False for i in range(n)]
4     je_v_sliki = [False for i in range(n)]
5     for k, v in slovar.items():
6         if 1 <= k <= n and 1 <= v <= n:
7             je_v_domeni[k - 1] = True
8             je_v_sliki[v - 1] = True
9         else:
10            return False
11    return all(je_v_domeni) and all(je_v_sliki)

```

6.4 Kalorije

6.4.1 1. podnalog: vrni kalorije

Cilj. `vrni_kalorije(niz)` naj niz s števili, ločenimi z vejicami, pretvori v seznam celih števil.

Niz razbijemo po vejicah z `.split(',')`:

```

1 def vrni_kalorije(niz):
2     l = niz.split(',')

```

Pripravimo prazen seznam za števila:

```

1 def vrni_kalorije(niz):
2     l = niz.split(',')
3     l2 = []

```

Vsak del pretvorimo v celo število in dodamo:

```

1 def vrni_kalorije(niz):
2     l = niz.split(',')
3     l2 = []
4     for x in l:
5         l2.append(int(x))
6     return l2

```

Konzola

```

>>> vrni_kalorije('10,4,7')
[10, 4, 7]

```

Končna koda.

```

1 def vrni_kalorije(niz):
2     l = niz.split(',')
3     l2 = []
4     for x in l:
5         l2.append(int(x))
6     return l2

```

6.4.2 2. podnalog: kalorije na dan

Cilj. `kalorije_na_dan(ime_datoteke)` naj za vsak dan vrne vsoto kalorij.

Pripravimo seznam vsot:

```
1 def kalorije_na_dan(ime_datoteke):
2     dnevne_vsote = []
```

Odpremo datoteko in se sprehodimo po vrsticah:

```
1 def kalorije_na_dan(ime_datoteke):
2     dnevne_vsote = []
3     with open(ime_datoteke) as f:
4         for vrstica in f:
```

Vrstico očistimo (`.strip()` odstrani novo vrstico), pretvorimo v seznam kalorij in seštejemo:

```
1 def kalorije_na_dan(ime_datoteke):
2     dnevne_vsote = []
3     with open(ime_datoteke) as f:
4         for vrstica in f:
5             kalorije = vrni_kalorije(vrstica.strip())
6             dnevne_vsote.append(sum(kalorije))
7     return dnevne_vsote
```

Končna koda.

```
1 def kalorije_na_dan(ime_datoteke):
2     dnevne_vsote = []
3     with open(ime_datoteke) as f:
4         for vrstica in f:
5             kalorije = vrni_kalorije(vrstica.strip())
6             dnevne_vsote.append(sum(kalorije))
7     return dnevne_vsote
```

6.4.3 3. podnalog: vsota kalorij

Cilj. `vsota_kalorij(ime_vhodne, ime_izhodne)` naj vsote zapiše v izhodno datoteko.

Dnevne vsote dobimo kar s prejšnjo funkcijo:

```
1 def vsota_kalorij(ime_vhodne, ime_izhodne):
2     dnevne_vsote = kalorije_na_dan(ime_vhodne)
```

Odpremo izhodno datoteko in vsako vsoto izpišemo v svojo vrstico:

```
1 def vsota_kalorij(ime_vhodne, ime_izhodne):
2     dnevne_vsote = kalorije_na_dan(ime_vhodne)
3     with open(ime_izhodne, 'w') as f:
4         for x in dnevne_vsote:
5             print(x, file=f)
```

Končna koda.

```
1 def vsota_kalorij(ime_vhodne, ime_izhodne):
2     dnevne_vsote = kalorije_na_dan(ime_vhodne)
3     with open(ime_izhodne, 'w') as f:
4         for x in dnevne_vsote:
5             print(x, file=f)
```

6.4.4 4. podnalog: povprečje kalorij

Cilj. `povprecje_kalorij(ime_vhodne, ime_izhodne)` naj za vsak dan zapiše zaporedno številko in povprečje obrokov na dve decimalki, v zadnjo vrstico pa skupno dnevno povprečje.

Pripravimo skupno vsoto in števec dni:

```
1 def povprecje_kalorij(ime_vhodne, ime_izhodne):
2     vsota_kalorij = 0
3     stevilo_dni = 0
```

Odpremo obe datoteki in se sprehodimo po vrsticah:

```
1 def povprecje_kalorij(ime_vhodne, ime_izhodne):
2     vsota_kalorij = 0
3     stevilo_dni = 0
4     with open(ime_vhodne) as f:
5         with open(ime_izhodne, 'w') as g:
6             for vrstica in f:
```

Za vsak dan izračunamo dnevni vnos in posodobimo skupne vsote:

```
1 def povprecje_kalorij(ime_vhodne, ime_izhodne):
2     vsota_kalorij = 0
3     stevilo_dni = 0
4     with open(ime_vhodne) as f:
5         with open(ime_izhodne, 'w') as g:
6             for vrstica in f:
7                 obroki = vrni_kalorije(vrstica)
8                 dnevni_vnos = sum(obroki)
9                 stevilo_dni += 1
10                vsota_kalorij += dnevni_vnos
```

V izhodno datoteko zapišemo povprečje dneva; `f'...:.2f'` zaokroži na dve decimalki:

```
1 def povprecje_kalorij(ime_vhodne, ime_izhodne):
2     vsota_kalorij = 0
3     stevilo_dni = 0
4     with open(ime_vhodne) as f:
5         with open(ime_izhodne, 'w') as g:
6             for vrstica in f:
7                 obroki = vrni_kalorije(vrstica)
8                 dnevni_vnos = sum(obroki)
9                 stevilo_dni += 1
10                vsota_kalorij += dnevni_vnos
11                print(f'{stevilo_dni} {dnevni_vnos / len(obroki):.2f}', file=g)
```

Na koncu (zunaj zanke, a znotraj datoteke) zapišemo še skupno povprečje:

```
1 def povprecje_kalorij(ime_vhodne, ime_izhodne):
2     vsota_kalorij = 0
3     stevilo_dni = 0
4     with open(ime_vhodne) as f:
5         with open(ime_izhodne, 'w') as g:
6             for vrstica in f:
7                 obroki = vrni_kalorije(vrstica)
8                 dnevni_vnos = sum(obroki)
9                 stevilo_dni += 1
10                vsota_kalorij += dnevni_vnos
11                print(f'{stevilo_dni} {dnevni_vnos / len(obroki):.2f}', file=g)
12                print(f'{vsota_kalorij / stevilo_dni:.2f}', file=g)
```

Novo v tem razdelku

F-nizi. `f'{x}'` vstavi vrednost `x` v niz. Z dvopičjem in obliko določimo obliko: `f'{x:.2f}'` izpiše `x` z dvema decimalnima.

Končna koda.

```

1 def povprecje_kalorij(ime_vhodne, ime_izhodne):
2     vsota_kalorij = 0
3     stevilo_dni = 0
4     with open(ime_vhodne) as f:
5         with open(ime_izhodne, 'w') as g:
6             for vrstica in f:
7                 obroki = vrni_kalorije(vrstica)
8                 dnevni_vnos = sum(obroki)
9                 stevilo_dni += 1
10                vsota_kalorij += dnevni_vnos
11                print(f'{stevilo_dni} {dnevni_vnos / len(obroki):.2f}', file=g)
12                print(f'{vsota_kalorij / stevilo_dni:.2f}', file=g)

```

6.5 Povzetek

- `d[kljuc] = v` doda par, `d.get(k, 0)` pa varno prebere;
- **izpeljan slovar** `{k: v for ...}` zgradi slovar v eni vrstici;
- `set(d.keys()) == set(d.values())` preveri bijekcijo;
- permutacije obravnavamo s sledenjem slikam (cikli);
- `str.split` in `str.join` pretvarjata med nizom in seznamom;
- **f-nizi** `f'{x:.2f}'` oblikujejo izpis.

Preveri se

1. Zakaj je `je_sifra` primerja množici ključev in vrednosti?
2. Kako cikli poskrbi, da se cikli ne ponavljajo?
3. Kaj naredi `shramba.get(sestavina, 0)`, če sestavine v shrambi ni?

Poglavje 7

Slovarji, grafi in hierarhije

V tem poglavju bomo gradili in preoblikovali slovarje: rimsko številstvo, kemijske formule, usmerjene grafe, recepte in hierarhijo zaposlenih.

7.1 Rimski imperij vrača udarec

7.1.1 1. podnaloga: v rimsko

Cilj. `v_rimsko(stevilo)` naj vrne rimsko številko za število med 1 in 3999.

Pripravimo si tabelo rimskih znakov po mestih (enice, desetice, stotice, tisočice):

```
1 tabela_rimskih = [  
2     ['', 'I', 'II', 'III', 'IV', 'V', 'VI', 'VII', 'VIII', 'IX'],  
3     ['', 'X', 'XX', 'XXX', 'XL', 'L', 'LX', 'LXX', 'LXXX', 'XC'],  
4     ['', 'C', 'CC', 'CCC', 'CD', 'D', 'DC', 'DCC', 'DCCC', 'CM'],  
5     ['', 'M', 'MM', 'MMM']  
6 ]
```

Rezultat začnemo s praznim nizom:

```
1 def v_rimsko(stevilo):  
2     rimsko = ''
```

Za vsako od štirih mest vzamemo ustrezní znak in ga dodamo *spredaj* (ker začnemo z enicami, ki so na desni):

```
1 def v_rimsko(stevilo):  
2     rimsko = ''  
3     for i in range(4):  
4         rimsko = tabela_rimskih[i][stevilo % 10] + rimsko
```

Nato število delimo z 10 in nadaljujemo z naslednjim mestom:

```
1 def v_rimsko(stevilo):  
2     rimsko = ''  
3     for i in range(4):  
4         rimsko = tabela_rimskih[i][stevilo % 10] + rimsko  
5         stevilo //= 10  
6     return rimsko
```

Zakaj deluje

Zadnja številka (`stevilo % 10`) določi enice, po deljenju z 10 pa predzadnja določi desetice itd. Ker nove znake dodajamo spredaj, dobimo pravilen vrstni red od tisočic do

enic.

Končna koda.

```

1 tabela_rimskih = [
2     ['', 'I', 'II', 'III', 'IV', 'V', 'VI', 'VII', 'VIII', 'IX'],
3     ['', 'X', 'XX', 'XXX', 'XL', 'L', 'LX', 'LXX', 'LXXX', 'XC'],
4     ['', 'C', 'CC', 'CCC', 'CD', 'D', 'DC', 'DCC', 'DCCC', 'CM'],
5     ['', 'M', 'MM', 'MMM']
6 ]
7
8 def v_rimsko(stevilo):
9     rimsko = ''
10    for i in range(4):
11        rimsko = tabela_rimskih[i][stevilo % 10] + rimsko
12        stevilo //= 10
13    return rimsko

```

7.1.2 2. podnalog: v arabsko

Cilj. `v_arabsko(niz)` naj rimsko številko pretvori nazaj v število ali vrne `None`, če ni veljavna.

Sestavimo slovar vseh veljavnih rimskih števil. Ker znamo pretvarjati v eno smer, ga zgradimo z izpeljanim slovarjem:

```
1 slovar_rimskih = {v_rimsko(i): i for i in range(1, 4000)}
```

Pretvorba je nato le iskanje v slovarju. Če niza ni v slovarju, ni veljaven:

```

1 def v_arabsko(niz):
2     if niz not in slovar_rimskih:
3         return None
4     return slovar_rimskih[niz]

```

Novo v tem razdelku

Izpeljan slovar z pogojem. `{f(i): i for i in range(1, 4000)}` zgradi slovar vseh parov. Ker je preslikava bijektivna, lahko z njim iščemo v obe smeri.

Končna koda.

```

1 slovar_rimskih = {v_rimsko(i): i for i in range(1, 4000)}
2
3 def v_arabsko(niz):
4     if niz not in slovar_rimskih:
5         return None
6     return slovar_rimskih[niz]

```

7.2 Mladi kemik**7.2.1 1. podnalog: spojina**

Cilj. `spojina(niz)` naj niz oblike `'H2 O'` pretvori v slovar `{'H': 2, 'O': 1}`.

Najprej napišimo pomožno funkcijo, ki en element oblike `'C16'` razbije na oznako in število. Pripravimo dva niza, za črke in za števke:

```

1 def element_stevilo(e):
2     a = ''
3     n = ''

```

Sprehodimo se po znakih: števke gredo v n, črke v a:

```

1 def element_stevilo(e):
2     a = ''
3     n = ''
4     for c in e:
5         if c.isdigit():
6             n += c
7         else:
8             a += c

```

Če števila ni, velja 1; sicer ga pretvorimo v celo število in vrnemo par:

```

1 def element_stevilo(e):
2     a = ''
3     n = ''
4     for c in e:
5         if c.isdigit():
6             n += c
7         else:
8             a += c
9     n = 1 if n == '' else int(n)
10    return a, n

```

Novo v tem razdelku

`c.isdigit()`. Vrne True, če je znak c števka.

Zdaj spojina. Pripravimo prazen slovar in niz razbijemo po presledkih:

```

1 def spojina(niz):
2     s = dict()
3     elementi = niz.split()

```

Vsak element razbijemo in njegovo število *prištejemo* k obstoječemu (če se element ponovi):

```

1 def spojina(niz):
2     s = dict()
3     elementi = niz.split()
4     for e in elementi:
5         a, n = element_stevilo(e)
6         s[a] = s.get(a, 0) + n
7     return s

```

Končna koda.

```

1 def element_stevilo(e):
2     a = ''
3     n = ''
4     for c in e:
5         if c.isdigit():
6             n += c
7         else:
8             a += c
9     n = 1 if n == '' else int(n)

```

```

10     return a, n
11
12 def spojina(niz):
13     s = dict()
14     elementi = niz.split()
15     for e in elementi:
16         a, n = element_stevilo(e)
17         s[a] = s.get(a, 0) + n
18     return s

```

7.2.2 2. podnaloga: izpis

Cilj. izpis(spojina) naj slovar pretvori nazaj v niz. Elementi naj bodo urejeni leksikografsko, enic ne pišemo.

Atome uredimo po abecedi:

```

1 def izpis(spojina):
2     atomi = sorted(spojina.keys())

```

Pripravimo prazen niz:

```

1 def izpis(spojina):
2     atomi = sorted(spojina.keys())
3     s = ''

```

Sprehodimo se po atomih. Vsakega razen prvega ločimo s presledkom:

```

1 def izpis(spojina):
2     atomi = sorted(spojina.keys())
3     s = ''
4     for a in atomi:
5         if s != '':
6             s += ' '
7         s += a

```

Število dodamo le, če je večje od 1:

```

1 def izpis(spojina):
2     atomi = sorted(spojina.keys())
3     s = ''
4     for a in atomi:
5         if s != '':
6             s += ' '
7         s += a
8         if spojina[a] > 1:
9             s += str(spojina[a])
10    return s

```

Končna koda.

```

1 def izpis(spojina):
2     atomi = sorted(spojina.keys())
3     s = ''
4     for a in atomi:
5         if s != '':
6             s += ' '
7         s += a
8         if spojina[a] > 1:
9             s += str(spojina[a])
10    return s

```

7.2.3 3. podnalog: uravnoteži

Cilj. `uravnoteži(prva, druga)` naj vrne manjkajoče atome na vsaki strani, da bo reakcija uravnotežena.

Pripravimo dva prazna slovarja:

```
1 def uravnoteži(prva, druga):
2     dod_prva, dod_druga = dict(), dict()
```

Za vsak element prve strani pogledamo razliko do druge. Manjkajoče damo na tisto stran, kjer jih primanjkuje:

```
1 def uravnoteži(prva, druga):
2     dod_prva, dod_druga = dict(), dict()
3     for k, v in prva.items():
4         z = v - druga.get(k, 0)
5         if z < 0:
6             dod_prva[k] = -z
7         elif z > 0:
8             dod_druga[k] = z
```

Nato dodamo še elemente, ki jih ima samo druga stran:

```
1 def uravnoteži(prva, druga):
2     dod_prva, dod_druga = dict(), dict()
3     for k, v in prva.items():
4         z = v - druga.get(k, 0)
5         if z < 0:
6             dod_prva[k] = -z
7         elif z > 0:
8             dod_druga[k] = z
9     for k, v in druga.items():
10        if (k not in prva) and (v != 0):
11            dod_prva[k] = v
12    return dod_prva, dod_druga
```

Končna koda.

```
1 def uravnoteži(prva, druga):
2     dod_prva, dod_druga = dict(), dict()
3     for k, v in prva.items():
4         z = v - druga.get(k, 0)
5         if z < 0:
6             dod_prva[k] = -z
7         elif z > 0:
8             dod_druga[k] = z
9     for k, v in druga.items():
10        if (k not in prva) and (v != 0):
11            dod_prva[k] = v
12    return dod_prva, dod_druga
```

7.2.4 4. podnalog: koliko modelov

Cilj. `koliko_modelov(spojina, zaloga)` naj vrne, koliko modelov spojine lahko sestavimo iz zaloge.

Začnemo z neskončno velikim številom modelov:

```
1 def koliko_modelov(spojina, zaloga):
2     n = float('inf')
```

Za vsak element izračunamo, koliko modelov zanj dopušča zaloga, in si zapomnimo najmanjšo omejitev:

```

1 def koliko_modelov(spojina, zaloga):
2     n = float('inf')
3     for k, v in spojina.items():
4         if v == 0:
5             continue
6         z = zaloga.get(k, 0)
7         if z // v < n:
8             n = z // v

```

Če je n že nič, lahko takoj nehamo, saj nič ne moremo izboljšati:

```

1 def koliko_modelov(spojina, zaloga):
2     n = float('inf')
3     for k, v in spojina.items():
4         if v == 0:
5             continue
6         z = zaloga.get(k, 0)
7         if z // v < n:
8             n = z // v
9         if n == 0:
10            break
11    return n

```

Zakaj deluje

Število modelov omeji najbolj redka sestavina, zato vzamemo najmanjši kvocient $z // v$ po vseh elementih. `float('inf')` je začetna zgornja meja.

Končna koda.

```

1 def koliko_modelov(spojina, zaloga):
2     n = float('inf')
3     for k, v in spojina.items():
4         if v == 0:
5             continue
6         z = zaloga.get(k, 0)
7         if z // v < n:
8             n = z // v
9         if n == 0:
10            break
11    return n

```

7.3 Usmerjeni grafi

7.3.1 1. podnalog: slovar naslednikov

Cilj. `slovar_naslednikov(seznam_povezav)` naj iz seznamu povezav sestavi slovar naslednikov (sezname uredi).

Pripravimo prazen slovar:

```

1 def slovar_naslednikov(seznam_povezav):
2     slovar = dict()

```

Za vsako povezavo (u , v) poskrbimo, da ima u svoj seznam, in dodamo v :

```

1 def slovar_naslednikov(seznam_povezav):
2     slovar = dict()
3     for u, v in seznam_povezav:
4         if u not in slovar:
5             slovar[u] = []
6             slovar[u].append(v)

```

Na koncu vsak seznam uredimo:

```

1 def slovar_naslednikov(seznam_povezav):
2     slovar = dict()
3     for u, v in seznam_povezav:
4         if u not in slovar:
5             slovar[u] = []
6             slovar[u].append(v)
7     for u in slovar:
8         slovar[u].sort()
9     return slovar

```

Končna koda.

```

1 def slovar_naslednikov(seznam_povezav):
2     slovar = dict()
3     for u, v in seznam_povezav:
4         if u not in slovar:
5             slovar[u] = []
6             slovar[u].append(v)
7     for u in slovar:
8         slovar[u].sort()
9     return slovar

```

7.3.2 2. podnalog: seznam povezav

Cilj. `seznam_povezav(digraf)` naj naredi obratno: iz slovarja naslednikov sestavi urejen seznam povezav.

Pripravimo prazen seznam:

```

1 def seznam_povezav(digraf):
2     sez_pov = []

```

Za vsako vozlišče `u` in vsakega njegovega naslednika `v` dodamo povezavo:

```

1 def seznam_povezav(digraf):
2     sez_pov = []
3     for u, nasl in digraf.items():
4         for v in nasl:
5             sez_pov.append((u, v))

```

Seznam uredimo in vrnemo:

```

1 def seznam_povezav(digraf):
2     sez_pov = []
3     for u, nasl in digraf.items():
4         for v in nasl:
5             sez_pov.append((u, v))
6     sez_pov.sort()
7     return sez_pov

```

Novo v tem razdelku

Sortiranje terkah. `sez_pov.sort()` uredi pare (u, v) najprej po u , ob izenačenju pa po v .

Končna koda.

```

1 def seznam_povezav(digraf):
2     sez_pov = []
3     for u, nasl in digraf.items():
4         for v in nasl:
5             sez_pov.append((u, v))
6     sez_pov.sort()
7     return sez_pov

```

7.3.3 3. podnalog: nasprotni graf

Cilj. `nasprotni_graf(digraf)` naj vrne graf z obrnjenimi povezavami.

Pripravimo prazen slovar:

```

1 def nasprotni_graf(digraf):
2     nasprotni = dict()

```

Za vsako povezavo $u \rightarrow v$ v nasprotnem grafu velja $v \rightarrow u$:

```

1 def nasprotni_graf(digraf):
2     nasprotni = dict()
3     for u, nasl in digraf.items():
4         for v in nasl:
5             if v not in nasprotni:
6                 nasprotni[v] = []
7                 nasprotni[v].append(u)

```

Sezname uredimo in vrnemo graf:

```

1 def nasprotni_graf(digraf):
2     nasprotni = dict()
3     for u, nasl in digraf.items():
4         for v in nasl:
5             if v not in nasprotni:
6                 nasprotni[v] = []
7                 nasprotni[v].append(u)
8     for u in nasprotni:
9         nasprotni[u].sort()
10    return nasprotni

```

Končna koda.

```

1 def nasprotni_graf(digraf):
2     nasprotni = dict()
3     for u, nasl in digraf.items():
4         for v in nasl:
5             if v not in nasprotni:
6                 nasprotni[v] = []
7                 nasprotni[v].append(u)
8     for u in nasprotni:
9         nasprotni[u].sort()
10    return nasprotni

```

7.4 Gensko spremenjena hrana

7.4.1 1. podnalog: odpravi okrajšave

Cilj. `odpravi_okrajsave(recepti)` naj pri jedeh s praznim slovarjem sestavin doda jed samo sebe kot edino sestavino.

Pripravimo nov slovar:

```
1 def odpravi_okrajsave(recepti):
2     brez_okrajsav = dict()
```

Sprehodimo se po jedeh in njihovih sestavinah:

```
1 def odpravi_okrajsave(recepti):
2     brez_okrajsav = dict()
3     for jed, sestavine in recepti.items():
```

Če je sestavin prazna množica, je jed sama sebi sestavina (z količino 1):

```
1 def odpravi_okrajsave(recepti):
2     brez_okrajsav = dict()
3     for jed, sestavine in recepti.items():
4         if len(sestavine) == 0:
5             brez_okrajsav[jed] = {jed: 1}
6         else:
7             brez_okrajsav[jed] = sestavine
8     return brez_okrajsav
```

Končna koda.

```
1 def odpravi_okrajsave(recepti):
2     brez_okrajsav = dict()
3     for jed, sestavine in recepti.items():
4         if len(sestavine) == 0:
5             brez_okrajsav[jed] = {jed: 1}
6         else:
7             brez_okrajsav[jed] = sestavine
8     return brez_okrajsav
```

7.4.2 2. podnalog: normaliziraj količine

Cilj. `normaliziraj_kolicine(recepti)` naj vsak recept pretvori tako, da bo vsota količin 100.

Najprej odpravimo okrajšave:

```
1 def normaliziraj_kolicine(recepti):
2     brez_okrajsav = odpravi_okrajsave(recepti)
```

Za vsak recept izračunamo vsoto količin:

```
1 def normaliziraj_kolicine(recepti):
2     brez_okrajsav = odpravi_okrajsave(recepti)
3     for jed, sestavine in brez_okrajsav.items():
4         vsota = sum(sestavine.values())
```

Vsako količino pretvorimo v odstotek z izpeljanim slovarjem in vrnemo:

```
1 def normaliziraj_kolicine(recepti):
2     brez_okrajsav = odpravi_okrajsave(recepti)
3     for jed, sestavine in brez_okrajsav.items():
```

```

4     vsota = sum(sestavine.values())
5     brez_okrajsav[jed] = {k: 100 * v / vsota for k, v in sestavine.items()}
6     return brez_okrajsav

```

Novo v tem razdelku

`dict.values(). sestavine.values()` vrne vse količine; `sum(...)` jih sešteje.

Končna koda.

```

1 def normaliziraj_kolicine(recepti):
2     brez_okrajsav = odpravi_okrajsave(recepti)
3     for jed, sestavine in brez_okrajsav.items():
4         vsota = sum(sestavine.values())
5         brez_okrajsav[jed] = {k: 100 * v / vsota for k, v in sestavine.items()}
6     return brez_okrajsav

```

7.4.3 3. podnalog: delež GSO

Cilj. `delez_gso(recepti, elementarne)` naj za vsako jed izračuna delež GSO.

Recepte najprej normaliziramo:

```

1 def delez_gso(recepti, elementarne):
2     norm_recepti = normaliziraj_kolicine(recepti)

```

Pripravimo prazen slovar:

```

1 def delez_gso(recepti, elementarne):
2     norm_recepti = normaliziraj_kolicine(recepti)
3     gso = dict()

```

Delež jedi je vsota deležev sestavin, uteženih z njihovimi odstotki (zato delimo s 100):

```

1 def delez_gso(recepti, elementarne):
2     norm_recepti = normaliziraj_kolicine(recepti)
3     gso = dict()
4     for jed, sestavine in norm_recepti.items():
5         gso_jedi = sum(v * elementarne[k] for k, v in sestavine.items()) / 100.0
6         gso[jed] = gso_jedi
7     return gso

```

Končna koda.

```

1 def delez_gso(recepti, elementarne):
2     norm_recepti = normaliziraj_kolicine(recepti)
3     gso = dict()
4     for jed, sestavine in norm_recepti.items():
5         gso_jedi = sum(v * elementarne[k] for k, v in sestavine.items()) / 100.0
6         gso[jed] = gso_jedi
7     return gso

```

7.4.4 4. podnalog: delež GSO special

Cilj. `delez_gso_special(recepti, elementarne)` naj vrne interval (min, max) GSO za vsako jed. Za neznane sestavine vzamemo 0% oz. 100%.

Recepte normaliziramo in pripravimo slovar:

```

1 def delez_gso_special(recepti, elementarne):
2     norm_recepti = normaliziraj_kolicine(recepti)
3     gso = dict()

```

Za neznano sestavino `.get(k, 0)` vrne 0 (minimum), `.get(k, 100)` pa 100 (maksimum):

```

1 def delez_gso_special(recepti, elementarne):
2     norm_recepti = normaliziraj_kolicine(recepti)
3     gso = dict()
4     for jed, sestavine in norm_recepti.items():
5         min_gso = sum(v * elementarne.get(k, 0) for k, v in sestavine.items()) /
6             100.0
7         max_gso = sum(v * elementarne.get(k, 100) for k, v in sestavine.items()) /
8             100.0
9         gso[jed] = (min_gso, max_gso)
10    return gso

```

Končna koda.

```

1 def delez_gso_special(recepti, elementarne):
2     norm_recepti = normaliziraj_kolicine(recepti)
3     gso = dict()
4     for jed, sestavine in norm_recepti.items():
5         min_gso = sum(v * elementarne.get(k, 0) for k, v in sestavine.items()) /
6             100.0
7         max_gso = sum(v * elementarne.get(k, 100) for k, v in sestavine.items()) /
8             100.0
9         gso[jed] = (min_gso, max_gso)
10    return gso

```

7.5 Veliki šef in njegovi podrejeni

7.5.1 1. podnalog: slovar šefov

Cilj. `slovar_sefov(seznam)` naj iz seznama parov (`usluzbenec`, `sef`) sestavi slovar šefov.

Pripravimo prazen slovar in se sprehodimo po parih:

```

1 def slovar_sefov(seznam):
2     slovar = {}
3     for usluzbenec, sef in seznam:

```

Vsakemu uslužbencu priredimo njegovega šefa:

```

1 def slovar_sefov(seznam):
2     slovar = {}
3     for usluzbenec, sef in seznam:
4         slovar[usluzbenec] = sef
5     return slovar

```

Končna koda.

```

1 def slovar_sefov(seznam):
2     slovar = {}
3     for usluzbenec, sef in seznam:
4         slovar[usluzbenec] = sef
5     return slovar

```

7.5.2 2. podnalog: neposredno podrejeni

Cilj. `neposredno_podrejeni(seznam)` naj vrne slovar, kjer vsakemu šefu pripada množica neposrednih podrejenih.

Pripravimo prazen slovar:

```
1 def neposredno_podrejeni(seznam):
2     podrejeni = {}
```

Za vsak par poskrbimo, da ima šef svojo množico, in dodamo uslužbenca:

```
1 def neposredno_podrejeni(seznam):
2     podrejeni = {}
3     for uslužbenec, sef in seznam:
4         if sef not in podrejeni:
5             podrejeni[sef] = set()
6             podrejeni[sef].add(uslužbenec)
7     return podrejeni
```

Končna koda.

```
1 def neposredno_podrejeni(seznam):
2     podrejeni = {}
3     for uslužbenec, sef in seznam:
4         if sef not in podrejeni:
5             podrejeni[sef] = set()
6             podrejeni[sef].add(uslužbenec)
7     return podrejeni
```

7.5.3 3. podnalog: veriga nadrejenih

Cilj. `veriga_nadrejenih(uslužbenec, slovar)` naj vrne seznam nadrejenih od neposrednega šefa navzgor.

Pripravimo prazen seznam:

```
1 def veriga_nadrejenih(uslužbenec, slovar):
2     veriga = []
```

Dokler ima trenutni uslužbenec šefa, se povzpnejo nanj in ga dodamo v verigo:

```
1 def veriga_nadrejenih(uslužbenec, slovar):
2     veriga = []
3     while uslužbenec in slovar:
4         uslužbenec = slovar[uslužbenec]
5         veriga.append(uslužbenec)
6     return veriga
```

Konzola

```
>>> veriga_nadrejenih('Mojca', {'Andrej': 'Tilen', 'Mojca': 'Tilen', 'Tilen': 'Zoran'})
['Tilen', 'Zoran']
```

Končna koda.

```
1 def veriga_nadrejenih(uslužbenec, slovar):
2     veriga = []
3     while uslužbenec in slovar:
```

```

4     uslužbenec = slovar[uslužbenec]
5     veriga.append(uslužbenec)
6     return veriga

```

7.5.4 4. podnalogi: množica podrejenih

Cilj. `mnozica_podrejenih(uslužbenec, slovar)` naj vrne vse (posredne in neposredne) podrejene.

Uporabili bomo **iskanje v širino**. Začnemo s seznamom, v katerem je izhodiščna oseba, in prazno množico:

```

1 def mnozica_podrejenih(uslužbenec, slovar):
2     dodaj = [uslužbenec]
3     podrejeni = set()

```

Dokler imamo kandidate, jih obdelujemo:

```

1 def mnozica_podrejenih(uslužbenec, slovar):
2     dodaj = [uslužbenec]
3     podrejeni = set()
4     while len(dodaj) > 0:

```

Vzamemo enega in, če ni v slovarju, nadaljujemo:

```

1 def mnozica_podrejenih(uslužbenec, slovar):
2     dodaj = [uslužbenec]
3     podrejeni = set()
4     while len(dodaj) > 0:
5         u = dodaj.pop()
6         if u not in slovar:
7             continue

```

Vsakega njegovega podrejenega dodamo med najdene in v čakalno vrsto:

```

1 def mnozica_podrejenih(uslužbenec, slovar):
2     dodaj = [uslužbenec]
3     podrejeni = set()
4     while len(dodaj) > 0:
5         u = dodaj.pop()
6         if u not in slovar:
7             continue
8         for v in slovar[u]:
9             if v not in podrejeni:
10                 dodaj.append(v)
11                 podrejeni.add(v)
12     return podrejeni

```

Zakaj deluje

Vsak podrejeni je dodan v čakalno vrsto natanko enkrat (preverimo `v not in podrejeni`), zato se postopek ustavi in obišče vsakogar dosegljivega.

Končna koda.

```

1 def mnozica_podrejenih(uslužbenec, slovar):
2     dodaj = [uslužbenec]
3     podrejeni = set()
4     while len(dodaj) > 0:

```

```

5     u = dodaj.pop()
6     if u not in slovar:
7         continue
8     for v in slovar[u]:
9         if v not in podrejeni:
10            dodaj.append(v)
11            podrejeni.add(v)
12     return podrejeni

```

7.5.5 5. podnalog: big boss

Cilj. `big_boss(slovar)` naj vrne osebo brez nadrejenih, ki je nadrejena vsem ostalim, ali `None`, če je ni.

Če slovar ni prazen, izberemo poljubnega uslužbenca in pogledamo vrh njegove verige nadrejenih:

```

1 def big_boss(slovar):
2     if len(slovar) == 0:
3         return None
4     uslužbenec = list(slovar)[0]
5     nadrejeni = veriga_nadrejenih(uslužbenec, slovar)
6     kandidat = nadrejeni[-1] if len(nadrejeni) > 0 else uslužbenec

```

Nato preverimo, ali je ta kandidat na vrhu verige *vsakogar*:

```

1 def big_boss(slovar):
2     if len(slovar) == 0:
3         return None
4     uslužbenec = list(slovar)[0]
5     nadrejeni = veriga_nadrejenih(uslužbenec, slovar)
6     kandidat = nadrejeni[-1] if len(nadrejeni) > 0 else uslužbenec
7     for u in slovar:
8         if u == kandidat:
9             continue
10            nadrejeni = veriga_nadrejenih(u, slovar)
11            if len(nadrejeni) == 0 or nadrejeni[-1] != kandidat:
12                return None
13     return kandidat

```

Zakaj deluje

Kandidat je nujno na vrhu verige izbrane osebe. Če je big boss, mora biti na vrhu verige prav vsakega zaposlenega; če kdaj ni, big boss ne obstaja.

Končna koda.

```

1 def big_boss(slovar):
2     if len(slovar) == 0:
3         return None
4     uslužbenec = list(slovar)[0]
5     nadrejeni = veriga_nadrejenih(uslužbenec, slovar)
6     kandidat = nadrejeni[-1] if len(nadrejeni) > 0 else uslužbenec
7     for u in slovar:
8         if u == kandidat:
9             continue
10            nadrejeni = veriga_nadrejenih(u, slovar)
11            if len(nadrejeni) == 0 or nadrejeni[-1] != kandidat:

```

```
12         return None
13     return kandidat
```

7.6 Povzetek

- slovar lahko zgradimo tudi *inverzno* (rimska števila);
- `str.split` in `str.isdigit` obdelujeta kemijske formule;
- usmerjene grafe pretvarjamo med slovarjem naslednikov in seznamom povezav;
- **iskanje v širino** (čakalna vrsta) poišče vse dosegljive podrejene;
- `float('inf')` je priročna začetna vrednost za minimum.

Preveri se

1. Zakaj `v_rimsko` dodaja znake spredaj in ne zadaj?
2. Kako `normaliziraj_kolicine` poskrbi, da je vsota 100?
3. Zakaj `mnozica_podrejenih` uporabi `set` in ne seznama?

Poglavje 8

Datoteke in besedilo

V tem poglavju bomo brali in pisali datoteke (tudi v kodni tabeli UTF-8), obdelovali besedilo in pretvarjali med besedilom ter HTML.

8.1 HTML datoteke

8.1.1 1. podnaloge: html2txt

Cilj. `html2txt(vhodna, izhodna)` naj iz vhodne datoteke odstrani vse značke in rezultat zapiše v izhodno. Značke se začnejo z `<` in končajo z `>`, lahko pa se raztezajo čez več vrstic.

Uporabili bomo **stikalo znacka**, ki pove, ali smo trenutno znotraj značke. Na začetku smo zunaj:

```
1 def html2txt(vhodna, izhodna):  
2     znacka = False
```

Odpremo obe datoteki v kodni tabeli UTF-8:

```
1 def html2txt(vhodna, izhodna):  
2     znacka = False  
3     with open(vhodna, encoding='utf-8') as html:  
4         with open(izhodna, 'w', encoding='utf-8') as txt:
```

Za vsako vrstico pripravimo prazen niz, v katerega zberemo besedilo:

```
1 def html2txt(vhodna, izhodna):  
2     znacka = False  
3     with open(vhodna, encoding='utf-8') as html:  
4         with open(izhodna, 'w', encoding='utf-8') as txt:  
5             for vrstica in html:  
6                 txt_vrstica = ''
```

Pregledamo vsak znak. Ob `<` ali `>` stikalo obrnemo; sicer znak dodamo, če smo zunaj značke:

```
1         for vrstica in html:  
2             txt_vrstica = ''  
3             for znak in vrstica:  
4                 if znak in '<>':  
5                     znacka = not znacka  
6                 elif not znacka:  
7                     txt_vrstica += znak
```

Na koncu vrstico zapišemo. Z `end=""` preprečimo dodatno novo vrstico, saj jo vrstica že vsebuje:

```
1 print(txt_vrstica, file=txt, end='')
```

Novo v tem razdelku

`print(..., end=")`. `print` privzeto doda novo vrstico; `end=""` to prepreči.

Končna koda.

```
1 def html2txt(vhodna, izhodna):
2     znacka = False
3     with open(vhodna, encoding='utf-8') as html:
4         with open(izhodna, 'w', encoding='utf-8') as txt:
5             for vrstica in html:
6                 txt_vrstica = ''
7                 for znak in vrstica:
8                     if znak in '<>':
9                         znacka = not znacka
10                    elif not znacka:
11                        txt_vrstica += znak
12                print(txt_vrstica, file=txt, end='')

```

8.1.2 2. podnalog: tabela

Cilj. tabela(ime_vhodne, ime_izhodne) naj vrstice s podatki, ločenimi z vejicami, pretvori v HTML tabelo.

Odpremo obe datoteki in izpišemo začetek tabele:

```
1 def tabela(ime_vhodne, ime_izhodne):
2     with open(ime_vhodne, encoding='utf-8') as vhodna:
3         with open(ime_izhodne, 'w', encoding='utf-8') as izhodna:
4             print('<table>', file=izhodna)

```

Za vsako vrstico odpremo vrstico tabele in podatke razbijemo po vejicah:

```
1     for vrstica in vhodna:
2         print(' <tr>', file=izhodna)
3         podatki = vrstica.strip().split(',')

```

Vsak podatek zapišemo v svojo celico `<td>`:

```
1         for podatek in podatki:
2             print(' <td>' + podatek + '</td>', file=izhodna)

```

Zapremo vrstico tabele in na koncu še celotno tabelo:

```
1         print(' </tr>', file=izhodna)
2     print('</table>', file=izhodna)

```

Končna koda.

```
1 def tabela(ime_vhodne, ime_izhodne):
2     with open(ime_vhodne, encoding='utf-8') as vhodna:
3         with open(ime_izhodne, 'w', encoding='utf-8') as izhodna:
4             print('<table>', file=izhodna)
5             for vrstica in vhodna:
6                 print(' <tr>', file=izhodna)
7                 podatki = vrstica.strip().split(',')
8                 for podatek in podatki:

```

```

9         print('    <td>' + podatek + '</td>', file=izhodna)
10        print(' </tr>', file=izhodna)
11        print('</table>', file=izhodna)

```

8.1.3 3. podnalog: seznam

Cilj. seznam(ime_vhodne, ime_izhodne) naj vrstice, ki se začnejo z *, pretvori v neurejen HTML seznam.

Spet uporabimo stikalo seznam, ki pove, ali smo znotraj seznama:

```

1 def seznam(ime_vhodne, ime_izhodne):
2     seznam = False

```

Odpremo datoteki in se sprehodimo po vrsticah:

```

1 def seznam(ime_vhodne, ime_izhodne):
2     seznam = False
3     with open(ime_vhodne, encoding='utf-8') as vhodna:
4         with open(ime_izhodne, 'w', encoding='utf-8') as izhodna:
5             for vrstica in vhodna:

```

Če se vrstica začne z zvezdico in seznamu še nismo odprli, ga odpremo; nato izpišemo element brez zvezdice:

```

1         if vrstica[0] == '*':
2             if not seznam:
3                 seznam = True
4                 print('<ul>', file=izhodna)
5                 print(' <li>' + vrstica[1:].strip() + '</li>', file=izhodna)

```

Če vrstica ni element seznama, morebitni odprti seznam zapremo in izpišemo vrstico:

```

1         else:
2             if seznam:
3                 seznam = False
4                 print('</ul>', file=izhodna)
5                 print(vrstica, file=izhodna, end='')

```

Na koncu zapremo seznam, če je še odprt:

```

1         if seznam:
2             print('</ul>', file=izhodna)

```

Končna koda.

```

1 def seznam(ime_vhodne, ime_izhodne):
2     seznam = False
3     with open(ime_vhodne, encoding='utf-8') as vhodna:
4         with open(ime_izhodne, 'w', encoding='utf-8') as izhodna:
5             for vrstica in vhodna:
6                 if vrstica[0] == '*':
7                     if not seznam:
8                         seznam = True
9                         print('<ul>', file=izhodna)
10                    print(' <li>' + vrstica[1:].strip() + '</li>', file=izhodna)
11                else:
12                    if seznam:
13                        seznam = False
14                        print('</ul>', file=izhodna)
15                    print(vrstica, file=izhodna, end='')

```

```

16         if seznam:
17             print('</ul>', file=izhodna)

```

8.1.4 4. podnalog: gnezdeni seznamami

Cilj. gnezdeni_seznam(ime_vhodne, ime_izhodne) naj iz zamika pred elementom ugotovi globino gnezdenja in zgradi gnezden HTML seznam.

Trenutna globina je na začetku nič, enota zamika pa dva presledka:

```

1 def gnezdeni_seznam(ime_vhodne, ime_izhodne):
2     nivo = 0
3     zamik = 2

```

Odpremo datoteki in se sprehodimo po vrsticah:

```

1 def gnezdeni_seznam(ime_vhodne, ime_izhodne):
2     nivo = 0
3     zamik = 2
4     with open(ime_vhodne, encoding='utf-8') as vhodna:
5         with open(ime_izhodne, 'w', encoding='utf-8') as izhodna:
6             for vrstica in vhodna:

```

Preštejemo vodilne presledke in iz njih izračunamo globino n:

```

1         prvi = 0
2         while vrstica[prvi] == ' ':
3             prvi += 1
4         n = prvi // zamik + 1
5         vrstica = vrstica.strip()

```

Če smo globlje kot prej, odpremo nov seznam :

```

1         if n > nivo:
2             print(2 * zamik * nivo * ' ' + '<ul>', file=izhodna)
3             nivo += 1

```

Če smo plitveje, zapremo toliko seznamov, kolikor nivojev se spustimo:

```

1         while n < nivo:
2             nivo -= 1
3             print(2 * zamik * nivo * ' ' + '</ul>', file=izhodna)

```

Izpišemo element (brez zapiranja, kot zahteva naloga):

```

1         print((2 * zamik * nivo - zamik) * ' ' + '<li>' + vrstica, file=
            izhodna)

```

Na koncu zapremo vse še odprte sezname:

```

1         while nivo > 0:
2             nivo -= 1
3             print(2 * zamik * nivo * ' ' + '</ul>', file=izhodna)

```

Zakaj deluje

Globino določa število vodilnih presledkov, deljeno z enoto zamika. Ko se globina poveča, odpremo nov ; ko se zmanjša, zapremo ustrezno število seznamov.

Končna koda.

```

1 def gnezdeni_seznami(ime_vhodne, ime_izhodne):
2     nivo = 0
3     zamik = 2
4     with open(ime_vhodne, encoding='utf-8') as vhodna:
5         with open(ime_izhodne, 'w', encoding='utf-8') as izhodna:
6             for vrstica in vhodna:
7                 prvi = 0
8                 while vrstica[prvi] == ' ':
9                     prvi += 1
10                n = prvi // zamik + 1
11                vrstica = vrstica.strip()
12                if n > nivo:
13                    print(2 * zamik * nivo * ' ' + '<ul>', file=izhodna)
14                    nivo += 1
15                while n < nivo:
16                    nivo -= 1
17                    print(2 * zamik * nivo * ' ' + '</ul>', file=izhodna)
18                print((2 * zamik * nivo - zamik) * ' ' + '<li>' + vrstica, file=
19                    izhodna)
20                while nivo > 0:
21                    nivo -= 1
22                    print(2 * zamik * nivo * ' ' + '</ul>', file=izhodna)

```

8.2 Kolesarski maraton Franja

8.2.1 1. podnaloga: cas maratona

Cilj. `cas_maratona(niz)` naj čas oblike '4:13:59' pretvori v seznam [4, 13, 59].

Niz razbijemo po dvopičjih in vsak del pretvorimo v število z izpeljanim seznamom:

```

1 def cas_maratona(s):
2     return [int(x) for x in s.split(':')]

```

Končna koda.

```

1 def cas_maratona(s):
2     return [int(x) for x in s.split(':')]

```

8.2.2 2. podnaloga: vsi casi

Cilj. `vsi_casi(ime)` naj prebere datoteko s časi in vrne seznam trojčkov.

Pripravimo prazen seznam:

```

1 def vsi_casi(ime):
2     casi = []

```

Odpremo datoteko in za vsako vrstico uporabimo `cas_maratona`:

```

1 def vsi_casi(ime):
2     casi = []
3     with open(ime) as f:
4         for vrstica in f:
5             casi.append(cas_maratona(vrstica.strip()))
6     return casi

```

Končna koda.

```

1 def vsi_casi(ime):
2     casi = []
3     with open(ime) as f:
4         for vrstica in f:
5             casi.append(cas_maratona(vrstica.strip()))
6     return casi

```

8.2.3 3. podnalog: nagrajani

Cilj. nagrajani(vhodna, u, m, s) naj vrne število tekmovalcev z enakim ali boljšim časom od podanega.

Preberemo vse čase in podani čas pretvorimo v sekunde:

```

1 def nagrajani(vhodna, u, m, s):
2     casi = vsi_casi(vhodna)
3     t = u * 3600 + m * 60 + s

```

Vsak čas prav tako pretvorimo v sekunde in preštejemo tiste, ki so manjši ali enaki:

```

1 def nagrajani(vhodna, u, m, s):
2     casi = vsi_casi(vhodna)
3     t = u * 3600 + m * 60 + s
4     return len([cas for cas in casi if cas[0] * 3600 + cas[1] * 60 + cas[2] <= t])

```

Zakaj deluje

Čase primerjamo tako, da jih pretvorimo v skupno enoto (sekunde). Število nagrajenih je dolžina seznama ustreznih tekmovalcev.

Končna koda.

```

1 def nagrajani(vhodna, u, m, s):
2     casi = vsi_casi(vhodna)
3     t = u * 3600 + m * 60 + s
4     return len([cas for cas in casi if cas[0] * 3600 + cas[1] * 60 + cas[2] <= t])

```

8.2.4 4. podnalog: urejeni casi

Cilj. urejeni_casi(vhodna, izhodna) naj čase uredi in zapiše v izhodno datoteko.

Preberemo in uredimo čase. Python zna urediti tudi sezname trojčkov:

```

1 def urejeni_casi(vhodna, izhodna):
2     casi = vsi_casi(vhodna)
3     casi.sort()

```

V izhodno datoteko zapišemo vsak čas nazaj v obliki u:m:s. Števila najprej pretvorimo v nize in jih združimo z ':' :

```

1 def urejeni_casi(vhodna, izhodna):
2     casi = vsi_casi(vhodna)
3     casi.sort()
4     with open(izhodna, 'w') as f:
5         for cas in casi:
6             print(':','.join([str(x) for x in cas]), file=f)

```

Končna koda.

```

1 def urejeni_casi(vhodna, izhodna):
2     casi = vsi_casi(vhodna)
3     casi.sort()
4     with open(izhodna, 'w') as f:
5         for cas in casi:
6             print(':','.join([str(x) for x in cas]), file=f)

```

8.3 Imena

8.3.1 1. podnalog: kolikokrat se pojavi

Cilj. `kolikokrat_se_pojavi(niz, ime)` naj vrne število pojavitev imena v nizu, kjer so imena ločena z vejicami.

Niz razbijemo po vejicah in preštejemo pojavitve z metodo `.count`:

```

1 def kolikokrat_se_pojavi(niz, ime):
2     return niz.split(',').count(ime)

```

Novo v tem razdelku

`list.count.seznam.count(x)` vrne, kolikokrat se `x` pojavi v seznamu.

Končna koda.

```

1 def kolikokrat_se_pojavi(niz, ime):
2     return niz.split(',').count(ime)

```

8.3.2 2. podnalog: koliko

Cilj. `koliko(niz, datoteka)` naj v izhodno datoteko za vsako ime zapiše, kolikokrat se pojavi, v vrstnem redu prvih pojavitev.

Imena razbijemo in sestavimo seznam brez ponovitev (prve pojavitve):

```

1 def koliko(niz, datoteka):
2     imena = niz.split(',')
3     brez_ponovitev = []
4     for ime in imena:
5         if ime not in brez_ponovitev:
6             brez_ponovitev.append(ime)

```

Za vsako ime zapišemo ime in njegovo število pojavitev:

```

1     with open(datoteka, 'w', encoding='utf-8') as f:
2         for ime in brez_ponovitev:
3             print(ime, imena.count(ime), file=f)

```

Končna koda.

```

1 def koliko(niz, datoteka):
2     imena = niz.split(',')
3     brez_ponovitev = []
4     for ime in imena:
5         if ime not in brez_ponovitev:
6             brez_ponovitev.append(ime)
7     with open(datoteka, 'w', encoding='utf-8') as f:
8         for ime in brez_ponovitev:
9             print(ime, imena.count(ime), file=f)

```

8.3.3 3. podnalog: koliko iz datoteke

Cilj. `koliko_iz_datoteke(vhodna, izhodna)` je kot `koliko`, le da bere iz datoteke.

Preberemo vse vrstice in jih očistimo:

```
1 def koliko_iz_datoteke(vhodna, izhodna):
2     vrstice = []
3     with open(vhodna, encoding='utf-8') as f:
4         for vrstica in f:
5             vrstice.append(vrstica.strip())
```

Vrstice združimo v en niz z vejicami in prepustimo delo funkciji `koliko`:

```
1 imena = ','.join(vrstice)
2 koliko(imena, izhodna)
```

Novo v tem razdelku

`','.join(...)`. Elemente seznama združi v en niz, mednje pa vstavi vejice.

Končna koda.

```
1 def koliko_iz_datoteke(vhodna, izhodna):
2     vrstice = []
3     with open(vhodna, encoding='utf-8') as f:
4         for vrstica in f:
5             vrstice.append(vrstica.strip())
6     imena = ','.join(vrstice)
7     koliko(imena, izhodna)
```

8.3.4 4. podnalog: koliko urejen

Cilj. `koliko_urejen(vhod, izhod)` naj izpiše imena in njihove frekvence, urejene padajoče po frekveni, ob izenačenju pa leksikografsko.

Zberemo vsa imena in seznam imen brez ponovitev:

```
1 def koliko_urejen(vhod, izhod):
2     imena = []
3     brez_ponovitev = []
4     with open(vhod, encoding='utf-8') as f:
5         for vrstica in f:
6             for ime in vrstica.strip().split(','):
7                 imena.append(ime)
8                 if ime not in brez_ponovitev:
9                     brez_ponovitev.append(ime)
```

Za vsako ime naredimo par. Frekvenco shranimo *negativno*, da bo sortiranje po njej padajoče, ime pa ostane pozitivno (leksikografsko):

```
1 pari = []
2 for ime in brez_ponovitev:
3     pari.append((-imena.count(ime), ime))
```

Par ureja najprej po prvem elementu (negativna frekvenca), ob izenačenju po drugem (ime):

```
1 pari.sort()
```

Na koncu izpišemo ime in (spet pozitivno) frekvenco:

```
1 with open(izhod, 'w', encoding='utf-8') as f:
2     for s, ime in pari:
3         print(ime, -s, file=f)
```

Zakaj deluje

Terke se sortirajo leksikografsko. Ker je frekvenca negativna, so najpogostejša imena prva; pri enaki frekvenci odloči ime po abecedi.

Končna koda.

```
1 def koliko_urejen(vhod, izhod):
2     imena = []
3     brez_ponovitev = []
4     with open(vhod, encoding='utf-8') as f:
5         for vrstica in f:
6             for ime in vrstica.strip().split(','):
7                 imena.append(ime)
8                 if ime not in brez_ponovitev:
9                     brez_ponovitev.append(ime)
10    pari = []
11    for ime in brez_ponovitev:
12        pari.append((-imena.count(ime), ime))
13    pari.sort()
14    with open(izhod, 'w', encoding='utf-8') as f:
15        for s, ime in pari:
16            print(ime, -s, file=f)
```

8.4 Povzetek

- datoteke odpiramo z `open(..., encoding='utf-8')`;
- **stikalo** (logična spremenljivka) si zapomni stanje med vrsticami (npr. ali smo znotraj značke ali seznama);
- `print(..., end=")` prepreči novo vrstico;
- `str.split` in `str.join` pretvarjata med nizom in seznamom;
- terke sortiramo leksikografsko, trik z negativno vrednostjo pa da padajoč vrstni red.

Preveri se

1. Zakaj `html2txt` potrebuje stikalo **znacka**?
2. Kako `koliko_urejen` doseže padajoče sortiranje po frekvenci?
3. Zakaj pri branju datoteke uporabimo `encoding='utf-8'`?

Poglavje 9

Funkcije višjih redov

V tem poglavju bomo funkcije obravnavali kot vrednosti: funkcija lahko sprejme drugo funkcijo kot argument ali pa jo vrne kot rezultat. Take funkcije imenujemo **funkcije višjih redov**.

Novo v tem razdelku

`lambda`. `lambda x: a * x + b` je kratka, brezimena funkcija, enakovredna

```
1 def f(x):  
2     return a * x + b
```

Uporabimo jo, kadar funkcijo potrebujemo samo na enem mestu.

9.1 Funkcije višjih redov

9.1.1 1. podnalog: linearna

Cilj. `linearna(a, b)` naj vrne *funkcijo*, ki vsak x preslika v $ax + b$.

Funkcija, ki jo vrnemo, mora “vedeti”, kakšna sta a in b . Vrne jo kot `lambda`, ki ta dva podatka ujame v svoji okolici:

```
1 def linearna(a, b):  
2     return lambda x: a * x + b
```

Konzola

```
>>> f = linearna(3, 2)  
>>> f(0.5)  
3.5
```

Zakaj deluje

Vrnjena funkcija si zapomni a in b iz okolice, v kateri je bila ustvarjena. Temu pravimo *zaprte* (closure). Zato vsak klic `linearna` vrne svojo lastno funkcijo.

Končna koda.

```
1 def linearna(a, b):  
2     return lambda x: a * x + b
```

9.1.2 2. podnalog: kompozitum

Cilj. kompozitum(f, g) naj vrne funkcijo $x \mapsto f(g(x))$.

Vrnemo lambda, ki najprej uporabi g, nato pa rezultat še f:

```
1 def kompozitum(f, g):
2     return lambda x: f(g(x))
```

Konzola

```
>>> import math
>>> f = kompozitum(abs, math.sin)
>>> f(3 * math.pi / 2)
1.0
```

Končna koda.

```
1 def kompozitum(f, g):
2     return lambda x: f(g(x))
```

9.1.3 3. podnalog: odvod

Cilj. odvod(f, epsilon=10e-5) naj vrne funkcijo, ki v točki x oceni odvod funkcije f.

Odvod ocenimo z diferenčno formulo $(f(x + \frac{\epsilon}{2}) - f(x - \frac{\epsilon}{2})) / \epsilon$:

```
1 def odvod(f, epsilon=10e-5):
2     return lambda x: (f(x + 0.5 * epsilon) - f(x - 0.5 * epsilon)) / epsilon
```

Konzola

```
>>> f = odvod(lambda x: x * x + 1)
>>> f(1.0)
2.000000000000002
```

Končna koda.

```
1 def odvod(f, epsilon=10e-5):
2     return lambda x: (f(x + 0.5 * epsilon) - f(x - 0.5 * epsilon)) / epsilon
```

9.1.4 4. podnalog: uporabi nkrat

Cilj. uporabi_nkrat(f, x, n) naj funkcijo f na x uporabi n-krat.

Rezultat začnemo s x:

```
1 def uporabi_nkrat(f, x, n):
2     for i in range(n):
3         x = f(x)
4     return x
```

Vsak korak nadomesti x z f(x); po n korakih vrnemo rezultat:

```
1 def uporabi_nkrat(f, x, n):
2     for i in range(n):
3         x = f(x)
4     return x
```

Konzola

```
>>> collatz = lambda x: x // 2 if x % 2 == 0 else 3 * x + 1
>>> uporabi_nkrat(collatz, 13, 7)
4
```

Končna koda.

```
1 def uporabi_nkrat(f, x, n):
2     for i in range(n):
3         x = f(x)
4     return x
```

9.1.5 5. podnalog: uporabi vse

Cilj. `uporabi_vse(sez, x)` naj vsako funkcijo iz seznama `sez` uporabi na `x` in vrne seznam rezultatov.

Za vsako funkcijo `f` iz seznama izračunamo `f(x)` in rezultate zberemo z izpeljanim seznamom:

```
1 def uporabi_vse(sez, x):
2     return [f(x) for f in sez]
```

Konzola

```
>>> import math
>>> uporabi_vse([math.sin, math.cos, abs, int], math.pi / 2)
[1.0, 6.123233995736766e-17, 1.5707963267948966, 1]
```

Končna koda.

```
1 def uporabi_vse(sez, x):
2     return [f(x) for f in sez]
```

9.2 Povzetek

- **funkcija je vrednost:** lahko jo podamo kot argument ali vrnemo kot rezultat;
- **lambda** je kratka brezimena funkcija;
- vrnjena funkcija si zapomni okolico (*zaprtje*);
- **kompozitum**, **odvod** in **uporabi_vse** so primeri funkcij višjih redov.

Preveri se

1. Zakaj `linearne(3, 2)` vrne funkcijo in ne števila?
2. Kaj naredi `kompozitum(abs, math.sin)(3 * math.pi / 2)`?
3. Zakaj `odvod` potrebuje parameter `epsilon`?

Poglavje 10

Rekurzija in sestopanje

Nekatere naloge je najlažje rešiti tako, da jih *prevedemo na manjšo različico iste naloge*. Ta ideja se imenuje **rekurzija**. Kadar pa pri iskanju rešitve poskušamo možnosti in se ob slepi ulici vračamo, govorimo o **sestopanju** (angl. *backtracking*).

V tem poglavju bomo programe *gradili* po korakih: vsako vrstico bomo posebej predstavili in pojasnili, *zakaj* je tam, šele nato pa jo pokazali. **Obarvane vrstice** so tiste, ki so se pravkar dodale.

10.1 Od zanke do rekurzije

Cilj. Izpišimo števila od n navzdol do nič, na koncu pa "Nič!".

Najprej s katero zanko. Najbolj očitna rešitev uporabi zanko `while`. Dokler je n večji od nič, izpišemo n in ga zmanjšamo; ko n pade na nič, izpišemo "Nič!".

```
1 def odstevaj(n):  
2     while n > 0:  
3         print(n)  
4         n = n - 1  
5     print("Nič!")
```

Deluje. A pogledjmo pobliže, kaj se dogaja znotraj zanke: ko izpišemo n in ga zmanjšamo, moramo narediti *natanko isto stvar*, le z manjšim številom. Kaj pa, če bi to »isto stvar« zapakirali v funkcijo in jo preprosto poklicali?

Osnovni primer. Vsaka taka funkcija se mora nekoč ustaviti. Ko je n enak nič, izpišemo "Nič!" in nehamo:

```
1 def odstevaj(n):  
2     if n == 0:  
3         print("Nič!")
```

Rekurzivni korak. Sicer (v `else`) najprej izpišemo n :

```
1 def odstevaj(n):  
2     if n == 0:  
3         print("Nič!")  
4     else:  
5         print(n)
```

Nato zahtevamo isto za $n - 1$; s tem funkcija pokliče *samo sebe*:

```
1 def odstevaj(n):  
2     if n == 0:  
3         print("Nič!")  
4     else:
```

```

5     print(n)
6     odstevej(n - 1)

```

Pravkar smo odkrili **rekurzijo**: funkcija `odstevej` pokliče *samo sebe*, le z manjšim argumentom. Vsaka rekurzivna funkcija ima dve sestavini:

1. **osnovni primer** — primer, ki ga rešimo brez klica (`n == 0`);
2. **rekurzivni korak** — klic samega sebe z manjšim problemom (`odstevej(n - 1)`).

Sledimo klicu `odstevej(3)`:

Klic	Kaj naredi
<code>odstevej(3)</code>	izpiše 3, nato pokliče <code>odstevej(2)</code>
<code>odstevej(2)</code>	izpiše 2, nato pokliče <code>odstevej(1)</code>
<code>odstevej(1)</code>	izpiše 1, nato pokliče <code>odstevej(0)</code>
<code>odstevej(0)</code>	izpiše Nič!, ne pokliče nikogar več

Vsak klic počaka, da se prejšnji konča. Računalnik si nedokončane klice hrani na *skladu klicev*. Zato je ključno, da rekurzija *nekoč* pride do osnovnega primera.

Pogoste napake

Pozabljen osnovni primer je najpogostejša napaka pri rekurziji: funkcija se kliče v nedogled in program javi `RecursionError`. Vedno se vprašaj: »Kdaj je naloga tako majhna, da jo znam rešiti brez rekurziije?«

Rekurzivna vera

Ko pišemo rekurzivno funkcijo, se ni treba v mislih sprehoditi skozi vse klice. Dovolj je, da odgovorimo na dve vprašanji:

1. Ali znam rešiti najmanjši primer (osnovni primer)?
2. Če *zaupam*, da manjši klic že vrne pravilen rezultat, znam iz njega sestaviti rezultat za trenutni primer?

Temu pravimo *rekurzivna vera*. V nadaljevanju jo bomo uporabljali ves čas.

10.2 Enostavne naloge z rekurzijo

Za ogrevanje bomo napisali štiri kratke rekurzivne funkcije. Pri vseh velja pravilo: **brez zank** `for` in `while`.

10.2.1 1. podnaloga: fakulteta

Cilj. Funkcija `fakulteta(n)` naj izračuna $n!$, pri čemer je $0! = 1$.

Začnimo pri najmanjšem primeru. Po definiciji je $0! = 1$, zato v tem primeru vrnemo 1:

```

1 def fakulteta(n):
2     if n == 0:
3         return 1

```

Za $n \geq 1$ velja $n! = n \cdot (n - 1)!$. Zato zmnožimo `n` z rezultatom manjšega klica:

```

1 def fakulteta(n):
2     if n == 0:
3         return 1
4     return n * fakulteta(n - 1)

```

Sledimo klicu `fakulteta(4)` — vrednosti se vračajo od spodaj navzgor:

Klic	Vrednost
<code>fakulteta(0)</code>	1
<code>fakulteta(1)</code>	$1 \cdot 1 = 1$
<code>fakulteta(2)</code>	$2 \cdot 1 = 2$
<code>fakulteta(3)</code>	$3 \cdot 2 = 6$
<code>fakulteta(4)</code>	$4 \cdot 6 = 24$

Zakaj deluje

Rekurzivni korak je *dobesedno* matematična definicija fakultete, osnovni primer pa je pravilen ($0! = 1$). Ker vsak klic zmanjša `n` za ena, rekurzija nekoč doseže nič in se ustavi.

Pogoste napake

Osnovni primer `if n == 1` ne deluje: pri `fakulteta(0)` pademo v `fakulteta(-1)` in rekurzija se nikoli ne ustavi. Pravilen osnovni primer je `n == 0`.

Končna koda.

```

1 def fakulteta(n):
2     if n == 0:
3         return 1
4     return n * fakulteta(n - 1)

```

10.2.2 2. podnalog: največji skupni delitelj

Cilj. `gcd(m, n)` naj vrne največji skupni delitelj; če sta oba argumenta nič, naj vrne `None`.

Vsaka rekurzija se mora nekoč ustaviti. Kdaj je največji skupni delitelj očiten? Ko je drugi argument nič — tedaj je odgovor prvi argument. Zato najprej obravnavamo `n == 0` in vrnemo `m`:

```

1 def gcd(m, n):
2     if n == 0:
3         return m

```

Sicer uporabimo Evklidov algoritem, ki pravi $\text{gcd}(m, n) = \text{gcd}(n, m \bmod n)$. Vsak klic zmanjša problem, zato se bo rekurzija nekoč ustavila:

```

1 def gcd(m, n):
2     if n == 0:
3         return m
4     return gcd(n, m % n)

```

Naloga še zahteva, da za `gcd(0, 0)` vrnemo `None`. Ker se oba ujameta v `n == 0`, to preverimo prav tam:

```

1 def gcd(m, n):
2     if n == 0:
3         if m == 0:
4             return None
5         return m
6     return gcd(n, m % n)

```

Zakaj deluje

Operator % vrne ostanek, ki je vedno strogo manjši od delitelja. Zato se n v vsakem koraku zmanjša in rekurzija se ustavi. Ko je n enak nič, je m ravno največji skupni delitelj.

Končna koda.

```

1 def gcd(m, n):
2     if n == 0:
3         if m == 0:
4             return None
5         return m
6     return gcd(n, m % n)

```

10.2.3 3. podnalog: Cantorjeva množica

Cilj. `cantor(n)` naj vrne niz dolžine 3^n : srednja tretjina so presledki, prva in zadnja tretjina pa sta $(n - 1)$ -ti približek. Velja `cantor(0) = '*'`.

Najprej osnovni primer: 0-ti približek je en sam zvezdica:

```

1 def cantor(n):
2     if n == 0:
3         return '*'

```

Približek gradimo iz *dveh kopij* prejšnjega približka. Ker ga potrebujemo dvakrat, ga shranimo v spremenljivko `manjsi`:

```

1 def cantor(n):
2     if n == 0:
3         return '*'
4     manjsi = cantor(n - 1)

```

Rezultat so trije deli: `manjsi`, presledki in spet `manjsi`. Sredina mora biti dolga toliko, kolikor je dolg `manjsi`, zato naredimo `len(manjsi)` presledkov:

```

1 def cantor(n):
2     if n == 0:
3         return '*'
4     manjsi = cantor(n - 1)
5     return manjsi + len(manjsi) * ' ' + manjsi

```

Konzola

```

>>> cantor(0)
'*'
>>> cantor(1)
'* *'
>>> cantor(2)

```

```
'* * * *'
```

Zakaj deluje

Dolžina `cantor(n-1)` je 3^{n-1} , zato je natanko toliko presledkov potrebnih v sredini. Dolžina se vsakič potroji, kar ustreza 3^n .

Končna koda.

```
1 def cantor(n):
2     if n == 0:
3         return '*'
4     manjsi = cantor(n - 1)
5     return manjsi + len(manjsi) * ' ' + manjsi
```

10.2.4 4. podnalog: Fibonacci z akumulatorji

Cilj. `fibonacci(n)` naj vrne n -to Fibonaccijevo število (0, 1, 1, 2, 3, 5, 8, ...). Testi bodo preverili velike n , zato mora biti rešitev hitra.

Novo v tem razdelku

Akumulatorja sta pomožna argumenta, v katerih si podajamo delne rezultate; z rekurzijo tako *simuliramo zanko*. Primer za potenco:

```
1 def potenca(a, n, x=1):
2     if n == 0:
3         return x
4     return potenca(a, n - 1, a * x)
```

Argument x ima *privzeto vrednost* 1: klic `potenca(2, 10)` se začne z $x = 1$ in ga v vsakem koraku pomnoži z a .

Fibonaccijevo zaporedje začnemo z $a = 0$ in $b = 1$; to sta akumulatorja, ki imata privzeti vrednosti, zato ju ni treba podajati pri klicu. Ko je n enak nič, je iskani člen ravno a :

```
1 def fibonacci(n, a=0, b=1):
2     if n == 0:
3         return a
```

V vsakem koraku se pomaknemo na naslednji par členov: novi a postane stari b , novi b pa vsota obeh. Nato zmanjšamo n :

```
1 def fibonacci(n, a=0, b=1):
2     if n == 0:
3         return a
4     return fibonacci(n - 1, b, a + b)
```

Sledimo `fibonacci(7)` (v vrstici sta akumulatorja a in b):

n	7	6	5	4	3	2	1
a	0	1	1	2	3	5	8
b	1	1	2	3	5	8	13

Pri $n == 0$ vrnemo a , torej je `fibonacci(7) = 13`.

Zakaj deluje

Vsak klic opravi konstantno dela in zmanjša n za ena, zato je klicev $n + 1$ in čas $O(n)$.

Končna koda.

```
1 def fibonacci(n, a=0, b=1):
2     if n == 0:
3         return a
4     return fibonacci(n - 1, b, a + b)
```

10.3 Nepadajoča zaporedja

Zaporedje je **nepadajoče**, če velja $a_i \leq a_{i+1}$ za vsak i .

10.3.1 1. podnaloga: ali je zaporedje nepadajoče

Cilj. `nepadajoce(s)` naj vrne `True`, če je seznam `s` nepadajoč.

Seznam z največ enim elementom nima nobenega para, ki bi bil neurejen, zato je avtomatično nepadajoč; v tem primeru vrnemo `True`:

```
1 def nepadajoce(s):
2     if len(s) <= 1:
3         return True
```

Sicer morata biti prva dva elementa urejena. To preverimo s `s[0] <= s[1]` in povežemo z `and` zahtevo, da je nepadajoč tudi rep `s[1:]`:

```
1 def nepadajoce(s):
2     if len(s) <= 1:
3         return True
4     return s[0] <= s[1] and nepadajoce(s[1:])
```

Zakaj deluje

Vsak klic odstrani prvi element, zato se rekurzija ustavi. Povezava `and` je lena: če prvi pogoj ne velja, se drugi niti ne preveri.

Poskusi sam

Kaj vrne `nepadajoce([])`? Prazno zaporedje nima nobenega para, zato `True`.

Končna koda.

```
1 def nepadajoce(s):
2     if len(s) <= 1:
3         return True
4     return s[0] <= s[1] and nepadajoce(s[1:])
```

10.3.2 2. podnaloga: nagni zaporedje

Cilj. `nagni(s)` naj vrne $a_1, a_2 + 1, a_3 + 2, \dots, a_n + (n - 1)$.

Osnovni primer: prazen seznam nagnemo v prazen seznam:

```

1 def nagni(s):
2     if len(s) == 0:
3         return []

```

Sicer najprej nagnemo vse razen zadnjega elementa. To dobimo z rezino $s[:-1]$, ki vrne seznam brez zadnjega elementa:

```

1 def nagni(s):
2     if len(s) == 0:
3         return []
4     return nagni(s[:-1])

```

Nagnjenemu delu nato pripnemo zadnji element prvotnega seznama ($s[-1]$), povečan za odmik $\text{len}(s) - 1$:

```

1 def nagni(s):
2     if len(s) == 0:
3         return []
4     return nagni(s[:-1]) + [s[-1] + len(s) - 1]

```

Zakaj deluje

Element na mestu k (šteto od 1) se poveča za $k - 1$. Za zadnji element je $\text{len}(s) - 1$ ravno ta odmik, zato je vsak element povečan pravilno.

Končna koda.

```

1 def nagni(s):
2     if len(s) == 0:
3         return []
4     return nagni(s[:-1]) + [s[-1] + len(s) - 1]

```

10.3.3 3. podnalog: ponavljalj, dokler ni urejeno

Cilj. `nagibaj(s)` naj `nagni` ponavlja, dokler zaporedje ne postane nepadajoče.

Osnovni primer: če je zaporedje že nepadajoče, ga vrnemo nespremenjenega:

```

1 def nagibaj(s):
2     if nepadajoce(s):
3         return s

```

Sicer ga nagnemo in postopek ponovimo z rekurzivnim klicem:

```

1 def nagibaj(s):
2     if nepadajoce(s):
3         return s
4     return nagibaj(nagni(s))

```

Zakaj deluje

Vsak `nagni` poveča razlike med sosedi, zato se zaporedje približuje urejenemu in se postopek nekoč ustavi.

Končna koda.

```

1 def nagibaj(s):
2     if nepadajoce(s):
3         return s
4     return nagibaj(nagni(s))

```

10.4 Šahovske dame

Položaj figure predstavimo s parom (**st**, **vr**); **st** je stolpec, **vr** vrstica (obe med 1 in 8). Dami se napadata, če sta v isti vrstici, stolpcu ali na isti diagonali.

10.4.1 1. podnalog: ali se dami napadata

Cilj. `se_napadata(poz1, poz2)` naj vrne `True`, kadar se dami napadata.

Koordinate najprej razpakiramo v imenovane spremenljivke, da je koda berljivejša. Par (**st**, **vr**) razdelimo na stolpec in vrstico:

```

1 def se_napadata(poz1, poz2):
2     st1, vr1 = poz1
3     st2, vr2 = poz2

```

Dami se napadata, če sta v istem stolpcu. To preverimo s primerjavo stolpcev:

```

1 def se_napadata(poz1, poz2):
2     st1, vr1 = poz1
3     st2, vr2 = poz2
4     return st1 == st2

```

Napadata se tudi, če sta v isti vrstici, zato dodamo še ta pogoj, povezan z `or`:

```

1 def se_napadata(poz1, poz2):
2     st1, vr1 = poz1
3     st2, vr2 = poz2
4     return st1 == st2 or vr1 == vr2

```

In še diagonala: na isti diagonali sta natanko tedaj, ko sta navpična in vodoravna razdalja enaki. Razdalji izračunamo z `abs`:

```

1 def se_napadata(poz1, poz2):
2     st1, vr1 = poz1
3     st2, vr2 = poz2
4     return st1 == st2 or vr1 == vr2 or abs(vr1 - vr2) == abs(st1 - st2)

```

Zakaj deluje

To so *edini* trije načini, kako se dami napadata, zato jih vežemo z `or`: zadošča že eden. `abs` vrne absolutno vrednost, npr. `abs(2 - 5)` vrne 3.

Končna koda.

```

1 def se_napadata(poz1, poz2):
2     st1, vr1 = poz1
3     st2, vr2 = poz2
4     return st1 == st2 or vr1 == vr2 or abs(vr1 - vr2) == abs(st1 - st2)

```

10.4.2 2. podnalog: proste vrstice

Cilj. `proste_vrstice(dame, st)` naj vrne množico vrstic v stolpcu `st`, ki jih nobena postavljena dama ne napada.

Novo v tem razdelku

Izpeljava množice zgradi množico v eni vrstici:

```
1 {x for x in [1, 2, 3, 4] if x % 2 == 0}    # {2, 4}
```

`all` vrne `True`, če je vsak element resničen:

```
1 all([True, True, False])    # False
2 all([])                     # True  <-- prazno je "vse res"
```

V stolpcu `st` so vrstice od 1 do 8. Za začetek jih zberimo vse; to naredi izpeljava množice z zanko `for` vr `in range(1, 9)`:

```
1 def proste_vrstice(dame, st):
2     return {vr for vr in range(1, 9)}
```

Vrstico pa obdržimo le, če je *nobena* dama ne napada. »Nobena« izrazimo z `all`: za vsako damo iz `dame` mora veljati `not se_napadata((st, vr), dama)`. Pogoji dodamo v izpeljavo z `if`:

```
1 def proste_vrstice(dame, st):
2     return {vr for vr in range(1, 9)
3             if all(not se_napadata((st, vr), dama) for dama in dame)}
```

Zakaj deluje

`all` vrne `True` le, če *vse* dame polje pustijo pri miru. Če je `dame` prazna, je `all([])` res `in` so proste vse vrstice — kar je pravilno.

Končna koda.

```
1 def proste_vrstice(dame, st):
2     return {vr for vr in range(1, 9)
3             if all(not se_napadata((st, vr), dama) for dama in dame)}
```

10.4.3 3. podnalog: osem dam

Cilj. `osem_dam()` naj vrne razporeditev osmih dam, da se nobeni dve ne napadata, ali `None`.

Dame postavljamo stolpec za stolpcem. Delo bo opravila pomožna funkcija `postavi(dame, st)`, zato glavno funkcijo zapišimo najprej; začnemo s prazno množico dam v 1. stolpcu:

```
1 def osem_dam():
2     return postavi(set(), 1)
```

Zdaj pomožna funkcija. Osnovni primer: če smo šli čez vseh osem stolpcev, smo postavili vse dame in vrnemo razporeditev:

```
1 def postavi(dame, st):
2     if st > 8:
3         return dame
```

Sicer v stolpcu `st` poskusimo vsako prosto vrstico; to so natanko vrstice, ki jih vrne `proste_vrstice`:

```

1 def postavi(dame, st):
2     if st > 8:
3         return dame
4     for vr in proste_vrstice(dame, st):

```

Damo postavimo na (st, vr) in nadaljujemo v naslednjem stolpcu. Razporeditev, v kateri je ta dama, sestavimo z $dame \mid \{(st, vr)\}$ (to je unija množic); rezultat rekurzivnega klica shranimo v `resitev`:

```

1 def postavi(dame, st):
2     if st > 8:
3         return dame
4     for vr in proste_vrstice(dame, st):
5         resitev = postavi(dame  $\mid$   $\{(st, vr)\}$ , st + 1)

```

Če ta pot vodi do rešitve, jo vrnemo:

```

1 def postavi(dame, st):
2     if st > 8:
3         return dame
4     for vr in proste_vrstice(dame, st):
5         resitev = postavi(dame  $\mid$   $\{(st, vr)\}$ , st + 1)
6         if resitev is not None:
7             return resitev

```

Če nobena vrstica ne uspe, javimo neuspeh z `None`:

```

1 def postavi(dame, st):
2     if st > 8:
3         return dame
4     for vr in proste_vrstice(dame, st):
5         resitev = postavi(dame  $\mid$   $\{(st, vr)\}$ , st + 1)
6         if resitev is not None:
7             return resitev
8     return None

```

Ker `postavi` uporablja le `osem_dam`, jo skrijemo vanjo:

```

1 def osem_dam():
2     def postavi(dame, st):
3         if st > 8:
4             return dame
5         for vr in proste_vrstice(dame, st):
6             resitev = postavi(dame  $\mid$   $\{(st, vr)\}$ , st + 1)
7             if resitev is not None:
8                 return resitev
9         return None
10    return postavi(set(), 1)

```

Zakaj deluje

Vsak klic `postavi` natanko eno damo v svoj stolpec. Če v nekem stolpcu nobena vrstica ne omogoča dokončanja, se vrnemo v prejšnji stolpec in poskusimo naslednjo vrstico. Ker je stolpcev in vrstic končno mnogo, postopek nekoč najde rešitev ali izčrpa vse možnosti.

Poskusi sam

Zakaj takoj vrnemo `resitev`, ko ni `None`? Ker zadošča *ena* rešitev; če bi nadaljevali zanko, bi po nepotrebnem iskali še druge.

Končna koda.

```

1 def osem_dam():
2     def postavi(dame, st):
3         if st > 8:
4             return dame
5         for vr in proste_vrstice(dame, st):
6             resitev = postavi(dame | {(st, vr)}, st + 1)
7             if resitev is not None:
8                 return resitev
9         return None
10    return postavi(set(), 1)

```

10.5 Sudoku

Sudoku je tabela 9×9 , predstavljena s seznamom seznamov `t`; `t[i][j]` je polje v vrstici `i` in stolpcu `j`, prazna polja so `None`. V vsaki vrstici, stolpcu in podkvadratu 3×3 se vsako število od 1 do 9 pojavi natanko enkrat.

10.5.1 1. podnalog: ali je sudoku rešen

Cilj. `je_resen(t)` naj vrne `True`, če je sudoku pravilno in v celoti rešen.

Vrstica je pravilna, če njena množica vrednosti ustreza $1, \dots, 9$. Zato si najprej pripravimo referenčno množico `s`:

```

1 def je_resen(t):
2     s = set(range(1, 10))

```

Sprehodimo se po vrsticah `i`:

```

1 def je_resen(t):
2     s = set(range(1, 10))
3     for i in range(9):

```

Če se množica `i`-te vrstice razlikuje od `s`, sudoku ni rešen:

```

1 def je_resen(t):
2     s = set(range(1, 10))
3     for i in range(9):
4         if set(t[i]) != s:
5             return False

```

Enako preverimo še stolpce. `i`-ti stolpec sestavimo z izpeljavo množice `{t[j][i] for j in range(9)}`:

```

1 def je_resen(t):
2     s = set(range(1, 10))
3     for i in range(9):
4         if set(t[i]) != s:
5             return False
6         if {t[j][i] for j in range(9)} != s:
7             return False

```

Preveriti moramo še devet podkvadratov 3×3 . Z zankama i in j po tretjinah izberemo podkvadrat, njegove elemente pa zberemo z dvojno izpeljavo:

```

1 def je_resen(t):
2     s = set(range(1, 10))
3     for i in range(9):
4         if set(t[i]) != s:
5             return False
6         if {t[j][i] for j in range(9)} != s:
7             return False
8     for i in range(3):
9         for j in range(3):
10            if {t[k][m] for k in range(3 * i, 3 * i + 3)
11                for m in range(3 * j, 3 * j + 3)} != s:
12                return False

```

Če preživimo vse preizkuse, vrnemo True:

```

1 def je_resen(t):
2     s = set(range(1, 10))
3     for i in range(9):
4         if set(t[i]) != s:
5             return False
6         if {t[j][i] for j in range(9)} != s:
7             return False
8     for i in range(3):
9         for j in range(3):
10            if {t[k][m] for k in range(3 * i, 3 * i + 3)
11                for m in range(3 * j, 3 * j + 3)} != s:
12                return False
13     return True

```

Zakaj deluje

Množici sta enaki natanko tedaj, ko vsebujeta iste elemente. Če je v vrstici podvojeno število ali None, se množici ne ujemata in funkcija vrne False. Če preživi vse preizkuse, vrne True.

Končna koda.

```

1 def je_resen(t):
2     s = set(range(1, 10))
3     for i in range(9):
4         if set(t[i]) != s:
5             return False
6         if {t[j][i] for j in range(9)} != s:
7             return False
8     for i in range(3):
9         for j in range(3):
10            if {t[k][m] for k in range(3 * i, 3 * i + 3)
11                for m in range(3 * j, 3 * j + 3)} != s:
12                return False
13     return True

```

10.5.2 2. podnalog: možne poteze

Cilj. `mozne_poteze(t, i, j)` naj vrne števila, ki jih smemo vpisati v `t[i][j]`.

Če je polje že izpolnjeno, ni nobene možne poteze; vrnemo prazno množico:

```
1 def mozne_poteze(t, i, j):
2     if t[i][j] is not None:
3         return set()
```

Začnemo z vsemi števili od 1 do 9 in odstranimo tista, ki so že v vrstici. Vrstico i zberemo z izpeljavo množice:

```
1 def mozne_poteze(t, i, j):
2     if t[i][j] is not None:
3         return set()
4     vrstica = {t[i][k] for k in range(9)}
```

Od začetne množice odštejemo prepovedana števila iz vrstice (z operatorjem $-$):

```
1 def mozne_poteze(t, i, j):
2     if t[i][j] is not None:
3         return set()
4     vrstica = {t[i][k] for k in range(9)}
5     return set(range(1, 10)) - vrstica
```

Enako odstranimo še števila iz stolpca j :

```
1 def mozne_poteze(t, i, j):
2     if t[i][j] is not None:
3         return set()
4     vrstica = {t[i][k] for k in range(9)}
5     stolpec = {t[k][j] for k in range(9)}
6     return set(range(1, 10)) - vrstica - stolpec
```

In še števila iz podkvadrata 3×3 . Podkvadrat določimo s tretjinama $3 * (i // 3)$ in $3 * (j // 3)$:

```
1 def mozne_poteze(t, i, j):
2     if t[i][j] is not None:
3         return set()
4     vrstica = {t[i][k] for k in range(9)}
5     stolpec = {t[k][j] for k in range(9)}
6     kvadrat = {t[k][m]
7                 for k in range(3 * (i // 3), 3 * (i // 3) + 3)
8                 for m in range(3 * (j // 3), 3 * (j // 3) + 3)}
9     return set(range(1, 10)) - vrstica - stolpec - kvadrat
```

Novo v tem razdelku

Operator $//$ je celoštevilsko deljenje: $7 // 3$ vrne 2. **Odštevanje množic $A - B$** odstrani iz A vse elemente B . None nas ne moti, saj ga v $\{1, \dots, 9\}$ tako ali tako ni.

Pogoste napake

Pri podkvadratu je pogosta napaka uporabiti i in j namesto $3 * (i // 3)$. Podkvadrat je odvisen le od tretjine, v kateri sta i in j .

Končna koda.

```
1 def mozne_poteze(t, i, j):
2     if t[i][j] is not None:
3         return set()
```

```

4  vrstica = {t[i][k] for k in range(9)}
5  stolpec = {t[k][j] for k in range(9)}
6  kvadrat = {t[k][m]
7              for k in range(3 * (i // 3), 3 * (i // 3) + 3)
8              for m in range(3 * (j // 3), 3 * (j // 3) + 3)}
9  return set(range(1, 10)) - vrstica - stolpec - kvadrat

```

10.5.3 3. podnalog: reši sudoku

Cilj. `sudoku(t)` naj sudoku reši in vrne rešitev ali `None`.

Spet bomo sestopali. Poiščemo prazno polje, vanj poskusimo vpisati vsako možno število in rekurzivno nadaljujemo. Najprej se sprehodimo po vseh poljih; ko naletimo na prazno (`t[i][j]` is `None`), poskusimo vsako možno vrednost iz `mozne_poteze`:

```

1 def sudoku(t):
2     def sestopaj():
3         for i in range(9):
4             for j in range(9):
5                 if t[i][j] is None:
6                     for n in mozne_poteze(t, i, j):

```

Vrednost vpišemo in poskusimo dokončati sudoku z rekurzivnim klicem:

```

1 def sudoku(t):
2     def sestopaj():
3         for i in range(9):
4             for j in range(9):
5                 if t[i][j] is None:
6                     for n in mozne_poteze(t, i, j):
7                         t[i][j] = n
8                         if sestopaj():
9                             return True

```

Ko poskus spodleti, moramo vpisano vrednost *zbrisati*, preden poskusimo naslednjo. Če nobena ne uspe, javimo neuspeh. Če praznih polj ni več, je sudoku rešen:

```

1 def sudoku(t):
2     def sestopaj():
3         for i in range(9):
4             for j in range(9):
5                 if t[i][j] is None:
6                     for n in mozne_poteze(t, i, j):
7                         t[i][j] = n
8                         if sestopaj():
9                             return True
10                    t[i][j] = None
11                    return False
12    return True

```

Tabelo na začetku prekopiramo, da ne pokvarimo izvirnika. Če je sudoku rešljiv, vrnemo rešitev, sicer `None`:

```

1 def sudoku(t):
2     def sestopaj():
3         for i in range(9):
4             for j in range(9):
5                 if t[i][j] is None:
6                     for n in mozne_poteze(t, i, j):

```

```

7         t[i][j] = n
8         if sestopaj():
9             return True
10        t[i][j] = None
11        return False
12    return True
13    t = [vrstica[:] for vrstica in t]
14    if sestopaj():
15        return t
16    return None

```

Zakaj deluje

Za vsako možnost vpišemo vrednost, poskusimo dokončati sudoku in ob neuspehu vrednost zberemo ter poskusimo naslednjo. Tabelo `t` prekopiramo, da ne pokvarimo izvirnika.

Pogoste napake

Pozabljen `t[i][j] = None` pomeni, da nadaljujemo na pokvarjeni tabeli. Enako pomembna je kopija `t = [vrstica[:] for vrstica in t]`; brez nje funkcija spremeni tabelo, ki jo je prejela.

Končna koda.

```

1 def sudoku(t):
2     def sestopaj():
3         for i in range(9):
4             for j in range(9):
5                 if t[i][j] is None:
6                     for n in mozne_poteze(t, i, j):
7                         t[i][j] = n
8                         if sestopaj():
9                             return True
10                        t[i][j] = None
11                    return False
12            return True
13    t = [vrstica[:] for vrstica in t]
14    if sestopaj():
15        return t
16    return None

```

10.6 Povzetek

- **rekurzija** — funkcija pokliče samo sebe; potrebujeta osnovni primer in rekurzivni korak;
- **rekurzivna vera** — zaupamo, da manjši klic že deluje;
- **akumulatorja** — z njima rekurzivno simuliramo zanko (`fibonacci`);
- **strukturna rekurzija** — krajšanje seznama ali niza (`nepadajoce`, `nagni`);
- **sestopanje** — poskusimo potezo, ob slepi ulici pa jo *razveljavimo* in poskusimo drugo (`osem_dam`, `sudoku`).

Preveri se

1. Kaj se zgodi, če rekurzivna funkcija nima osnovnega primera?
2. Zakaj je `fibonacci` z akumulatorjema časovno $O(n)$?
3. V `proste_vrstice` — kaj bi se zgodilo, če bi `all` zamenjali z `any`?
4. Pri `sudoku` — zakaj moramo ob neuspelem poskusu zapisati `t[i][j] = None`?

Poglavje 11

Bisekcija

Bisekcija (binarno iskanje) je postopek iskanja v *urejeni* tabeli: pogledamo srednji element in se odločimo, v kateri polovici nadaljujemo. Tako v vsakem koraku prepolovimo obseg iskanja, zato je časovna zahtevnost $O(\log n)$.

11.1 Bisekcija z rekurzijo

Cilj. najdi(*a*, *x*) naj v urejeni tabeli *a* poišče indeks elementa *x* (ali *None*). Tokrat rekurzivno, brez zank.

Najprej si pogledjmo zančno različico, ki jo bomo prevedli v rekurzijo:

```
1 def najdi(a, x):
2     levo = 0
3     desno = len(a) - 1
4     while levo <= desno:
5         sredina = (levo + desno) // 2
6         if x == a[sredina]:
7             return sredina
8         elif x < a[sredina]:
9             desno = sredina - 1
10        else:
11            levo = sredina + 1
12    return None
```

Zanko nadomestimo z rekurzivno pomožno funkcijo *isci*(*levo*, *desno*). Osnovni primer: če se meji križata, elementa ni:

```
1 def najdi(a, x):
2     def isci(levo, desno):
3         if levo > desno:
4             return None
```

Sicer izračunamo sredino:

```
1 def najdi(a, x):
2     def isci(levo, desno):
3         if levo > desno:
4             return None
5     sredina = (levo + desno) // 2
```

Če je *x* na sredini, vrnemo indeks:

```
1 def najdi(a, x):
2     def isci(levo, desno):
3         if levo > desno:
```

```

4         return None
5     sredina = (levo + desno) // 2
6     if x == a[sredina]:
7         return sredina

```

Če je x manjši, iščemo v levi polovici, sicer v desni. To sta rekurzivna klica:

```

1 def najdi(a, x):
2     def isci(levo, desno):
3         if levo > desno:
4             return None
5         sredina = (levo + desno) // 2
6         if x == a[sredina]:
7             return sredina
8         elif x < a[sredina]:
9             return isci(levo, sredina - 1)
10        else:
11            return isci(sredina + 1, desno)

```

Zunanja funkcija le sproži iskanje čez celotno tabelo:

```

1     return isci(0, len(a) - 1)

```

Zakaj deluje

Vsak klic razdeli obseg na pol, zato je klicev približno $\log_2 n$ in časovna zahtevnost $O(\log n)$. Ker se meje vsakič strogo zmanjšajo, se rekurzija nekoč ustavi.

Končna koda.

```

1 def najdi(a, x):
2     def isci(levo, desno):
3         if levo > desno:
4             return None
5         sredina = (levo + desno) // 2
6         if x == a[sredina]:
7             return sredina
8         elif x < a[sredina]:
9             return isci(levo, sredina - 1)
10        else:
11            return isci(sredina + 1, desno)
12     return isci(0, len(a) - 1)

```

11.2 Bisekcija z dvema tabelama

Cilj. najdi(ime, priimek, i, p) naj poišče indeks osebe z imenom i in priimkom p . Spisek je urejen po priimku, nato po imenu.

Ključna ideja: osebo predstavimo s parom (priimek, ime). Taka terka se ureja najprej po priimku, ob izenačenju pa po imenu — natanko kot spisek. Zato lahko uporabimo običajno bisekcijo:

```

1 def najdi(ime, priimek, i, p):
2     levo = 0
3     desno = len(ime) - 1

```

Ponavljamo, dokler se meji ne križata:

```

1 def najdi(ime, priimek, i, p):
2     levo = 0
3     desno = len(ime) - 1
4     while levo <= desno:

```

Izračunamo sredino:

```

1     sredina = (levo + desno) // 2

```

Če se terka sredine ujema z iskano, vrnemo indeks:

```

1     if (p, i) == (priimek[sredina], ime[sredina]):
2         return sredina

```

Če je iskana oseba “manjša”, iščemo levo, sicer desno:

```

1     elif (p, i) < (priimek[sredina], ime[sredina]):
2         desno = sredina - 1
3     else:
4         levo = sredina + 1
5     return None

```

Zakaj deluje

Ker terke primerjamo leksikografsko, primerjava $(p, i) < (\text{priimek}[k], \text{ime}[k])$ pravilno upošteva urejenost po priimku in nato po imenu.

Končna koda.

```

1 def najdi(ime, priimek, i, p):
2     levo = 0
3     desno = len(ime) - 1
4     while levo <= desno:
5         sredina = (levo + desno) // 2
6         if (p, i) == (priimek[sredina], ime[sredina]):
7             return sredina
8         elif (p, i) < (priimek[sredina], ime[sredina]):
9             desno = sredina - 1
10        else:
11            levo = sredina + 1
12    return None

```

11.3 Bisekcija na ponovljenih elementih

Cilj. najdi(a , x) naj vrne območje (i, j) , na katerem se v urejeni tabeli (z morebitnimi ponovitvami) pojavlja x . Rešitev naj bo $O(\log n)$.

Najprej obravnavamo prazen seznam:

```

1 def najdi(a, x):
2     if len(a) == 0:
3         return None

```

Nastavimo meji in iščemo *levi rob*. Premikamo se tako, da n kaže na prvo možno pojavitev:

```

1 def najdi(a, x):
2     if len(a) == 0:
3         return None

```

```

4 m = 0
5 n = len(a) - 1
6 while m < n:

```

Vzamemo sredino; če je $a[k] < x$, je levi rob desno, sicer levo:

```

1 k = (m + n) // 2
2 if a[k] < x:
3     m = k + 1
4 else:
5     n = k

```

Ko se ustavi, je m levi rob. Če $a[m]$ ni x , elementa ni:

```

1 if a[m] != x:
2     return None

```

Levi rob si zapomnimo in zdaj iščemo *desni rob* (z zaokroževanjem sredine navzgor):

```

1 else:
2     i = m
3     n = len(a) - 1
4     while m < n:
5         k = (m + n + 1) // 2
6         if a[k] > x:
7             n = k - 1
8         else:
9             m = k
10    j = m
11    return (i, j + 1)

```

Zakaj deluje

Prva bisekcija poišče prvo pojavitev, druga zadnjo. Območje vrnemo kot $(i, j+1)$, ker $j+1$ ustreza rezini $a[i:j+1]$. Obe iskanji sta $O(\log n)$.

Končna koda.

```

1 def najdi(a, x):
2     if len(a) == 0:
3         return None
4     m = 0
5     n = len(a) - 1
6     while m < n:
7         k = (m + n) // 2
8         if a[k] < x:
9             m = k + 1
10        else:
11            n = k
12    if a[m] != x:
13        return None
14    else:
15        i = m
16        n = len(a) - 1
17        while m < n:
18            k = (m + n + 1) // 2
19            if a[k] > x:
20                n = k - 1
21            else:

```

```

22         m = k
23     j = m
24     return (i, j + 1)

```

11.4 Manjkajoči element

11.4.1 1. podnaloga: urejena tabela

Cilj. V urejeni tabeli, ki vsebuje vsa števila od 0 do n razen enega, poišči manjkajoče število v času $O(\log n)$.

Ključna ugotovitev: dokler ni luknje, velja $a[k] == k$. Ko se to prvič pokvari, smo za luknjo. Zato z bisekcijo iščemo prvi indeks, kjer $a[m] != m$.

Nastavimo meji in obravnavamo robni primer (manjka zadnji element):

```

1 def kdo_manjka(a):
2     i = 0
3     j = len(a) - 1
4     if len(a) == 0 or a[j] == j:
5         return len(a)

```

Bisekcija: če na sredini velja $a[m] == m$, je luknja desno, sicer levo:

```

1     while i < j:
2         m = (i + j) // 2
3         if a[m] == m:
4             i = m + 1
5         else:
6             j = m
7     return i

```

Zakaj deluje

Indeksi pred luknjo zadoščajo $a[m] == m$, za njo pa $a[m] > m$. Zato bisekcija najde mejo v $O(\log n)$.

Končna koda.

```

1 def kdo_manjka(a):
2     i = 0
3     j = len(a) - 1
4     if len(a) == 0 or a[j] == j:
5         return len(a)
6     while i < j:
7         m = (i + j) // 2
8         if a[m] == m:
9             i = m + 1
10        else:
11            j = m
12    return i

```

11.4.2 2. podnaloga: neurejena tabela

Cilj. Enako, a tabela ni urejena, čas $O(n)$ in konstantno dodatnega prostora.

Trik: manjkajočega elementa ne iščemo, ampak ga *izračunamo*. Vsota vseh števil od 0 do n je $n(n+1)/2$; če od nje odštejemo vsoto tabele, dobimo manjkajoče število:

```

1 def kdo_manjka_neurejen(l):
2     n = len(l)
3     return n * (n + 1) // 2 - sum(l)

```

Konzola

```

>>> kdo_manjka_neurejen([11, 6, 5, 1, 8, 4, 2, 13, 10, 12, 3, 9, 0])
7

```

Končna koda.

```

1 def kdo_manjka_neurejen(l):
2     n = len(l)
3     return n * (n + 1) // 2 - sum(l)

```

11.5 Poštevanka

Cilj. `tabela_mnozenj(n, m, k)` naj vrne k -to najmanjše število v tabeli množenj velikosti $n \times m$.

Če k ni smiselno, vrnemo `None`:

```

1 def tabela_mnozenj(n, m, k):
2     if k < 1 or k > m * n:
3         return None

```

Zaradi simetrije naj bo n manjši od m (da je zanka v pomožnem izračunu čim krajša):

```

1     else:
2         if n > m:
3             n, m = m, n

```

Z bisekcijo iščemo največje število, za katero je v tabeli natanko $k - 1$ manjših števil. Spodnja in zgornja meja:

```

1     i = 1
2     j = m * n

```

Ponavljamo, dokler se meji ne srečata:

```

1     while i < j:

```

Vzamemo sredino (zaokroženo navzgor):

```

1     mid = (i + j + 1) // 2

```

Preštejemo, koliko števil v tabeli je strogo manjših od mid . Za vsako vrstico i je to $mid // i$, zmanjšano za 1, če je mid deljiv z i (da ne štejemo samega sebe), in omejeno z m :

```

1     manjsih = sum(min(mid // i - (1 if mid % i == 0 else 0), m)
2                   for i in range(1, n + 1))

```

Glede na to premaknemo mejo in na koncu vrnemo i :

```

1         if manjsih > k - 1:
2             j = mid - 1
3         else:
4             i = mid
5     return i

```

Zakaj deluje

Pomožni izračun manjsih je $O(n)$, bisekcija pa naredi $O(\log(nm))$ korakov, zato je skupna zahtevnost $O(n \log(nm))$.

Končna koda.

```

1 def tabela_mnozenj(n, m, k):
2     if k < 1 or k > m * n:
3         return None
4     else:
5         if n > m:
6             n, m = m, n
7         i = 1
8         j = m * n
9         while i < j:
10            mid = (i + j + 1) // 2
11            manjsih = sum(min(mid // i - (1 if mid % i == 0 else 0), m)
12                           for i in range(1, n + 1))
13            if manjsih > k - 1:
14                j = mid - 1
15            else:
16                i = mid
17        return i

```

11.6 Povzetek

- bisekcija prepolovi obseg in dela v času $O(\log n)$;
- rekurzivna različica nadomesti zanko s klici na ožji obseg;
- pri dveh tabelah primerjamo *terke* (priimek, ime);
- območje ponovitev dobimo z dvema bisekcijama (levi in desni rob);
- v neurejeni tabeli manjkajoči element *izračunamo* iz vsote.

Preveri se

1. Zakaj bisekcija zahteva urejeno tabelo?
2. Zakaj pri iskanju desnega roba zaokrožimo sredino navzgor?
3. Kako `kdo_manjka_neurejen` deluje v času $O(n)$ in brez dodatnega prostora?

Poglavje 12

Urejanje tabel

V tem poglavju bomo spoznali dva klasična algoritma za urejanje: **urejanje z vstavljanjem** in **urejanje z zlivanjem**.

12.1 Urejanje z vstavljanjem

Cilj. `uredi_z_vstavljanjem(a)` naj tabelo `a` uredi *na mestu* (brez nove tabele). Ideja: tabelo razdelimo na urejeni del (sprej) in neurejeni del (zadaj); prvi element neurejenega dela vstavimo na pravo mesto v urejeni del.

Urejeni del na začetku vsebuje prvi element, zato se zunanja zanka začne pri indeksu 1:

```
1 def uredi_z_vstavljanjem(a):
2     for i in range(1, len(a)):
```

Element, ki ga vstavljamo, shranimo v `k`, kazalec `j` pa postavimo na konec urejenega dela:

```
1 def uredi_z_vstavljanjem(a):
2     for i in range(1, len(a)):
3         j = i - 1
4         k = a[i]
```

Dokler je element levo večji od `k`, ga premaknemo za eno mesto v desno:

```
1 def uredi_z_vstavljanjem(a):
2     for i in range(1, len(a)):
3         j = i - 1
4         k = a[i]
5         while j >= 0 and a[j] > k:
6             a[j + 1] = a[j]
7             j -= 1
```

Ko se ustavimo, na prosto mesto vpišemo `k`:

```
1     a[j + 1] = k
```

Zakaj deluje

V vsakem koraku je del `a[0:i]` urejen, `k` pa vstavimo na ustrezno mesto. Ker `k` najprej shranimo, ga pri premikanju ne izgubimo.

Končna koda.

```
1 def uredi_z_vstavljanjem(a):
2     for i in range(1, len(a)):
```

```

3     j = i - 1
4     k = a[i]
5     while j >= 0 and a[j] > k:
6         a[j + 1] = a[j]
7         j -= 1
8     a[j + 1] = k

```

12.2 Urejanje z zlivanjem

12.2.1 1. podnalog: zlij2

Cilj. `zlij2(a, b)` naj zlije dva urejena seznama v nov urejen seznam, pri čemer naj uporablja `.append` in ne tabele vnaprej določene velikosti.

Zapomnimo si dolžini in pripravimo prazen seznam ter kazalca:

```

1 def zlij2(a, b):
2     m, n = len(a), len(b)
3     c = []
4     i, j = 0, 0

```

Ponavljamo, dokler nismo prepisali vseh elementov:

```

1     while len(c) < m + n:

```

Če je na vrsti element iz `a` (ker je `a` še neizčrpan in bodisi je `b` izčrpan bodisi je `a[i]` manjši), ga dodamo:

```

1         if i < m and (j >= n or a[i] < b[j]):
2             c.append(a[i])
3             i += 1

```

Sicer dodamo element iz `b`:

```

1         else:
2             c.append(b[j])
3             j += 1
4     return c

```

Zakaj deluje

V vsakem koraku dodamo najmanjšega od preostalih čelnih elementov. Ker sta vhodna seznama urejena, je končni `c` urejen. Zanko ustavimo, ko je v `c` toliko elementov, kolikor jih je v obeh vhodih skupaj.

Končna koda.

```

1 def zlij2(a, b):
2     m, n = len(a), len(b)
3     c = []
4     i, j = 0, 0
5     while len(c) < m + n:
6         if i < m and (j >= n or a[i] < b[j]):
7             c.append(a[i])
8             i += 1
9         else:
10            c.append(b[j])
11            j += 1
12    return c

```

12.2.2 2. podnalog: sort1 in sort2

Cilj. S pomočjo zlivanja uredimo celotno tabelo. Naredimo dve različici: `sort1` uporablja zlivanje v vnaprej pripravljeno tabelo (`zlij1`), `sort2` pa zlivanje z `.append` (`zlij2`). Nato primerjamo hitrost.

Najprej zlivanje v tabelo vnaprej določene velikosti (podano v nalogi):

```

1 def zlij1(a, b):
2     c = [None] * (len(a) + len(b))
3     i, j, k = 0, 0, 0
4     while i < len(a) and j < len(b):
5         if a[i] < b[j]:
6             c[k] = a[i]
7             i = i + 1
8         else:
9             c[k] = b[j]
10            j = j + 1
11            k = k + 1
12    while i < len(a):
13        c[k] = a[i]
14        i = i + 1
15        k = k + 1
16    while j < len(b):
17        c[k] = b[j]
18        j = j + 1
19        k = k + 1
20    return c

```

Urejanje z zlivanjem razdeli tabelo na polovici, vsako uredi rekurzivno in ju zlije. `sort1` uporabi `zlij1`:

```

1 def sort1(a):
2     if len(a) <= 1:
3         return a
4     sredina = len(a) // 2
5     levo = sort1(a[:sredina])
6     desno = sort1(a[sredina:])
7     return zlij1(levo, desno)

```

`sort2` je enak, le da uporabi `zlij2`:

```

1 def sort2(a):
2     if len(a) <= 1:
3         return a
4     sredina = len(a) // 2
5     levo = sort2(a[:sredina])
6     desno = sort2(a[sredina:])
7     return zlij2(levo, desno)

```

Hitrost primerjamo s knjižnico `timeit` v samostojnem programu:

```

1 if __name__ == '__main__':
2     import timeit
3     import random
4     t = list(range(1000))
5     random.shuffle(t)
6     print(timeit.timeit("sort1(t)",
7                         number=100,
8                         setup="from __main__ import t, sort1"))
9     print(timeit.timeit("sort2(t)",

```

```
10         number=100,  
11         setup="from __main__ import t, sort2"))
```

Zakaj deluje

Urejanje z zlivanjem vsakič razdeli tabelo na dve polovici ($O(\log n)$ ravni), na vsaki ravni pa zlijemo v času $O(n)$. Skupna zahtevnost je zato $O(n \log n)$ — bistveno hitreje od urejanja z vstavljanjem, ki je v najslabšem primeru $O(n^2)$.

Končna koda.

```
1 def sort2(a):  
2     if len(a) <= 1:  
3         return a  
4     sredina = len(a) // 2  
5     levo = sort2(a[:sredina])  
6     desno = sort2(a[sredina:])  
7     return zlij2(levo, desno)
```

12.3 Povzetek

- **urejanje z vstavljanjem** dela na mestu, a v najslabšem primeru v času $O(n^2)$;
- **zlivanje** dveh urejenih seznamov je $O(n)$;
- **urejanje z zlivanjem** razdeli in zlije, zato je $O(n \log n)$;
- hitrost merimo s knjižnico `timeit`.

Preveri se

1. Zakaj pri `uredi_z_vstavljanjem` shranimo `k = a[i]`?
2. Zakaj je urejanje z zlivanjem hitrejše od vstavljanja?
3. Zakaj `zlij2` uporabi `len(c) < m + n` kot pogoj zanke?

Poglavje 13

Deli in vladaj

Deli in vladaj je pristop, pri katerem problem razdelimo na manjše podprobleme iste vrste, jih rešimo rekurzivno in rezultate sestavimo.

13.1 Barvanje ograje

Cilj. `stevilo_potez(ograja)` naj vrne najmanjše število potez, s katerimi prebarvamo ograjo, opisano z višinami deščic.

Osnovna primera: prazna ograja zahteva nič potez, ena deščica pa eno:

```
1 def stevilo_potez(ograja):  
2     if len(ograja) == 0:  
3         return 0  
4     if len(ograja) == 1:  
5         return 1
```

Najnižjo deščico prebarvamo z vodoravnimi potezami — natanko toliko, kolikor je njena višina. To je začetno število potez:

```
1 min_visina = min(ograja)  
2 poteze = min_visina
```

Nad najnižjo deščico ostanejo “otoki” višjih deščic. Vsak tak otok je nov podproblem, ki ga rešimo rekurzivno. Pripravimo prazen otok `p`:

```
1 p = []
```

Sprehodimo se po deščicah. Če je deščica višja od najnižje, jo dodamo v trenutni otok (odštejemo najnižjo višino):

```
1 for i in range(len(ograja)):  
2     if ograia[i] != min_visina:  
3         p.append(ograja[i] - min_visina)
```

Ko naletimo na najnižjo deščico, je otok končan: rekurzivno ga prebarvamo in začnemo nov otok:

```
1     else:  
2         poteze += stevilo_potez(p)  
3         p = []
```

Zadnji otok obravnavamo po zanki. Na koncu izberemo manjše od dveh možnosti: vodoravno barvanje (`poteze`) ali barvanje vsake deščice posebej (`len(ograja)`):

```
1 poteze += stevilo_potez(p)  
2 return min(poteze, len(ograja))
```

Zakaj deluje

Najnižjo raven prebarvamo vodoravno (za `min_visina` potez), nad njo pa se odprejo neodvisni otoki, ki jih rešimo rekurzivno. Včasih se bolj splača vsako deščico prebarvati navpično, zato na koncu vzamemo minimum.

Končna koda.

```

1 def stevilo_potez(ograja):
2     if len(ograja) == 0:
3         return 0
4     if len(ograja) == 1:
5         return 1
6     min_visina = min(ograja)
7     poteze = min_visina
8     p = []
9     for i in range(len(ograja)):
10        if ograia[i] != min_visina:
11            p.append(ograja[i] - min_visina)
12        else:
13            poteze += stevilo_potez(p)
14            p = []
15    poteze += stevilo_potez(p)
16    return min(poteze, len(ograja))

```

13.2 Bitna slika

Cilj. kodiraj(`bitna_slika`) naj bitno sliko (seznam nizov z znaki '0' in '1') zakodira po pravilu četrtin: če je podmatrika sama iz ničel ali samih enic, jo kodira z '0' ali '1'; sicer jo razdeli na štiri podmatrike in vrne 'D' ter štiri kode.

Najprej pomožna funkcija prešteje ničle in enice v podmatriki med vrsticami [`a`, `b`) in stolpci [`c`, `d`). Pripravimo števec:

```

1 def stej(bitna_slika, a, b, c, d):
2     st_nicel, st_enic = 0, 0

```

Pregledamo vse celice podmatrike:

```

1     for i in range(a, b):
2         for j in range(c, d):
3             if bitna_slika[i][j] == '0':
4                 st_nicel += 1
5             else:
6                 st_enic += 1
7     return st_nicel, st_enic
\end{lsthighlight}

```

Zdaj `\py{kodiraj}`. Če meje niso podane, privzamemo celotno sliko:

```

12 \lsthight{11,12,13}
13 \begin{lstlisting}
14 def kodiraj(bitna_slika, a=0, b=None, c=0, d=None):
15     if b is None:
16         b = len(bitna_slika)

```

Če je podmatrika prazna, vrnemo prazen niz:

```

1  if b <= a:
2      return ''
3  if d is None:
4      d = len(bitna_slika[0])
5  if d <= c:
6      return ''
7  \end{lsthighlight}
8
9  Preštejemo ničle in enice. Če je samih enic, vrnemo \py{'1'}; če samih
10 ničel, \py{'0'}:
11
12 \lsthighlight{19,20,21,22,23}
13 \begin{lstlisting}
14     st_nicel, st_enic = stej(bitna_slika, a, b, c, d)
15     if st_nicel == 0:
16         return '1'
17     if st_enic == 0:
18         return '0'

```

Sicer matriko razdelimo na štiri četrtine. Če je vrstic liho, sta zgornji polovici večji (zato + 1); enako za stolpce:

```

1  ab_mid = (a + b + 1) // 2
2  cd_mid = (c + d + 1) // 2

```

Vrnemo 'D' in štiri kode: zgornjo levo, zgornjo desno, spodnjo levo in spodnjo desno:

```

1  return ('D'
2      + kodiraj(bitna_slika, a, ab_mid, c, cd_mid)
3      + kodiraj(bitna_slika, a, ab_mid, cd_mid, d)
4      + kodiraj(bitna_slika, ab_mid, b, c, cd_mid)
5      + kodiraj(bitna_slika, ab_mid, b, cd_mid, d))

```

Zakaj deluje

Razdelitev na štiri četrtine je rekurzivna (del in vladaj). Zaokrožitev $(a + b + 1) // 2$ poskrbi, da sta pri lihih dimenzijah zgornja oz. leva polovica večji, kot zahteva naloga.

Končna koda.

```

1  def stej(bitna_slika, a, b, c, d):
2      st_nicel, st_enic = 0, 0
3      for i in range(a, b):
4          for j in range(c, d):
5              if bitna_slika[i][j] == '0':
6                  st_nicel += 1
7              else:
8                  st_enic += 1
9      return st_nicel, st_enic
10
11 def kodiraj(bitna_slika, a=0, b=None, c=0, d=None):
12     if b is None:
13         b = len(bitna_slika)
14     if b <= a:
15         return ''
16     if d is None:
17         d = len(bitna_slika[0])
18     if d <= c:

```

```
19     return ''
20     st_nicel, st_enic = stej(bitna_slika, a, b, c, d)
21     if st_nicel == 0:
22         return '1'
23     if st_enic == 0:
24         return '0'
25     ab_mid = (a + b + 1) // 2
26     cd_mid = (c + d + 1) // 2
27     return ('D'
28           + kodiraj(bitna_slika, a, ab_mid, c, cd_mid)
29           + kodiraj(bitna_slika, a, ab_mid, cd_mid, d)
30           + kodiraj(bitna_slika, ab_mid, b, c, cd_mid)
31           + kodiraj(bitna_slika, ab_mid, b, cd_mid, d))
```

13.3 Povzetek

- **deli in vladaaj:** problem razdelimo, rešimo podprobleme in sestavimo rezultat;
- pri `stevilo_potez` vodoravno prebarvamo najnižjo raven, otoke nad njo pa rešimo rekurzivno;
- pri `kodiraj` matriko razdelimo na štiri četrtine (kvadrantno drevo).

Preveri se

1. Zakaj pri `stevilo_potez` na koncu vzamemo minimum?
2. Zakaj pri delitvi na četrtine uporabimo `(a + b + 1) // 2`?
3. Kdaj `kodiraj` vrne '0' in kdaj '1'?

Poglavje 14

Dinamično programiranje

Dinamično programiranje (DP) je pristop, pri katerem problem razbijemo na podprobleme, njihove rezultate *shranimo* in jih sestavimo od najmanjših proti večjim. Tako se vsak podproblem reši le enkrat.

14.1 Minsko polje

Cilj. `stevilo_poti(n, m, mine)` naj vrne število poti iz zgornjega levega v spodnji desni vogal polja $n \times m$, če se premikamo samo navzdol ali desno in se izognemo minam.

Ideja: naj bo `s[i][j]` število poti od polja (i, j) do cilja. Tabelo naredimo z gnezdeno izpeljavo:

```
1 def stevilo_poti(n, m, mine):
2     s = [[0 for j in range(m)] for i in range(n)]
```

Na cilju (spodnji desni vogal) smo “že na cilju”, zato je tam ena pot:

```
1     s[n - 1][m - 1] = 1
```

Tabelo polnimo od spodaj navzgor in od desne proti levi (tako so vsi podatki, ki jih potrebujemo, že izračunani):

```
1     for i in range(n - 1, -1, -1):
2         for j in range(m - 1, -1, -1):
```

Če na polju ni mine, seštejemo poti iz polja pod njim in iz polja desno od njega (če obstajata):

```
1         if (i, j) not in mine:
2             if i < n - 1:
3                 s[i][j] += s[i + 1][j]
4             if j < m - 1:
5                 s[i][j] += s[i][j + 1]
```

Rezultat je vrednost v zgornjem levem vogalu:

```
1     return s[0][0]
```

Zakaj deluje

Vsaka pot iz (i, j) se začne bodisi s korakom navzdol bodisi desno, zato je $s[i][j] = s[i+1][j] + s[i][j+1]$. Na minah pustimo vrednost 0, s čimer se take poti samodejno izločijo. Rešitev je v `s[0][0]`.

Končna koda.

```

1 def stevilo_poti(n, m, mine):
2     s = [[0 for j in range(m)] for i in range(n)]
3     s[n - 1][m - 1] = 1
4     for i in range(n - 1, -1, -1):
5         for j in range(m - 1, -1, -1):
6             if (i, j) not in mine:
7                 if i < n - 1:
8                     s[i][j] += s[i + 1][j]
9                 if j < m - 1:
10                    s[i][j] += s[i][j + 1]
11     return s[0][0]

```

14.2 Vlečenje vrvi

14.2.1 1. podnalog: razdeli (sestopanje)

Cilj. `razdeli(mase)` naj vrne maso manjše skupine pri najbolj pravični delitvi. Rešujemo s sestopanjem.

Skupno maso izračunamo vnaprej:

```

1 def razdeli(mase):
2     skupaj = sum(mase)

```

Pomožna funkcija `deli(levi, i)` preizkusi vse možnosti. Če smo obravnavali vse udeležence, vrnemo maso leve skupine:

```

1 def deli(levi, i):
2     if i >= len(mase):
3         return levi

```

Če bi *i*-ti udeleženec presegel polovico, ga damo na desno (sicer bi leva skupina postala težja):

```

1     if levi + mase[i] > skupaj // 2:
2         return deli(levi, i + 1)

```

Sicer preizkusimo obe možnosti: da *i*-tega damo na levo ali na desno:

```

1     else:
2         resitev1 = deli(levi + mase[i], i + 1)
3         resitev2 = deli(levi, i + 1)

```

Izberemo tisto, pri kateri je razlika med skupinama manjša:

```

1         if abs(skupaj - 2 * resitev1) < abs(skupaj - 2 * resitev2):
2             return resitev1
3         else:
4             return resitev2
5     return deli(0, 0)

```

Zakaj deluje

`abs(skupaj - 2 * resitev)` je razlika med skupinama, če ima ena maso `resitev`. Manjša razlika pomeni pravičnejšo delitev.

Končna koda.

```

1 def razdeli(mase):
2     skupaj = sum(mase)
3
4     def deli(levi, i):
5         if i >= len(mase):
6             return levi
7         if levi + mase[i] > skupaj // 2:
8             return deli(levi, i + 1)
9         else:
10            resitev1 = deli(levi + mase[i], i + 1)
11            resitev2 = deli(levi, i + 1)
12            if abs(skupaj - 2 * resitev1) < abs(skupaj - 2 * resitev2):
13                return resitev1
14            else:
15                return resitev2
16     return deli(0, 0)

```

14.2.2 2. podnalog: razdeli dinamično

Cilj. Enako, a z dinamičnim programiranjem (problem 0-1 nahrbtnika).

Vzdrževali bomo množico vseh dosegljivih mas ene skupine. Na začetku je dosegljiva le masa 0 (prazna skupina):

```

1 def razdeli_dinamicno(mase):
2     mozna = set()
3     mozna.add(0)

```

Za vsakega udeleženca z maso k dodamo vse nove mase $i + k$. Ker med spreminjanjem ne smemo brati novih vrednosti, se sprehodimo po *kopiji*:

```

1     for k in mase:
2         for i in mozna.copy():
3             mozna.add(i + k)

```

Iščemo največjo maso, ki ne presega polovice skupne mase:

```

1     i = sum(mase) // 2
2     while i not in mozna:
3         i -= 1
4     return i

```

Zakaj deluje

Množica *mozna* vsebuje vse mase, ki jih lahko doseže ena skupina. Ker iščemo čim bolj izenačeni skupini, je masa manjše skupine največja dosegljiva masa, ki ne presega polovice skupne mase. Pomožna zanka teče po kopiji, da novih mas ne uporabimo dvakrat za istega udeleženca.

Končna koda.

```

1 def razdeli_dinamicno(mase):
2     mozna = set()
3     mozna.add(0)
4     for k in mase:
5         for i in mozna.copy():
6             mozna.add(i + k)

```

```

7   i = sum(mase) // 2
8   while i not in mozna:
9       i -= 1
10  return i

```

14.2.3 3. podnalog: razdeli udeležence

Cilj. Vrniti še seznam mas udeležencev, ki sestavljajo manjšo skupino.

Namesto množice mas vodimo slovar `skupine`, ki vsaki masi priredi seznam udeležencev. Prazna skupina ima maso 0:

```

1 def razdeli_udelezence(mase):
2     skupine = {0: []}

```

Za vsakega udeleženca z maso `k` zgradimo nove skupine, ki mu dodajo `k`:

```

1     for k in mase:
2         nove = {}
3         for (i, s) in skupine.items():
4             nove[i + k] = s + [k]
5         skupine.update(nove)

```

Poiščemo največjo maso do polovice in vrnemo pripadajočo skupino:

```

1     i = sum(mase) // 2
2     while i not in skupine:
3         i -= 1
4     return skupine[i]

```

Zakaj deluje

Novi pari se zberejo v `nove` in se šele nato dodajo v `skupine`, da iste udeležence uporabimo največ enkrat (problem 0-1). Na koncu vrnemo seznam udeležencev pri največji dosegljivi masi.

Končna koda.

```

1 def razdeli_udelezence(mase):
2     skupine = {0: []}
3     for k in mase:
4         nove = {}
5         for (i, s) in skupine.items():
6             nove[i + k] = s + [k]
7         skupine.update(nove)
8     i = sum(mase) // 2
9     while i not in skupine:
10        i -= 1
11    return skupine[i]

```

14.3 Povzetek

- **dinamično programiranje** shrani rezultate podproblemov in jih sestavi od najmanjših proti večjim;
- pri `stevilo_poti` tabelo polnimo od cilja nazaj;

- **problem 0-1 nahrbtnika** rešimo z množico ali slovarjem dosegljivih mas;
- pri prehodu po množici/slovarju uporabimo kopijo, da vsak element uporabimo največ enkrat.

Preveri se

1. Zakaj tabelo `s` polnimo od spodaj navzgor in od desne proti levi?
2. Zakaj pri `razdeli_dinamicno` zanka teče po `mozna.copy()`?
3. Zakaj je dinamično programiranje hitrejše od sestopanja?

Poglavje 15

Objektno programiranje

V tem poglavju bomo spoznali **razrede** in **objekte**. Razred je načrt, objekt pa konkretna stvar te vrste. Objekt ima *atribute* (podatke) in *metode* (funkcije, ki delujejo na podatkih).

Novo v tem razdelku

Razred in self.

```
1 class Tocka:
2     def __init__(self, x, y):    # konstruktor
3         self.x = x              # atribut x
4         self.y = y              # atribut y
```

`__init__` se pokliče ob ustvarjanju objekta (`Tocka(1, 2)`). `self` je objekt sam; prek njega dostopamo do atributov (`self.x`).

15.1 Datumi

15.1.1 Pomožne funkcije

1. podnalog: `je_prestopno`. Leto je prestopno, če je deljivo s 4 in ne s 100, ali pa je deljivo s 400:

```
1 def je_prestopno(leto):
2     return (leto % 4 == 0 and leto % 100 != 0) or leto % 400 == 0
```

2. podnalog: `stevilo_dni`. Prestopno leto ima 366 dni, sicer 365:

```
1 def stevilo_dni(leto):
2     return 365 if not je_prestopno(leto) else 366
```

3. podnalog: `dolzine_mesecev`. Vrnemo seznam dolžin vseh mesecev; februar je odvisen od prestopnosti:

```
1 def dolzine_mesecev(leto):
2     return [
3         31, (29 if je_prestopno(leto) else 28), 31, 30, 31, 30,
4         31, 31, 30, 31, 30, 31
5     ]
```

15.1.2 Razred Datum

4. podnalog: `konstruktor`. Objekt ima tri atribute: dan, mesec, leto:

```

1 class Datum:
2     def __init__(self, dan, mesec, leto):
3         self.dan = dan
4         self.mesec = mesec
5         self.leto = leto

```

5. podnaloga: `__str__`. Ta metoda določi, kako se objekt izpiše s `print`:

```

1     def __str__(self):
2         return '{0}. {1}. {2}'.format(self.dan, self.mesec, self.leto)

```

6. podnaloga: `__repr__`. Ta metoda določi, kako se objekt izpiše v konzoli (in v seznamih):

```

1     def __repr__(self):
2         return 'Datum({0}, {1}, {2})'.format(self.dan, self.mesec, self.leto)

```

7. podnaloga: `je_veljaven`. Datum je veljaven, če je mesec med 1 in 12 in dan ne presega dolžine meseca:

```

1     def je_veljaven(self):
2         return (1 <= self.mesec <= 12 and
3                 1 <= self.dan <= dolzine_mesecev(self.leto)[self.mesec - 1])

```

8. podnaloga: `__lt__`. Za primerjanje datumov uporabimo kar leksikografsko primerjavo terke (leto, mesec, dan):

```

1     def __lt__(self, other):
2         return (self.leto, self.mesec, self.dan) < (other.leto, other.mesec, other.dan)

```

9. podnaloga: `__eq__`. Datuma sta enaka, če se ujemajo vsi trije podatki:

```

1     def __eq__(self, other):
2         return (self.leto == other.leto and
3                 self.mesec == other.mesec and
4                 self.dan == other.dan)

```

10. podnaloga: `dan_v_leto`. Seštejemo dolžine mesecev pred tem mesecem in prištejemo dan:

```

1     def dan_v_leto(self):
2         return sum(dolzine_mesecev(self.leto)[:self.mesec - 1]) + self.dan

```

11. podnaloga: `dni_od_zacetka` in `razlika`. Preštejemo dni od začetka štetja (leto 0): dnevi v tem mesecu, dnevi v tem letu pred tem mesecem, 365 na vsako prejšnje leto in popravek za prestopna leta:

```

1     def dni_od_zacetka(self):
2         return (self.dan
3                 + sum(dolzine_mesecev(self.leto)[:self.mesec - 1])
4                 + 365 * (self.leto - 1)
5                 + (self.leto - 1) // 4
6                 - (self.leto - 1) // 100
7                 + (self.leto - 1) // 400)

```

Razlika je preprosto razlika obeh števil dni:

```

1     def razlika(self, other):
2         return self.dni_od_zacetka() - other.dni_od_zacetka()

```

Zakaj deluje

Namesto zanke po vseh letih (ki bi bila počasna) prestopna leta preštejemo z deljenjem: vsako četrto leto je prestopno, odštejemo vsako stoto in prištejemo vsako štiristo. Tako je izračun konstanten.

12. podnalog: `dan_v_tednu`. Uporabimo Zellerjev obrazec:

```

1  def dan_v_tednu(self):
2      q = self.dan
3      if self.mesec > 2:
4          m, l = self.mesec, self.leto
5      else:
6          m, l = self.mesec + 12, self.leto - 1
7      j = l // 100
8      k = l % 100
9      h = (q + ((m + 1) * 13) // 5 + k + k // 4 + j // 4 - 2 * j) % 7
10     return (h - 2) % 7 + 1

```

13. podnalog: `dan_v_letu_stat`. To je *statična* metoda (ne potrebuje objekta). Iz zaporedne številke dneva ugotovimo mesec in dan:

```

1  @staticmethod
2  def dan_v_letu_stat(leto, dan):
3      dolzine = dolzine_mesecev(leto)
4      mesec = 1
5      while dan > dolzine[mesec - 1]:
6          dan -= dolzine[mesec - 1]
7          mesec += 1
8      return Datum(dan, mesec, leto)

```

Novo v tem razdelku

Metode, ki jih Python prepozna po imenu. `__init__` (konstruktor), `__str__` (izpis), `__repr__` (predstavitev), `__eq__` (`==`), `__lt__` (`<`), `__add__` (`+`), `__mul__` (`*`), `__call__` (klic objekta). Take metode imenujemo *dunder* (double underscore).

Končna koda (razred `Datum`).

```

1  class Datum:
2      def __init__(self, dan, mesec, leto):
3          self.dan = dan
4          self.mesec = mesec
5          self.leto = leto
6
7      def __str__(self):
8          return '{0}. {1}. {2}'.format(self.dan, self.mesec, self.leto)
9
10     def __repr__(self):
11         return 'Datum({0}, {1}, {2})'.format(self.dan, self.mesec, self.leto)
12
13     def je_veljaven(self):
14         return (1 <= self.mesec <= 12 and
15               1 <= self.dan <= dolzine_mesecev(self.leto)[self.mesec - 1])
16
17     def __lt__(self, other):
18         return (self.leto, self.mesec, self.dan) < (other.leto, other.mesec, other.dan)

```

```

19
20 def __eq__(self, other):
21     return (self.letu == other.letu and
22             self.mesec == other.mesec and
23             self.dan == other.dan)
24
25 def dan_v_letu(self):
26     return sum(dolzine_mesecev(self.letu)[:self.mesec - 1]) + self.dan
27
28 def dni_od_zacetka(self):
29     return (self.dan
30             + sum(dolzine_mesecev(self.letu)[:self.mesec - 1])
31             + 365 * (self.letu - 1)
32             + (self.letu - 1) // 4
33             - (self.letu - 1) // 100
34             + (self.letu - 1) // 400)
35
36 def razlika(self, other):
37     return self.dni_od_zacetka() - other.dni_od_zacetka()
38
39 def dan_v_tednu(self):
40     q = self.dan
41     if self.mesec > 2:
42         m, l = self.mesec, self.letu
43     else:
44         m, l = self.mesec + 12, self.letu - 1
45     j = l // 100
46     k = l % 100
47     h = (q + ((m + 1) * 13) // 5 + k + k // 4 + j // 4 - 2 * j) % 7
48     return (h - 2) % 7 + 1
49
50 @staticmethod
51 def dan_v_letu_stat(letu, dan):
52     dolzine = dolzine_mesecev(letu)
53     mesec = 1
54     while dan > dolzine[mesec - 1]:
55         dan -= dolzine[mesec - 1]
56         mesec += 1
57     return Datum(dan, mesec, letu)

```

15.2 Ulomki

1. podnalogi: konstruktor. Ulomek hranimo okrajšan, imenovalc pa pozitiven. Najprej poskrbimo za pozitivni imenovalc, nato okrajšamo z največjim skupnim deliteljem:

```

1 class Ulomek:
2     def __init__(self, st, im):
3         if im < 0:
4             st, im = -st, -im
5         d = gcd(st, im)
6         self.st = st // d
7         self.im = im // d

```

Novo v tem razdelku

from fractions import gcd. Funkcija gcd(a, b) vrne največji skupni delitelj; z njim okrajšamo ulomek.

2. in 3. podnalogi: `__str__` in `__repr__`.

```

1 def __str__(self):
2     return '{0}/{1}'.format(self.st, self.im)
3
4 def __repr__(self):
5     return 'Ulomek({0}, {1})'.format(self.st, self.im)

```

4. podnalogi: `__eq__`. Ker sta ulomka vedno okrajšana, zadošča primerjati števec in imenovalc:

```

1 def __eq__(self, other):
2     return self.st == other.st and self.im == other.im

```

5. in 6. podnalogi: `__add__` in `__sub__`. Za skupni imenovalc uporabimo produkt imenovalcev:

```

1 def __add__(self, other):
2     a, b = self.st, self.im
3     c, d = other.st, other.im
4     return Ulomek(a * d + b * c, b * d)
5
6 def __sub__(self, other):
7     a, b = self.st, self.im
8     c, d = other.st, other.im
9     return Ulomek(a * d - b * c, b * d)

```

7. in 8. podnalogi: `__mul__` in `__truediv__`. Pri deljenju ulomek obrnemo:

```

1 def __mul__(self, other):
2     a, b = self.st, self.im
3     c, d = other.st, other.im
4     return Ulomek(a * c, b * d)
5
6 def __truediv__(self, other):
7     a, b = self.st, self.im
8     c, d = other.st, other.im
9     return Ulomek(a * d, b * c)

```

9. podnalogi: približek. Vsoto $1 + 1/1! + 1/2! + \dots + 1/n!$ seštejemo z ulomki. `sum` lahko prejme začetno vrednost `Ulomek(0, 1)`:

```

1 def približek(n):
2     def fakulteta(i):
3         return 1 if i == 0 else i * fakulteta(i - 1)
4     return sum((Ulomek(1, fakulteta(i)) for i in range(n + 1)), Ulomek(0, 1))

```

Končna koda (razred `Ulomek`).

```

1 class Ulomek:
2     def __init__(self, st, im):
3         if im < 0:
4             st, im = -st, -im
5         d = gcd(st, im)
6         self.st = st // d

```

```

7         self.im = im // d
8
9     def __str__(self):
10         return '{0}/{1}'.format(self.st, self.im)
11
12     def __repr__(self):
13         return 'Ulomek({0}, {1})'.format(self.st, self.im)
14
15     def __eq__(self, other):
16         return self.st == other.st and self.im == other.im
17
18     def __add__(self, other):
19         a, b = self.st, self.im
20         c, d = other.st, other.im
21         return Ulomek(a * d + b * c, b * d)
22
23     def __sub__(self, other):
24         a, b = self.st, self.im
25         c, d = other.st, other.im
26         return Ulomek(a * d - b * c, b * d)
27
28     def __mul__(self, other):
29         a, b = self.st, self.im
30         c, d = other.st, other.im
31         return Ulomek(a * c, b * d)
32
33     def __truediv__(self, other):
34         a, b = self.st, self.im
35         c, d = other.st, other.im
36         return Ulomek(a * d, b * c)

```

15.3 Polinomi

Polinom predstavimo s seznamom koeficientov od konstantnega člena navzgor.

1. podnaloga: konstruktor. Odstranimo vodilne ničle in naredimo kopijo seznama (da zunanje spremembe ne vplivajo na objekt):

```

1 class Polinom:
2     def __init__(self, koef):
3         zadnji = len(koef)
4         while zadnji > 0 and koef[zadnji - 1] == 0:
5             zadnji -= 1
6         self.koef = koef[:zadnji]

```

2. podnaloga: stopnja. Stopnja je dolžina seznama minus 1; ničelni polinom ima stopnjo $-\infty$:

```

1     def stopnja(self):
2         return len(self.koef) - 1 if self.koef else float('-inf')

```

3. in 4. podnalogi: `__repr__` in `__eq__`.

```

1     def __repr__(self):
2         return 'Polinom({0})'.format(self.koef)
3
4     def __eq__(self, other):
5         return self.koef == other.koef

```

5. podnalog: `__call__`. Vrednost izračunamo s Hornerjevim algoritmom: koeficiente vzamemo od vodilnega navzdol:

```

1 def __call__(self, x):
2     vrednost = 0
3     for k in reversed(self.koef):
4         vrednost = vrednost * x + k
5     return vrednost

```

Zakaj deluje

Hornerjev algoritem izračuna $a_n x^n + \dots + a_0$ z n množenji (namesto potenciranja), tako da postopoma množi in prišteva koeficiente.

6. podnalog: `__add__`. Koeficiente seštejemo po mestih. Poskrbimo, da ima levi polinom vsaj toliko členov kot desni:

```

1 def __add__(self, other):
2     if len(self.koef) < len(other.koef):
3         return other + self
4     koef_vsote = list(self.koef)
5     for i, a in enumerate(other.koef):
6         koef_vsote[i] += a
7     return Polinom(koef_vsote)

```

7. podnalog: `__mul__`. Produkt izračunamo s konvolucijo koeficientov; seznama podaljšamo z ničlami:

```

1 def __mul__(self, other):
2     if not self.koef or not other.koef:
3         return Polinom([])
4     levi = self.koef + len(other.koef) * [0]
5     desni = other.koef + len(self.koef) * [0]
6     koef_prod = [sum(levi[i] * desni[n - i] for i in range(n + 1))
7                  for n in range(self.stopnja() + other.stopnja() + 1)]
8     return Polinom(koef_prod)

```

8. podnalog: `odvod`. Odvod zniža stopnjo za 1; k-ti odvod ponovimo k-krat:

```

1 def odvod(self, k=1):
2     if k == 0:
3         return Polinom(self.koef)
4     q = Polinom([]) if self.stopnja() < 1 else \
5         Polinom([self.koef[i] * i for i in range(1, self.stopnja() + 1)])
6     if k == 1:
7         return q
8     return q.odvod(k - 1)

```

9. podnalog: `__str__`. Sestavimo čitljiv niz. Pomožna funkcija monom oblikuje en člen, nato jih združimo:

```

1 def __str__(self):
2     def monom(a, n):
3         monom = '{0}x^{1}'.format(a, n)
4         monom = monom.replace('x^1', 'x')
5         monom = monom.replace('x^0', '')
6         monom = monom.replace('1x', '[x]')
7         monom = monom.replace('-1x', '-x')
8         return monom.replace('[', '').replace(']', '')

```

Združimo monome z neničelnimi koeficienti (obrnjene, da je vodilni spredaj) in popravimo znake. Če je niz prazen, gre za ničelni polinom:

```

1 niz = ' + '.join(reversed([monom(a, n)
2                       for n, a in enumerate(self.koef) if a != 0]))
3 niz = niz.replace('+ -', '- ')
4 return niz or '0'

```

Končna koda (razred Polinom).

```

1 class Polinom:
2     def __init__(self, koef):
3         zadnji = len(koef)
4         while zadnji > 0 and koef[zadnji - 1] == 0:
5             zadnji -= 1
6         self.koef = koef[:zadnji]
7
8     def stopnja(self):
9         return len(self.koef) - 1 if self.koef else float('-inf')
10
11    def __repr__(self):
12        return 'Polinom({0})'.format(self.koef)
13
14    def __eq__(self, other):
15        return self.koef == other.koef
16
17    def __call__(self, x):
18        vrednost = 0
19        for k in reversed(self.koef):
20            vrednost = vrednost * x + k
21        return vrednost
22
23    def __add__(self, other):
24        if len(self.koef) < len(other.koef):
25            return other + self
26        koef_vsote = list(self.koef)
27        for i, a in enumerate(other.koef):
28            koef_vsote[i] += a
29        return Polinom(koef_vsote)
30
31    def __mul__(self, other):
32        if not self.koef or not other.koef:
33            return Polinom([])
34        levi = self.koef + len(other.koef) * [0]
35        desni = other.koef + len(self.koef) * [0]
36        koef_prod = [sum(levi[i] * desni[n - i] for i in range(n + 1))
37                     for n in range(self.stopnja() + other.stopnja() + 1)]
38        return Polinom(koef_prod)
39
40    def odvod(self, k=1):
41        if k == 0:
42            return Polinom(self.koef)
43        q = Polinom([]) if self.stopnja() < 1 else \
44            Polinom([self.koef[i] * i for i in range(1, self.stopnja() + 1)])
45        if k == 1:
46            return q
47        return q.odvod(k - 1)
48
49    def __str__(self):

```

```
50     def monom(a, n):
51         monom = '{0}x^{1}'.format(a, n)
52         monom = monom.replace('x^1', 'x')
53         monom = monom.replace('x^0', '')
54         monom = monom.replace('1x', 'x')
55         monom = monom.replace('-1x', '-x')
56         return monom.replace('[', '').replace(']', '')
57     niz = '+ '.join(reversed([monom(a, n)
58                             for n, a in enumerate(self.koef) if a != 0]))
59     niz = niz.replace('+ -', '- ')
60     return niz or '0'
```

15.4 Povzetek

- **razred** opisuje objekte; `__init__` nastavi attribute;
- **self** je objekt sam;
- **dunder** metode (`__str__`, `__eq__`, `__lt__`, `__add__`, `__mul__`, `__call__`) omogočijo operatorje;
- objekti so *nespremenljivi* le, če jih takšne naredimo (kopija seznama v `Polinom`).

Preveri se

1. Kaj je razlika med `__str__` in `__repr__`?
2. Zakaj `Ulolek` v konstruktorju okrajša ulomek?
3. Zakaj `Polinom` naredi kopijo seznama koeficientov?

Poglavje 16

Iteratorji in generatorji

Generator je funkcija, ki vrednosti proizvaja sproti z `yield`. Vsakič, ko jo pokličemo, nadaljuje tam, kjer je nazadnje obstala. **Iterator** pa je objekt, po katerem lahko tečemo z zanko `for`; zanj morata biti definirani metodi `__iter__` in `__next__`.

Novo v tem razdelku

yield. Generator se ob vsakem `yield` “ustavi” in vrne vrednost; ob naslednjem klicu nadaljuje. Primer:

```
1 def odstevaj(n):
2     while n > 0:
3         yield n
4         n -= 1
5
6 list(odstevaj(3))    # [3, 2, 1]
```

16.1 Enostavne naloge z generatorji

16.1.1 1. podnaloge: številke

Cilj. `številke(n)` naj generira številke števila `n` od enic navzgor.

Ponavljamo, dokler ima `n` še številko. Zadnjo številko vrnemo z `yield`, nato jo odstranimo:

```
1 def številke(n):
2     while n > 0:
3         yield n % 10
4         n //= 10
```

Konzola

```
>>> list(številke(1337))
[7, 3, 3, 1]
```

Končna koda.

```
1 def številke(n):
2     while n > 0:
3         yield n % 10
4         n //= 10
```

16.1.2 2. podnalog: delitelji

Cilj. `delitelji(n)` naj generira delitelje od najmanjšega proti največjemu.

Manjše delitelje (do $\sqrt{n}$) vračamo sproti, večje pa shranjujemo v pomožni seznam, da jih na koncu vrnemo v nasprotnem vrstnem redu. Pripravimo seznam in kandidat:

```
1 def delitelji(n):
2     d = []
3     i = 1
```

Preizkušamo delitelje do $\sqrt{n}$:

```
1 while i**2 < n:
```

Če i deli n , vrnemo i , večjega delitelja pa shranimo:

```
1     if n % i == 0:
2         yield i
3         d.append(n // i)
4     i += 1
```

Če je n popoln kvadrat, je srednji delitelj i :

```
1 if i**2 == n:
2     yield i
```

Na koncu vrnemo še večje delitelje v obratnem vrstnem redu:

```
1 while len(d) > 0:
2     yield d.pop()
```

Konzola

```
>>> list(delitelji(12))
[1, 2, 3, 4, 6, 12]
```

Končna koda.

```
1 def delitelji(n):
2     d = []
3     i = 1
4     while i**2 < n:
5         if n % i == 0:
6             yield i
7             d.append(n // i)
8         i += 1
9     if i**2 == n:
10        yield i
11    while len(d) > 0:
12        yield d.pop()
```

16.1.3 3. podnalog: potence naravnih

Cilj. `potence_naravnih(k)` naj generira neskončno zaporedje $1^k, 2^k, 3^k, \dots$

Zanka `while True` se nikoli ne ustavi; generator torej proizvaja v nedogled:

```
1 def potence_naravnih(k):
2     i = 0
```

```

3 while True:
4     i += 1
5     yield i**k

```

Končna koda.

```

1 def potence_naravnih(k):
2     i = 0
3     while True:
4         i += 1
5         yield i**k

```

16.1.4 4. podnalog: fakultete

Cilj. fakultete(n) naj generira neskončno zaporedje $n!$, $(n+1)!$, $(n+2)!$, ...

Če je n enak 0, najprej vrnemo 1 in nadaljujemo od 1:

```

1 def fakultete(n):
2     if n == 0:
3         yield 1
4         n += 1

```

Izračunamo $n!$:

```

1 ans = 1
2 for x in range(1, n):
3     ans *= x

```

Nato v nedogled množimo z naslednjim številom in vračamo:

```

1 while True:
2     ans *= n
3     yield ans
4     n += 1

```

Končna koda.

```

1 def fakultete(n):
2     if n == 0:
3         yield 1
4         n += 1
5     ans = 1
6     for x in range(1, n):
7         ans *= x
8     while True:
9         ans *= n
10        yield ans
11        n += 1

```

16.1.5 5. podnalog: collatz

Cilj. collatz(n) naj generira Collatzovo zaporedje od n do 1.

Ponavljamo, dokler n ni 1. Vsak člen vrnemo in nato izračunamo naslednjega:

```

1 def collatz(n):
2     while n > 1:
3         yield n
4         if n % 2 == 0:
5             n //= 2

```

```

6         else:
7             n = 3 * n + 1

```

Na koncu vrnemo še končno enico:

```

1     yield 1

```

Konzola

```

>>> list(collatz(20))
[20, 10, 5, 16, 8, 4, 2, 1]

```

Končna koda.

```

1 def collatz(n):
2     while n > 1:
3         yield n
4         if n % 2 == 0:
5             n //= 2
6         else:
7             n = 3 * n + 1
8     yield 1

```

16.2 Jožef Flavij

16.2.1 1. podnaloga: konstruktor

Cilj. Razred Izstevanka naj izštevaja ljudi v krogu: vsakega k -tega odstranimo. Konstruktor naj shrani imena in k , kazalec p pa naj bo na začetku 0.

```

1 class Izstevanka:
2     def __init__(self, imena, k):
3         self.imena = imena
4         self.k = k
5         self.p = 0

```

16.2.2 2. podnaloga: `__iter__`

Cilj. Iterator mora imeti metodo `__iter__`, ki vrne `self`:

```

1     def __iter__(self):
2         return self

```

16.2.3 3. podnaloga: `__next__`

Cilj. `__next__` vrne naslednje ime in posodobi stanje; ko zmanjka imen, sproži izjemo `StopIteration`.

Če ni več imen, sprožimo izjemo:

```

1     def __next__(self):
2         if len(self.imena) == 0:
3             raise StopIteration

```

Premaknemo kazalec p za $k - 1$ mest naprej (mod dolžina kroga) in vrnemo ime na tem mestu (ter ga odstranimo):

```

1     self.p = (self.p + (self.k - 1)) % len(self.imena)
2     return self.imena.pop(self.p)

```

Zakaj deluje

Ker po odstranitvi elementa naslednji element “zdrsne” na isto mesto, se premaknemo le za $k - 1$. Modul poskrbi, da ostanemo znotraj kroga.

16.2.4 4. podnalog: generator izstevanka

Cilj. Enako še kot generator.

Naredimo kopijo seznama (da ne pokvarimo vhodnega) in kazalec:

```

1 def izstevanka(imena, k):
2     seznam = list(imena)
3     i = 0

```

Dokler je v krogu kdo, se premaknemo za $k - 1$ in vrnemo ter odstranimo ime:

```

1     while len(seznam) > 0:
2         i += k - 1
3         i %= len(seznam)
4     yield seznam.pop(i)

```

Konzola

```

>>> list(izstevanka(range(1, 8), 3))
[3, 6, 2, 7, 5, 1, 4]

```

Končna koda.

```

1 class Izstevanka:
2     def __init__(self, imena, k):
3         self.imena = imena
4         self.k = k
5         self.p = 0
6
7     def __iter__(self):
8         return self
9
10    def __next__(self):
11        if len(self.imena) == 0:
12            raise StopIteration
13        self.p = (self.p + (self.k - 1)) % len(self.imena)
14        return self.imena.pop(self.p)
15
16 def izstevanka(imena, k):
17     seznam = list(imena)
18     i = 0
19     while len(seznam) > 0:
20         i += k - 1
21         i %= len(seznam)
22         yield seznam.pop(i)

```

16.3 Povzetek

- **generator** uporablja `yield` in proizvaja vrednosti sproti (tudi neskončno);
- **iterator** potrebuje `__iter__` in `__next__`, ob koncu pa sproži `StopIteration`;
- `list(generator)` porabi generator do konca.

Preveri se

1. Kakšna je razlika med `return` in `yield`?
2. Zakaj `potence_naravnih` deluje v nedogled, `collatz` pa ne?
3. Zakaj se pri izštevanki premaknemo za `k - 1` in ne `k`?

Poglavje 17

Regularni izrazi

Regularni izrazi so vzorci za iskanje in zamenjavo besedila. Modul `re` ponuja funkciji `re.findall` (najde vse ujemajoče dele) in `re.sub` (zamenja ujemajoče dele).

Novo v tem razdelku

Osnovni gradniki vzorcev.

1	<code>\w</code>	poljuben "besedni" znak (crka, številka, _)
2	<code>\b</code>	meja besede
3	<code>\d</code>	številka
4	<code>*</code>	ponovitev (nič ali večkrat)
5	<code>+</code>	ponovitev (vsaj enkrat)
6	<code>\2</code>	sklic na 2. skupino v vzorcu

Vzorec pišemo kot *surov niz*: `r'\w *'` (predpona `r` pomeni, da se poševnice ne obravnavajo posebej).

17.1 Martin Krpan z Vrha

17.1.1 1. podnalog: najdi besede

Cilj. `najdi_besede(besedilo, podniz)` naj vrne množico vseh besed, ki vsebujejo `podniz`.

Vzorec je: poljubni besedni znaki, nato iskani podniz, nato spet poljubni besedni znaki. `re.findall` vrne vse ujemanja, `set` pa jih zbere brez ponovitev:

```
1 def najdi_besede(besedilo, podniz):
2     return set(re.findall(r'\w*' + podniz + r'\w*', besedilo))
```

Konzola

```
>>> najdi_besede('Bil je silen in nosil.', 'il')
{'Bil', 'silen', 'nosil'}
```

Končna koda.

```
1 def najdi_besede(besedilo, podniz):
2     return set(re.findall(r'\w*' + podniz + r'\w*', besedilo))
```

17.1.2 2. podnalog: najdi predpono

Cilj. najdi_predpono(besedilo, predpona) naj vrne besede z dano predpono.

Dodamo mejo besede `\b` na začetek, da se ujemanje začne na začetku besede:

```
1 def najdi_predpono(besedilo, predpona):
2     return set(re.findall(r'\b' + predpona + r'\w*', besedilo))
```

Končna koda.

```
1 def najdi_predpono(besedilo, predpona):
2     return set(re.findall(r'\b' + predpona + r'\w*', besedilo))
```

17.1.3 3. podnalog: najdi pripono

Cilj. najdi_pripono(besedilo, pripona) naj vrne besede z dano pripono.

Tokrat je meja besede `\b` na koncu:

```
1 def najdi_pripono(besedilo, pripona):
2     return set(re.findall(r'\w*' + pripona + r'\b', besedilo))
```

Končna koda.

```
1 def najdi_pripono(besedilo, pripona):
2     return set(re.findall(r'\w*' + pripona + r'\b', besedilo))
```

17.1.4 4. podnalog: podvojene črke

Cilj. podvojene_crke(besedilo) naj vrne besede s podvojeno črko.

Vzorec poišče poljubno črko, ki ji takoj sledi ista črka. Oklepaji določijo skupino, `\2` pa se sklicuje na drugo skupino:

```
1 def podvojene_crke(besedilo):
2     return set(grupa[0] for grupa in re.findall(r'(\w*(\w)\2\w*)', besedilo))
```

Zakaj deluje

`(\w )` ujame poljubno črko, `\2` pa zahteva, da ji sledi ista črka. Zunanji oklepaj zajame celotno besedo, ki jo nato dodamo v množico.

Končna koda.

```
1 def podvojene_crke(besedilo):
2     return set(grupa[0] for grupa in re.findall(r'(\w*(\w)\2\w*)', besedilo))
```

17.2 Podnapisi

17.2.1 1. podnalog: pretvori

Cilj. pretvori(h, m, s, milis) naj čas pretvori v milisekunde.

Ure pretvorimo v minute, minute v sekunde, vse skupaj v milisekunde in prištejemo milisekunde:

```
1 def pretvori(h, m, s, milis):
2     return ((h * 60 + m) * 60 + s) * 1000 + milis
```

Končna koda.

```

1 def pretvori(h, m, s, milis):
2     return ((h * 60 + m) * 60) + s * 1000 + milis

```

17.2.2 2. podnalog: pretvori nazaj

Cilj. `pretvori_nazaj(milis)` naj milisekunde razstavi na ure, minute, sekunde in milisekunde.

Postopoma delimo in jemljemo ostanke. Najprej ločimo sekunde od milisekund:

```

1 def pretvori_nazaj(milis):
2     s = milis // 1000
3     milis %= 1000

```

Nato iz sekund dobimo minute:

```

1     m = s // 60
2     s %= 60

```

In iz minut ure:

```

1     h = m // 60
2     m %= 60
3     return (h, m, s, milis)

```

Končna koda.

```

1 def pretvori_nazaj(milis):
2     s = milis // 1000
3     milis %= 1000
4     m = s // 60
5     s %= 60
6     h = m // 60
7     m %= 60
8     return (h, m, s, milis)

```

17.2.3 3. podnalog: popravi

Cilj. `popravi(vhod, izhod, t, d)` naj v datoteki s podnapisi vse čase, ki so kasnejši od `t`, premakne za `d` milisekund.

Najprej pomožna funkcija `zamenjava`, ki jo bo poklical `re.sub` za vsak najden čas. Iz skupin sestavimo `h, m, s` in milisekunde:

```

1 def zamenjava(m):
2     h, m, s, milis = (int(m.group(1)), int(m.group(2)),
3                     int(m.group(3)), int(m.group(4)))
4     cas = pretvori(h, m, s, milis)

```

Če je čas za preskokom, ga premaknemo:

```

1     if cas > t:
2         cas += d

```

Rezultat vrnemo v obliki `hh:mm:ss,mmm` (dve oz. tri mesta, z vodilnimi ničlami):

```

1     return '{0:02d}:{1:02d}:{2:02d},{3:03d}'.format(*pretvori_nazaj(cas))

```

Preberemo vsebino, zamenjamo vse čase z vzorcem in zapišemo:

```

1 with open(vhod, encoding='utf-8') as f:
2     vsebina = f.read()
3     popravljena_vsebina = re.sub(r'(\d\d):(\d\d):(\d\d),(\d\d\d)', zamenjava, vsebina)
4     with open(izhod, 'w', encoding='utf-8') as f:
5         f.write(popravljena_vsebina)

```

Novo v tem razdelku

`re.sub`. `re.sub`(vzorec, funkcija, besedilo) vsako ujemanje nadomesti z rezultatom funkcije, ki ji podamo objekt `m`; `m.group(i)` vrne `i`-to skupino vzorca.

Končna koda.

```

1 def popravi(vhod, izhod, t, d):
2     def zamenjava(m):
3         h, m, s, milis = (int(m.group(1)), int(m.group(2)),
4                             int(m.group(3)), int(m.group(4)))
5         cas = pretvori(h, m, s, milis)
6         if cas > t:
7             cas += d
8         return '{0:02d}:{1:02d}:{2:02d},{3:03d}'.format(*pretvori_nazaj(cas))
9
10    with open(vhod, encoding='utf-8') as f:
11        vsebina = f.read()
12        popravljena_vsebina = re.sub(r'(\d\d):(\d\d):(\d\d),(\d\d\d)', zamenjava, vsebina)
13    with open(izhod, 'w', encoding='utf-8') as f:
14        f.write(popravljena_vsebina)

```

17.3 Povzetek

- `re.findall` vrne vse ujemajoče dele besedila;
- `re.sub` nadomesti ujemanja (lahko s funkcijo);
- `\w`, `\b`, `\d` opisujejo besedne znake, meje besed in številke;
- oklepaji določajo skupine, `\2` pa se sklicuje nanje.

Preveri se

1. Zakaj vzorec pišemo kot surov niz `r'...'`?
2. Kaj ujame vzorec `(\w+)\1`?
3. Kako `re.sub` za vsako ujemanje pokliče našo funkcijo?

Poglavje 18

Drevesa

Dvojiško drevo je podatkovna struktura, kjer ima vsako vozlišče največ dva otroka (levo in desno poddrevo). Prazno drevo predstavimo s posebnim objektom. Razred `Drevo` je v nalogah že podan:

```
1 class Drevo:
2     def __init__(self, *args, **kwargs):
3         if args:
4             self.prazno = False
5             self.vsebina = args[0]
6             self.levo = kwargs.get('levo', Drevo())
7             self.desno = kwargs.get('desno', Drevo())
8         else:
9             self.prazno = True
```

Drevo s slike sestavimo takole:

```
1 d = Drevo(5,
2         levo=Drevo(3, levo=Drevo(1)),
3         desno=Drevo(2, levo=Drevo(6), desno=Drevo(9)))
```

18.1 Preiskovanje dreves

18.1.1 1. podnaloge: vsota

Cilj. `vsota(self)` naj vrne vsoto vseh števil v drevesu.

Prazno drevo prispeva 0:

```
1 def vsota(self):
2     if self.prazno:
3         return 0
```

Sicer je vsota vsebina korena plus vsoti obeh poddreves:

```
1 else:
2     return self.vsebina + self.levo.vsota() + self.desno.vsota()
```

Zakaj deluje

Rekurzija se ustavi na praznem drevesu. Vsako vozlišče prispeva svojo vsebino, otroci pa se seštevajo rekurzivno.

Končna koda.

```

1 def vsota(self):
2     if self.prazno:
3         return 0
4     else:
5         return self.vsebina + self.levo.vsota() + self.desno.vsota()

```

18.1.2 2. podnalog: stevilo listov

Cilj. `stevilo_listov(self)` naj vrne število listov (vozlišč brez otrok).

Prazno drevo nima listov:

```

1 def stevilo_listov(self):
2     if self.prazno:
3         return 0

```

Če sta oba otroka prazna, je to list:

```

1     elif self.levo.prazno and self.desno.prazno:
2         return 1

```

Sicer seštejemo liste obeh poddreves:

```

1     else:
2         return self.levo.stevilo_listov() + self.desno.stevilo_listov()

```

Končna koda.

```

1 def stevilo_listov(self):
2     if self.prazno:
3         return 0
4     elif self.levo.prazno and self.desno.prazno:
5         return 1
6     else:
7         return self.levo.stevilo_listov() + self.desno.stevilo_listov()

```

18.1.3 3. podnalog: minimum

Cilj. `minimum(self)` naj vrne najmanjše število ali `None`, če je drevo prazno.

Zberemo kandidate: minimum levega, minimum desnega poddrevesa in vsebino korena (če drevo ni prazno):

```

1 def minimum(self):
2     kandidati = []
3     if not self.prazno:
4         kandidati = [self.levo.minimum(), self.desno.minimum(), self.vsebina]

```

Odstranimo `None` (prazna poddrevesa) in vrnemo najmanjšega; če je seznam prazen, vrnemo `None`:

```

1     stevilski_kandidati = [x for x in kandidati if x is not None]
2     return min(stevilski_kandidati, default=None)

```

Novo v tem razdelku

`min(..., default=...)`. `min` sprejme imenovani argument `default`, ki se vrne, kadar je seznam prazen.

Končna koda.

```

1 def minimum(self):
2     kandidati = []
3     if not self.prazno:
4         kandidati = [self.levo.minimum(), self.desno.minimum(), self.vsebina]
5     stevilski_kandidati = [x for x in kandidati if x is not None]
6     return min(stevilski_kandidati, default=None)

```

18.1.4 4. podnalog: premi pregled

Cilj. `premi_pregled(self)` naj vrne generator, ki vozlišča obiskuje v premem vrstnem redu: koren, levo, desno.

Če drevo ni prazno, najprej vrnemo koren:

```

1 def premi_pregled(self):
2     if not self.prazno:
3         yield self.vsebina

```

Nato vrnemo še vse iz levega in nato iz desnega poddrevesa:

```

1     for x in self.levo.premi_pregled():
2         yield x
3     for x in self.desno.premi_pregled():
4         yield x

```

Konzola

```

>>> [x for x in d.premi_pregled()]
[5, 3, 1, 2, 6, 9]

```

Končna koda.

```

1 def premi_pregled(self):
2     if not self.prazno:
3         yield self.vsebina
4         for x in self.levo.premi_pregled():
5             yield x
6         for x in self.desno.premi_pregled():
7             yield x

```

18.1.5 5. podnalog: vmesni pregled

Cilj. Enako, a v vmesnem vrstnem redu: levo, koren, desno.

```

1 def vmesni_pregled(self):
2     if not self.prazno:
3         for x in self.levo.vmesni_pregled():
4             yield x
5         yield self.vsebina
6         for x in self.desno.vmesni_pregled():
7             yield x

```

Konzola

```
>>> [x for x in d.vmesni_pregled()]
[1, 3, 5, 6, 2, 9]
```

Končna koda.

```
1 def vmesni_pregled(self):
2     if not self.prazno:
3         for x in self.levo.vmesni_pregled():
4             yield x
5         yield self.vsebina
6         for x in self.desno.vmesni_pregled():
7             yield x
```

18.1.6 6. podnalog: po nivojih

Cilj. `po_nivojih(self)` naj vozlišča obiskuje po nivojih (od korena navzdol, znotraj nivoja od leve proti desni).

Uporabimo **čakalno vrsto**. Vanjo damo koren:

```
1 def po_nivojih(self):
2     vrsta = [self]
```

Dokler je vrsta neprazna, vzamemo prvo drevo:

```
1 while vrsta:
2     drevo = vrsta.pop(0)
```

Če ni prazno, vrnemo vsebino in na konec vrste dodamo oba otroka:

```
1 if not drevo.prazno:
2     yield drevo.vsebina
3     vrsta.append(drevo.levo)
4     vrsta.append(drevo.desno)
```

Zakaj deluje

`vrsta.pop(0)` vzame najstarejši element (FIFO), zato vozlišča obdelamo po nivojih. Otroke dodajamo na konec, zato so na istem nivoju obdelani od leve proti desni.

Konzola

```
>>> [x for x in d.po_nivojih()]
[5, 3, 2, 1, 6, 9]
```

Končna koda.

```
1 def po_nivojih(self):
2     vrsta = [self]
3     while vrsta:
4         drevo = vrsta.pop(0)
5         if not drevo.prazno:
6             yield drevo.vsebina
7             vrsta.append(drevo.levo)
8             vrsta.append(drevo.desno)
```

18.2 Spreminjanje dreves

18.2.1 1. podnalog: odrezi

Cilj. `odrezi(self, n)` naj odstrani vsa vozlišča, globlja od nivoja `n`.

Prazno drevo pustimo pri miru:

```
1 def odrezi(self, n):
2     if self.prazno:
3         pass
```

Če je nivo padel pod 1, drevo izpraznimo in pobrišemo attribute:

```
1     elif n < 1:
2         self.prazno = True
3         del self.vsebina
4         del self.levo
5         del self.desno
```

Sicer rekurzivno odrežemo oba otroka z nivojem `n - 1`:

```
1     else:
2         self.levo.odrezi(n - 1)
3         self.desno.odrezi(n - 1)
```

Novo v tem razdelku

`del. del self.vsebina` odstrani atribut. Prazno drevo teh atributov ne potrebuje več.

Končna koda.

```
1 def odrezi(self, n):
2     if self.prazno:
3         pass
4     elif n < 1:
5         self.prazno = True
6         del self.vsebina
7         del self.levo
8         del self.desno
9     else:
10        self.levo.odrezi(n - 1)
11        self.desno.odrezi(n - 1)
```

18.2.2 2. podnalog: potroji

Cilj. `potroji(self)` naj pod vsakim vozliščem doda še dve njegovi kopiji.

Prazno drevo pustimo pri miru:

```
1 def potroji(self):
2     if self.prazno:
3         pass
```

Sicer najprej potrojimo oba otroka:

```
1     else:
2         self.levo.potroji()
3         self.desno.potroji()
```

Nato levo poddrevo ovijemo v novo vozlišče z isto vsebino, ki mu je staro podrejeno na levi; enako (zrcalno) za desno:

```

1 self.levo = Drevo(self.vsebina, levo=self.levo)
2 self.desno = Drevo(self.vsebina, desno=self.desno)

```

Zakaj deluje

Novo vmesno vozlišče ima isto vsebino kot koren. Levo staro poddrevo prestavimo levo od levega vmesnega vozlišča, desno staro poddrevo pa desno od desnega vmesnega vozlišča.

Končna koda.

```

1 def potroji(self):
2     if self.prazno:
3         pass
4     else:
5         self.levo.potroji()
6         self.desno.potroji()
7         self.levo = Drevo(self.vsebina, levo=self.levo)
8         self.desno = Drevo(self.vsebina, desno=self.desno)

```

18.3 Ustvarjanje dreves

18.3.1 1. podnalog: polno drevo

Cilj. `polno_drevo(n)` naj sestavi polno drevo globine `n`, kjer je vsebina vozlišča globina njegovega poddrevesa.

Če je `n` manjši od 1, je drevo prazno:

```

1 def polno_drevo(n):
2     if n < 1:
3         return Drevo()

```

Sicer je koren globine `n`, otroka pa sta polni drevesi globine `n - 1`:

```

1     else:
2         return Drevo(n, levo=polno_drevo(n - 1), desno=polno_drevo(n - 1))

```

Konzola

```

>>> polno_drevo(2)
Drevo(2,
      levo=Drevo(1),
      desno=Drevo(1))

```

Končna koda.

```

1 def polno_drevo(n):
2     if n < 1:
3         return Drevo()
4     else:
5         return Drevo(n, levo=polno_drevo(n - 1), desno=polno_drevo(n - 1))

```

18.3.2 2. podnalog: drevo zaporednih

Cilj. `drevo_zaporednih(n)` naj sestavi polno drevo globine n , ki ima pri pregledu po nivojih števila $1, 2, 3, \dots$

Kot pri polnem drevesu, a koren ima vrednost v (privzeto 1). Če je n enak 0, je drevo prazno:

```
1 def drevo_zaporednih(n, v=1):
2     if n == 0:
3         return Drevo()
```

Sicer je levi otrok $2*v$, desni pa $2*v + 1$ (tako se števila po nivojih povečajo):

```
1     else:
2         return Drevo(v, levo=drevo_zaporednih(n - 1, v=2 * v),
3                        desno=drevo_zaporednih(n - 1, v=2 * v + 1))
```

Zakaj deluje

Oštevilčenje $1, 2, 3, \dots$ po nivojih ustreza temu, da ima vozlišče s številko v otroka $2*v$ in $2*v + 1$.

Končna koda.

```
1 def drevo_zaporednih(n, v=1):
2     if n == 0:
3         return Drevo()
4     else:
5         return Drevo(v, levo=drevo_zaporednih(n - 1, v=2 * v),
6                        desno=drevo_zaporednih(n - 1, v=2 * v + 1))
```

18.4 Povzetek

- drevo obdelujemo rekurzivno: prazno drevo je osnovni primer;
- **premi pregled:** koren, levo, desno;
- **vmesni pregled:** levo, koren, desno;
- **po nivojih:** čakalna vrsta (FIFO);
- generatore pišemo z `yield`, tudi rekurzivno.

Preveri se

1. Zakaj **vsota** na praznem drevesu vrne 0?
2. V čem se razlikujeta **premi** in **vmesni pregled**?
3. Zakaj pri **po_nivojih** uporabimo `pop(0)` in ne `pop()`?

Poglavje 19

Iskalna in AVL drevesa

Iskalno drevo je dvojiško drevo, v katerem so v levem poddrevesu vsi podatki manjši, v desnem pa vsi večji od podatka v korenu. Zato lahko v njem iščemo hitro. **AVL drevo** je iskalno drevo, ki se ob vstavljanju samodejno uravnoteži.

19.1 Iskalna drevesa

Razred `IskalnoDrevo` je podoben razredu `Drevo`; konstruktor sprejme iterator in elemente po vrsti vstavi:

```
1 class IskalnoDrevo:
2     def __init__(self, vsebina=[]):
3         self.prazno = True
4         for n in vsebina:
5             self.dodaj(n)
```

19.1.1 1. podnalog: dodaj

Cilj. `dodaj(self, podatek)` naj vstavi nov podatek.

Če je drevo prazno, vanj zapišemo podatek in naredimo prazna otroka:

```
1 def dodaj(self, podatek):
2     if self.prazno:
3         self.prazno = False
4         self.vsebina = podatek
5         self.levo = IskalnoDrevo()
6         self.desno = IskalnoDrevo()
```

Če je podatek manjši od korena, ga vstavimo v levo poddrevo:

```
1 elif self.vsebina > podatek:
2     self.levo.dodaj(podatek)
```

Če je večji, v desno poddrevo:

```
1 elif self.vsebina < podatek:
2     self.desno.dodaj(podatek)
```

Zakaj deluje

Rekurzija sledi pravilu iskalnega drevesa: manjše levo, večje desno. Ko pridemo do praznega drevesa, tam ustvarimo novo vozlišče.

Končna koda.

```

1  def dodaj(self, podatek):
2      if self.prazno:
3          self.prazno = False
4          self.vsebina = podatek
5          self.levo = IskalnoDrevo()
6          self.desno = IskalnoDrevo()
7      elif self.vsebina > podatek:
8          self.levo.dodaj(podatek)
9      elif self.vsebina < podatek:
10         self.desno.dodaj(podatek)

```

19.1.2 2. podnalog: poisci

Cilj. `poisci(self, podatek)` naj vrne vozlišče s tem podatkom ali `None`.

Prazno drevo pomeni, da podatka ni:

```

1  def poisci(self, podatek):
2      if self.prazno:
3          return None

```

Če je podatek v korenu, vrnemo kar to vozlišče:

```

1      elif self.vsebina == podatek:
2          return self

```

Sicer iščemo naprej levo ali desno, kot velela urejenost:

```

1      elif self.vsebina > podatek:
2          return self.levo.poisci(podatek)
3      else:
4          return self.desno.poisci(podatek)

```

Končna koda.

```

1  def poisci(self, podatek):
2      if self.prazno:
3          return None
4      elif self.vsebina == podatek:
5          return self
6      elif self.vsebina > podatek:
7          return self.levo.poisci(podatek)
8      else:
9          return self.desno.poisci(podatek)

```

19.1.3 3. podnalog: po vrsti

Cilj. `po_vrsti(self)` naj generira vse podatke od najmanjšega do največjega.

To je natanko *vmesni* pregled: levo poddrevo, koren, desno poddrevo:

```

1  def po_vrsti(self):
2      if not self.prazno:
3          for n in self.levo.po_vrsti():
4              yield n
5          yield self.vsebina
6          for n in self.desno.po_vrsti():
7              yield n

```

Zakaj deluje

Ker je drevo iskalno, so vsi podatki v levem poddrevesu manjši od korena, vsi v desnem pa večji. Zato vmesni pregled našteje podatke urejeno.

Končna koda.

```

1  def po_vrsti(self):
2      if not self.prazno:
3          for n in self.levo.po_vrsti():
4              yield n
5          yield self.vsebina
6          for n in self.desno.po_vrsti():
7              yield n

```

19.2 AVL drevo

AVL drevo hrani v vsakem vozlišču še globina. Pomožna metoda `make` zgradi vozlišče “na roke” (brez uravnoteženja):

```

1  class AVLDrevo:
2      @classmethod
3      def make(cls, *args, **kwargs):
4          d = cls()
5          if args:
6              d.prazno = False
7              d.vsebina = args[0]
8              d.levo = kwargs.get('levo', cls())
9              d.desno = kwargs.get('desno', cls())
10             d.globina = max(d.levo.globina, d.desno.globina) + 1
11         return d
12
13     def __init__(self, vsebina=[]):
14         self.prazno = True
15         self.globina = 0
16         for x in vsebina:
17             self.dodaj(x)
18
19     def dodaj(self, x):
20         if self.prazno:
21             self.prazno = False
22             self.vsebina = x
23             self.levo = AVLDrevo()
24             self.desno = AVLDrevo()
25             self.globina = 1
26             return
27         if x < self.vsebina:
28             self.levo.dodaj(x)
29         else:
30             self.desno.dodaj(x)
31         self.globina = max(self.levo.globina, self.desno.globina) + 1
32         self.popravi()

```

19.2.1 1. podnalog: poisci

Cilj. Enako kot pri iskalnem drevesu:

```

1 def poisci(self, podatek):
2     if self.prazno:
3         return None
4     elif self.vsebina == podatek:
5         return self
6     elif podatek < self.vsebina:
7         return self.levo.poisci(podatek)
8     else:
9         return self.desno.poisci(podatek)

```

Končna koda.

```

1 def poisci(self, podatek):
2     if self.prazno:
3         return None
4     elif self.vsebina == podatek:
5         return self
6     elif podatek < self.vsebina:
7         return self.levo.poisci(podatek)
8     else:
9         return self.desno.poisci(podatek)

```

19.2.2 2. podnalog: leva rotacija

Cilj. `leva_rotacija(self)` naj drevo zavrti v levo.

Zapomnimo si levo poddrevo, desno poddrevo in njegova dva otroka:

```

1 def leva_rotacija(self):
2     l = self.levo
3     v = self.desno
4     d_1 = self.desno.levo
5     d_2 = self.desno.desno

```

Zamenjamo vsebini korena in desnega sina (vrh "postane" desni sin):

```

1 self.vsebina, v.vsebina = v.vsebina, self.vsebina

```

Prevežemo otroke po pravilu leve rotacije:

```

1 v.levo = l
2 v.desno = d_1
3 self.levo = v
4 self.desno = d_2

```

Na koncu popravimo globini obeh vozlišč:

```

1 v.globina = max(v.levo.globina, v.desno.globina) + 1
2 self.globina = max(self.levo.globina, self.desno.globina) + 1

```

Zakaj deluje

Leva rotacija dvigne desnega sina na mesto korena; njegovo levo poddrevo postane desno poddrevo starega korena. Globini obeh prizadetih vozlišč ponovno izračunamo od spodaj navzgor.

Končna koda.

```

1  def leva_rotacija(self):
2      l = self.levo
3      v = self.desno
4      d_1 = self.desno.levo
5      d_2 = self.desno.desno
6      self.vsebina, v.vsebina = v.vsebina, self.vsebina
7      v.levo = l
8      v.desno = d_1
9      self.levo = v
10     self.desno = d_2
11     v.globina = max(v.levo.globina, v.desno.globina) + 1
12     self.globina = max(self.levo.globina, self.desno.globina) + 1

```

19.2.3 3. podnalog: desna rotacija

Cilj. Zrcalna različica leve rotacije.

```

1  def desna_rotacija(self):
2      v = self.levo
3      l_1 = self.levo.levo
4      l_2 = self.levo.desno
5      d = self.desno

```

Zamenjamo vsebini in prevežemo otroke:

```

1      self.vsebina, v.vsebina = v.vsebina, self.vsebina
2      v.levo = l_2
3      v.desno = d
4      self.levo = l_1
5      self.desno = v

```

Popravimo globini:

```

1      v.globina = max(v.levo.globina, v.desno.globina) + 1
2      self.globina = max(self.levo.globina, self.desno.globina) + 1

```

Končna koda.

```

1  def desna_rotacija(self):
2      v = self.levo
3      l_1 = self.levo.levo
4      l_2 = self.levo.desno
5      d = self.desno
6      self.vsebina, v.vsebina = v.vsebina, self.vsebina
7      v.levo = l_2
8      v.desno = d
9      self.levo = l_1
10     self.desno = v
11     v.globina = max(v.levo.globina, v.desno.globina) + 1
12     self.globina = max(self.levo.globina, self.desno.globina) + 1

```

19.2.4 4. podnalog: popravi

Cilj. `popravi(self)` naj preveri ravnovesje in po potrebi izvede rotacijo (včasih s predrotacijo).

Če je desno poddrevo previsoko, naredimo levo rotacijo. Če je desni sin sam nagnjen v levo, najprej predrotacijo (desno) na njem:

```

1  def popravi(self):
2      if self.levo.globina + 1 < self.desno.globina:
3          if self.desno.levo.globina > self.desno.desno.globina:
4              self.desno.desna_rotacija()
5              self.leva_rotacija()

```

Če je levo poddrevo previsoko, je postopek zrcalen:

```

1      elif self.levo.globina > self.desno.globina + 1:
2          if self.levo.desno.globina > self.levo.levo.globina:
3              self.levo.leva_rotacija()
4              self.desna_rotacija()
5  \end{lsthighlight}
6
7  \begin{zakaj}
8  Poznamo štiri primere: levo-levo in desno-desno rešimo z eno rotacijo,
9  levo-desno in desno-levo pa z dvema (predrotacija + rotacija). Metoda
10 \py{popravi} te primere prepozna po globinah poddreves.
11 \end{zakaj}
12
13 \textbf{Končna koda.}
14 \lsthighlight{}
15 \begin{lstlisting}
16     def popravi(self):
17         if self.levo.globina + 1 < self.desno.globina:
18             if self.desno.levo.globina > self.desno.desno.globina:
19                 self.desno.desna_rotacija()
20                 self.leva_rotacija()
21         elif self.levo.globina > self.desno.globina + 1:
22             if self.levo.desno.globina > self.levo.levo.globina:
23                 self.levo.leva_rotacija()
24                 self.desna_rotacija()

```

19.3 Povzetek

- iskalno drevo: levo manjše, desno večje;
- `po_vrsti` je vmesni pregled in našteje podatke urejeno;
- AVL drevo hrani globine in se uravnava z rotacijami;
- štiri variante: levo-levo, desno-desno (ena rotacija) ter levo-desno, desno-levo (predrotacija + rotacija).

Preveri se

1. Zakaj `po_vrsti` vrne urejeno zaporedje?
2. Kaj naredi leva rotacija z levim poddrevesom desnega sina?
3. Kdaj je potrebna predrotacija?

Poglavje 20

Kombinatorično preiskovanje

V tem poglavju bomo s sestopanjem in rekurzijo preiskovali kombinatorične možnosti: razbitja, urejevalno razdaljo, particije, poti in involucije.

20.1 Stirlingova števila druge vrste

Cilj. `stirling_2(n, k)` naj vrne število razbitij množice z n elementi na k nepraznih podmnožic. Velja $S(n, k) = k S(n - 1, k) + S(n - 1, k - 1)$.

Osnovna pogoja: $S(0, 0) = 1$, sicer pa je rezultat 0, kadar je eden od argumentov nič:

```
1 def stirling_2(n, k):  
2     if n == 0 and k == 0:  
3         return 1  
4     if n == 0 or k == 0:  
5         return 0
```

Rekurzivna zveza:

```
1     return k * stirling_2(n - 1, k) + stirling_2(n - 1, k - 1)
```

Zakaj deluje

n -ti element lahko damo v eno od k obstoječih podmnožic (prvi del) ali pa zanj odpremo novo podmnožico (drugi del).

Konzola

```
>>> stirling_2(5, 2)  
15
```

Končna koda.

```
1 def stirling_2(n, k):  
2     if n == 0 and k == 0:  
3         return 1  
4     if n == 0 or k == 0:  
5         return 0  
6     return k * stirling_2(n - 1, k) + stirling_2(n - 1, k - 1)
```

20.2 Levenshteinova razdalja

Cilj. `edit_distance(a, b)` naj vrne najmanjše število operacij (brisanje, vstavljanje, zamenjava), da `a` spremenimo v `b`.

Če je prvi niz prazen, moramo vstaviti vse znake drugega (in obratno):

```
1 def edit_distance(a, b):
2     if a == '':
3         return len(b)
4     if b == '':
5         return len(a)
```

Če sta prva znaka enaka, ju preprosto preskočimo:

```
1     if a[0] == b[0]:
2         return edit_distance(a[1:], b[1:])
```

Sicer poskusimo vse tri operacije in vzamemo najboljšo (najmanjšo):

```
1     return 1 + min(edit_distance(a[1:], b),
2                     edit_distance(a, b[1:]),
3                     edit_distance(a[1:], b[1:]))
```

Zakaj deluje

`a[1:]`, `b` pomeni brisanje, `a`, `b[1:]` vstavljanje, `a[1:]`, `b[1:]` pa zamenjava. Vsaka operacija šteje 1, zato prištejemo 1 in vzamemo minimum.

Končna koda.

```
1 def edit_distance(a, b):
2     if a == '':
3         return len(b)
4     if b == '':
5         return len(a)
6     if a[0] == b[0]:
7         return edit_distance(a[1:], b[1:])
8     return 1 + min(edit_distance(a[1:], b),
9                     edit_distance(a, b[1:]),
10                    edit_distance(a[1:], b[1:]))
```

20.3 Particije in permutacije

20.3.1 1. in 2. podnalogi: permutacije

Cilj. `permutacije(n)` naj izpiše vse permutacije reda `n` v leksikografskem vrstnem redu.

Gradimo permutacijo `p` kot terko. Ko je dolga `red`, jo izpišemo:

```
1 def permutacije(red, p=()):
2     if len(p) == red:
3         print(' '.join(map(str, p)))
4         return
```

Sicer poskusimo vsako število, ki ga še ni v `p`:

```
1     for i in range(1, red + 1):
2         if i not in p:
3             permutacije(red, p + (i,))
```

Za obratni vrstni red preprosto tečemo od `red` navzdol:

```

1  for i in range(red, 0, -1):
2      if i not in p:
3          permutacije_od_zadaj(red, p + (i,))

```

Zakaj deluje

Ker `i` teče naraščajoče, so permutacije izpisane leksikografsko; če teče padajoče, dobimo obratni vrstni red.

Končna koda.

```

1  def permutacije(red, p=()):
2      if len(p) == red:
3          print(' '.join(map(str, p)))
4          return
5      for i in range(1, red + 1):
6          if i not in p:
7              permutacije(red, p + (i,))
8
9  def permutacije_od_zadaj(red, p=()):
10     if len(p) == red:
11         print(' '.join(map(str, p)))
12         return
13     for i in range(red, 0, -1):
14         if i not in p:
15             permutacije_od_zadaj(red, p + (i,))

```

20.3.2 3. in 4. podnalogi: particije

Cilj. `particije(n)` naj izpiše vse particije števila `n` in vrne njihovo število.

Pomožna funkcija `part(p, m, s)` dopolni particijo `p`; `m` je največje število, ki ga smemo še dodati, `s` pa preostala vsota:

```

1  def part(p, m, s):
2      count = 0
3      if len(p) > 0:
4          m = min(m, p[-1])

```

Če je `m` enak 1, dodamo še `s` enic in izpišemo:

```

1  if m == 1:
2      p += s * (1,)
3      print(' + '.join(map(str, p)))
4      return 1

```

Sicer za vsako število `i` kopij števila `m` nadaljujemo z `m - 1`:

```

1  for i in range(s // m, -1, -1):
2      count += part(p + i * (m,), m - 1, s - m * i)
3  return count
4
5  def particije(n):
6      return part(), n, n

```

Za obratni vrstni red spremenimo le smer zanke:

```

1  for i in range(s // m + 1):

```

Končna koda.

```

1 def part(p, m, s):
2     count = 0
3     if len(p) > 0:
4         m = min(m, p[-1])
5         if m == 1:
6             p += s * (1,)
7             print(' + '.join(map(str, p)))
8             return 1
9         for i in range(s // m, -1, -1):
10             count += part(p + i * (m,), m - 1, s - m * i)
11     return count
12
13 def particije(n):
14     return part([], n, n)

```

20.4 Naraščajoče poti

20.4.1 1. in 2. podnalogi: narasc in narascajoče poti

Cilj. narasc(pot, cilj) naj dokonča pot do cilja in vrne število vseh poti.

Zadnjo točko poti in cilj razpakiramo:

```

1 def narasc(pot, cilj):
2     x, y = pot[-1]
3     x_c, y_c = cilj

```

Če smo na cilju, pot izpišemo in preštejemo:

```

1     if (x == x_c) and (y == y_c):
2         print(pot)
3         return 1

```

Če smo cilj prešli, ta pot ni veljavna:

```

1     if (x > x_c) or (y > y_c):
2         return 0

```

Sicer poskusimo korak navzgor in korak desno (v tem vrstnem redu, da je izpis leksikografski) ter seštejemo:

```

1     count = narasc(pot + [(x, y + 1)], cilj)
2     count += narasc(pot + [(x + 1, y)], cilj)
3     return count
4
5 def narascajoce_poti(cilj):
6     return narasc([(0, 0)], cilj)

```

Končna koda.

```

1 def narasc(pot, cilj):
2     x, y = pot[-1]
3     x_c, y_c = cilj
4     if (x == x_c) and (y == y_c):
5         print(pot)
6         return 1
7     if (x > x_c) or (y > y_c):
8         return 0

```

```

9     count = narasc(pot + [(x, y + 1)], cilj)
10    count += narasc(pot + [(x + 1, y)], cilj)
11    return count
12
13 def narascajoce_poti(cilj):
14     return narasc([(0, 0)], cilj)

```

20.4.2 3. podnalog: seznam poti

Cilj. Enako, a namesto izpisa vrnemo seznam poti.

Osnovna primera vrnete seznam (z eno potjo oz. prazen), sicer pa seznama združimo:

```

1 def narasc_sez(pot, cilj):
2     x, y = pot[-1]
3     x_c, y_c = cilj
4     if (x == x_c) and (y == y_c):
5         return [pot]
6     if (x > x_c) or (y > y_c):
7         return []
8     return (narasc_sez(pot + [(x, y + 1)], cilj)
9             + narasc_sez(pot + [(x + 1, y)], cilj))

```

Končna koda.

```

1 def narasc_sez(pot, cilj):
2     x, y = pot[-1]
3     x_c, y_c = cilj
4     if (x == x_c) and (y == y_c):
5         return [pot]
6     if (x > x_c) or (y > y_c):
7         return []
8     return (narasc_sez(pot + [(x, y + 1)], cilj)
9             + narasc_sez(pot + [(x + 1, y)], cilj))
10
11 def seznam_poti(cilj):
12     return narasc_sez([(0, 0)], cilj)

```

20.5 Involucije brez fiksnih točk

Cilj. `invol(perm, n)` naj dokonča delno involucijo `perm` (množico parov) in vrne število vseh. Involucija brez fiksnih točk sestoji samo iz parov.

Če imamo $n/2$ parov, je involucija končana:

```

1 def invol(perm, n):
2     if len(perm) * 2 == n:
3         print(perm)
4         return 1

```

Poiščemo najmanjše število m , ki še ni uporabljeno:

```

1     count = 0
2     used = set()
3     for u, v in perm:
4         used.add(u)
5         used.add(v)
6     m = min([i for i in range(1, n + 1) if i not in used])

```

Nato m parimo z vsakim večjim neuporabljenim številom i :

```

1  for i in range(m + 1, n + 1):
2      if i in used:
3          continue
4      count += invol(perm + [(m, i)], n)
5  return count

```

Zakaj deluje

Najmanjše neuporabljeno število m mora biti v nekem paru. Ker je involucija brez fiksni točk, ga parimo z nekim večjim neuporabljenim številom. S tem vsak par zabeležimo natanko enkrat (v leksikografskem vrstnem redu).

Za vrnitev seznama namesto števila zamenjamo `count` z združevanjem seznamov:

```

1  if len(perm) * 2 == n:
2      return [perm]
3  ret = []
4  ...
5  ret += seznam_invol(perm + [(m, i)], n)
6  return ret

```

Končna koda.

```

1  def invol(perm, n):
2      if len(perm) * 2 == n:
3          print(perm)
4          return 1
5      count = 0
6      used = set()
7      for u, v in perm:
8          used.add(u)
9          used.add(v)
10     m = min([i for i in range(1, n + 1) if i not in used])
11     for i in range(m + 1, n + 1):
12         if i in used:
13             continue
14         count += invol(perm + [(m, i)], n)
15     return count
16
17 def involucije(n):
18     return invol([], n)
19
20 def seznam_invol(perm, n):
21     if len(perm) * 2 == n:
22         return [perm]
23     ret = []
24     used = set()
25     for u, v in perm:
26         used.add(u)
27         used.add(v)
28     m = min([i for i in range(1, n + 1) if i not in used])
29     for i in range(m + 1, n + 1):
30         if i in used:
31             continue
32         ret += seznam_invol(perm + [(m, i)], n)
33     return ret

```

```
34  
35 def seznam_involucij(n):  
36     return seznam_invol([], n)
```

20.6 Povzetek

- **Stirlingova števila**: rekurzivna zveza iz dveh izbir;
- **urejevalna razdalja**: tri operacije, vzamemo minimum;
- **permutacije in particije**: sestopanje z dodajanjem;
- **naraščajoče poti**: dve možni smeri koraka;
- **involucije**: parimo najmanjše neuporabljeno število.

Preveri se

1. Zakaj pri `stirling_2` faktor `k` stoji pred prvim členom?
2. Zakaj pri `edit_distance` preverimo enakost prvih znakov?
3. Kako pri `permutacije` dosežemo leksikografski vrstni red?

Poglavje 21

Najbližji par točk

Cilj. Med danimi točkami v ravnini poišči dve, ki sta si najbližje. Uporabili bomo *deli in vladaj*, da dobimo algoritem s časovno zahtevnostjo $O(n \log n)$.

21.1 Pomožne funkcije

21.1.1 1. podnaloga: dist2

Cilj. `dist2(u, v)` naj vrne kvadrat razdalje med točkama (kvadrat je strogo naraščajoča funkcija razdalje, zato se mu lahko izognemo korenu).

Izračunamo razliki koordinat in vsoto njunih kvadratov:

```
1 def dist2(u, v):  
2     dx = u[0] - v[0]  
3     dy = u[1] - v[1]  
4     return dx * dx + dy * dy
```

Konzola

```
>>> dist2((4, 1), (1, 3))  
13
```

Končna koda.

```
1 def dist2(u, v):  
2     dx = u[0] - v[0]  
3     dy = u[1] - v[1]  
4     return dx * dx + dy * dy
```

21.1.2 2. in 3. podnaloga: uredi po x in po y

Cilj. `uredi_po_x(t)` naj vrne sezname *indeksov* točk, urejenih po koordinati x (in enako za y).

`enumerate(t)` da pare (i, u) . Uredimo jih po koordinati; parameter `key` pove, po kateri vrednosti. Nato obdržimo le indekse:

```
1 def uredi_po_x(t):  
2     return [i for (i, u) in sorted(enumerate(t), key=lambda p: p[1][0])]
```

Za koordinato y je povsem enako, le na koncu izberemo drugi element para:

```

1 def uredi_po_y(t):
2     return [i for (i, u) in sorted(enumerate(t), key=lambda p: p[1][1])]

```

Novo v tem razdelku

`sorted(..., key=...)`. `key` je funkcija, ki za vsak element vrne vrednost, po kateri urejamo. Tu je `p[1][0]` koordinata `x`, torej točka `u = p[1]`, njena prva komponenta.

Končna koda.

```

1 def uredi_po_x(t):
2     return [i for (i, u) in sorted(enumerate(t), key=lambda p: p[1][0])]
3
4 def uredi_po_y(t):
5     return [i for (i, u) in sorted(enumerate(t), key=lambda p: p[1][1])]

```

21.2 Najbližji par

Cilj. `najblizji(t)` naj vrne indeksa najbližjih točk ali `None`.

Najprej pripravimo indekse, urejene po `x` in po `y`:

```

1 def najblizji(t):
2     ix = uredi_po_x(t)
3     iy = uredi_po_y(t)

```

Pomožna funkcija `isci(i, j)` išče najbližji par med točkami z indeksi `ix[i:j+1]`. Robna primera: nič ali ena točka $\rightarrow$ `None`, dve točki $\rightarrow$ kar ta par:

```

1 def isci(i, j):
2     if j <= i:
3         return None
4     if i + 1 == j:
5         return ix[i], ix[j]

```

Razdelimo na pol in rekurzivno poiščemo najboljša para v obeh polovicah:

```

1     mid = (i + j) // 2
2     levo = isci(i, mid)
3     desno = isci(mid + 1, j)

```

Izberemo boljšega od obeh kandidatov:

```

1     min_i, min_j = levo
2     d2 = dist2(t[min_i], t[min_j])
3     if desno is not None:
4         i_desno, j_desno = desno
5         d2_desno = dist2(t[i_desno], t[j_desno])
6         if d2_desno < d2:
7             min_i, min_j = desno
8             d2 = d2_desno

```

Izračunamo navpičnico, ki deli območje, in v pas širine $2d$ zberemo točke, ki bi lahko bile bližje od trenutnega najboljšega para:

```

1     x_0 = (t[ix[mid]][0] + t[ix[mid + 1]][0]) / 2
2     pas = [ii for ii in iy if (t[ii][0] - x_0) ** 2 <= d2]

```

Preiščemo pas. Zadošča primerjati vsako točko z naslednjimi največ petimi (to je geometrijska lastnost, da je v tako majhnem območju lahko le omejeno mnogo točk):

```

1  for ii in range(len(pas)):
2      for jj in range(ii + 1, min(ii + 6, len(pas))):
3          d2_novi = dist2(t[pas[ii]], t[pas[jj]])
4          if d2_novi < d2:
5              d2 = d2_novi
6              min_i = pas[ii]
7              min_j = pas[jj]
8  return min_i, min_j

```

Zunanja funkcija sproži iskanje:

```

1  return isci(0, len(t) - 1)

```

Zakaj deluje

Faza “deli” razdeli točke na pol in reši vsako polovico rekurzivno. Faza “vladaj” preveri še pas okoli delilne črte: le tam je lahko par, ki je bližje od najboljšega znotraj posamezne polovice. Ker je v pasu malo točk, je celotna zahtevnost $O(n \log n)$.

Končna koda.

```

1  def najblizji(t):
2      ix = uredi_po_x(t)
3      iy = uredi_po_y(t)
4
5      def isci(i, j):
6          if j <= i:
7              return None
8          if i + 1 == j:
9              return ix[i], ix[j]
10
11         mid = (i + j) // 2
12         levo = isci(i, mid)
13         desno = isci(mid + 1, j)
14
15         min_i, min_j = levo
16         d2 = dist2(t[min_i], t[min_j])
17         if desno is not None:
18             i_desno, j_desno = desno
19             d2_desno = dist2(t[i_desno], t[j_desno])
20             if d2_desno < d2:
21                 min_i, min_j = desno
22                 d2 = d2_desno
23
24         x_0 = (t[ix[mid]][0] + t[ix[mid + 1]][0]) / 2
25         pas = [ii for ii in iy if (t[ii][0] - x_0) ** 2 <= d2]
26
27         for ii in range(len(pas)):
28             for jj in range(ii + 1, min(ii + 6, len(pas))):
29                 d2_novi = dist2(t[pas[ii]], t[pas[jj]])
30                 if d2_novi < d2:
31                     d2 = d2_novi
32                     min_i = pas[ii]
33                     min_j = pas[jj]
34

```

```
35     return min_i, min_j
36
37     return isci(0, len(t) - 1)
```

21.3 Povzetek

- razdalje primerjamo po kvadratu, da se izognemo korenu;
- indekse točk uredimo po x in po y ;
- deli in vladaj: rešimo polovici in nato še pas ob delilni črti;
- v pasu zadošča preveriti le omejeno mnogo sosedov.

Preveri se

1. Zakaj zadošča primerjati kvadrate razdalj?
2. Kaj je delilna črta x_0 in zakaj gledamo pas širine $2d$?
3. Zakaj v notranji zanki pregledamo le naslednjih pet točk?

Poglavje 22

Floyd-Warshall

Floyd-Warshallov algoritem poišče dolžine najkrajših poti med *vsem* pari vozlišč v uteženem grafu. Graf podamo z *matriko razdalj* d : $d[i][j]$ je cena povezave od i do j ali $\text{float}('inf')$, če povezave ni.

22.1 Floyd-Warshall

22.1.1 1. podnalog: floyd_warshall

Cilj. `floyd_warshall(d)` naj vrne matriko f , kjer je $f[i][j]$ cena najkrajše poti od i do j .

Najprej naredimo kopijo matrike (da ne pokvarimo vhodne):

```
1 def floyd_warshall(d):
2     n = len(d)
3     f = [[x for x in vrstica] for vrstica in d]
```

Za vsako vmesno vozlišče k :

```
1     for k in range(n):
```

Za vsak začetek i :

```
1         for i in range(n):
```

In vsak cilj j poskusimo izboljšati pot prek k :

```
1             for j in range(n):
2                 f[i][j] = min(f[i][j], f[i][k] + f[k][j])
3     return f
```

Zakaj deluje

Matrika f postopoma vsebuje najkrajše poti, ki smejo uporabljati prva k vozlišča kot vmesna. Če je pot prek k krajša, jo zamenjamo. Po vseh k imamo najkrajše poti med vsemi pari.

Končna koda.

```
1 def floyd_warshall(d):
2     n = len(d)
3     f = [[x for x in vrstica] for vrstica in d]
4     for k in range(n):
5         for i in range(n):
6             for j in range(n):
```

```

7         f[i][j] = min(f[i][j], f[i][k] + f[k][j])
8     return f

```

22.1.2 2. podnalog: najkrajša pot

Cilj. `najkrajša_pot(d, f, i, j)` naj vrne seznam vozlišč na najkrajši poti od `i` do `j` ali `None`, če poti ni.

Če je cena neskončna, vozlišči nista povezani:

```

1 def najkrajša_pot(d, f, i, j):
2     if f[i][j] == inf:
3         return None

```

Pot začnemo z začetnim vozliščem:

```

1     n = len(d)
2     pot = [i]

```

Dokler nismo pri cilju, poiščemo naslednje vozlišče `k`, za katero velja, da je povezava `i -> k` plus najkrajša pot `k -> j` natanko najkrajša pot `i -> j`:

```

1     while i != j:
2         for k in range(n):
3             if k == i or d[i][k] == inf or k in pot:
4                 continue
5             if d[i][k] + f[k][j] == f[i][j]:
6                 i = k
7                 break
8         pot.append(i)
9     return pot

```

Zakaj deluje

Zaradi Floyd-Warshalla velja $f[i][j] = d[i][k] + f[k][j]$ natanko za naslednje vozlišče `k` na optimalni poti. Tako pot rekonstruiramo korak za korakom.

Končna koda.

```

1 def najkrajša_pot(d, f, i, j):
2     if f[i][j] == inf:
3         return None
4     n = len(d)
5     pot = [i]
6     while i != j:
7         for k in range(n):
8             if k == i or d[i][k] == inf or k in pot:
9                 continue
10            if d[i][k] + f[k][j] == f[i][j]:
11                i = k
12                break
13        pot.append(i)
14    return pot

```

22.2 Povzetek

- Floyd-Warshall poišče najkrajše poti med vsemi pari v času $O(n^3)$;

- ideja: postopoma dovolimo vedno več vmesnih vozlišč;
- pot rekonstruiramo tako, da iščemo naslednje vozlišče k , kjer je $d[i][k] + f[k][j] == f[i][j]$.

Preveri se

1. Kaj pomeni `float('inf')` v matriki razdalj?
2. Zakaj v zunanji zanki uporabimo k in ne i ?
3. Kako iz matrike f razberemo posamezno pot?

Poglavje 23

Permutacije in geometrija

V tem poglavju bomo utrdili objektno programiranje: najprej z razredom za permutacije, nato z razredi za vektorje, točke in premice.

23.1 Permutacija

Permutacijo zapišemo kot produkt disjunktnih ciklov, npr. `[[1, 5, 2], [3, 7]]`.

23.1.1 1. podnalog: konstruktor

Cilj. Konstruktor `__init__(self, cikli, stopnja=None)` naj nastavi `stopnja` in `cikli`. Vsak cikel naj se začne z najmanjšim elementom, cikli naj bodo urejeni, cikle dolžine 1 pa odstranimo.

Če `stopnja` ni podana, je to največje število v ciklih:

```
1 class Permutacija:
2     def __init__(self, cikli, stopnja=None):
3         if stopnja is None:
4             self.stopnja = max(map(max, cikli))
5         else:
6             self.stopnja = stopnja
```

Obdržimo le cikle dolžine več kot 1 (in naredimo kopije seznamov):

```
1 self.cikli = [list(cikel) for cikel in cikli if len(cikel) > 1]
```

Vsak cikel zavrtimo tako, da je najmanjši element na začetku:

```
1 for k in range(len(self.cikli)):
2     cikel = self.cikli[k]
3     m = 0
4     for i in range(len(cikel)):
5         if cikel[i] < cikel[m]:
6             m = i
7     self.cikli[k] = cikel[m:] + cikel[:m]
```

Cikle uredimo:

```
1 self.cikli.sort()
```

Zakaj deluje

Cikel `[1, 5, 2]` je isti kot `[5, 2, 1]` — z vrtenjem ga normaliziramo. S tem dobimo enolično predstavitev, kar olajša primerjanje.

Končna koda.

```

1 class Permutacija:
2     def __init__(self, cikli, stopnja=None):
3         if stopnja is None:
4             self.stopnja = max(map(max, cikli))
5         else:
6             self.stopnja = stopnja
7     self.cikli = [list(cikel) for cikel in cikli if len(cikel) > 1]
8     for k in range(len(self.cikli)):
9         cikel = self.cikli[k]
10        m = 0
11        for i in range(len(cikel)):
12            if cikel[i] < cikel[m]:
13                m = i
14        self.cikli[k] = cikel[m:] + cikel[:m]
15    self.cikli.sort()

```

23.1.2 2. podnalog: inverz

Cilj. `inverz(self)` naj vrne inverz permutacije: vsak cikel obrnemo:

```

1 def inverz(self):
2     return Permutacija([c[::-1] for c in self.cikli], stopnja=self.stopnja)

```

Zakaj deluje

Inverz cikla (1 5 2) je (1 2 5) — isti cikel, prebran v obratni smeri. Zato vsak cikel obrnemo z rezino `c[::-1]`, konstruktor pa ga spet normalizira.

Končna koda.

```

1 def inverz(self):
2     return Permutacija([c[::-1] for c in self.cikli], stopnja=self.stopnja)

```

23.1.3 3. podnalog: ciklični tip

Cilj. `ciklicni_tip(self)` naj vrne nabor, kjer *i*-ti element pove, koliko je ciklov dolžine *i* + 1.

Zberemo dolžine ciklov in najdaljšo:

```

1 def ciklicni_tip(self):
2     dolzine = [len(c) for c in self.cikli]
3     najdaljsi_cikel = max(dolzine)

```

Preštejemo cikle vsake dolžine:

```

1 ctip = [dolzine.count(i) for i in range(1, najdaljsi_cikel + 1)]

```

Število ciklov dolžine 1 so vse fiksne točke, ki jih nismo izpisali:

```

1 ctip[0] = self.stopnja - sum(dolzine)
2 return tuple(ctip)

```

Končna koda.

```

1 def ciklicni_tip(self):
2     dolzine = [len(c) for c in self.cikli]
3     najdaljsi_cikel = max(dolzine)
4     ctip = [dolzine.count(i) for i in range(1, najdaljsi_cikel + 1)]
5     ctip[0] = self.stopnja - sum(dolzine)
6     return tuple(ctip)

```

23.1.4 4. podnalog: red

Cilj. `red(self)` naj vrne red permutacije (najmanjši skupni večkratnik dolžin ciklov).

Pomožna funkcija za najmanjši skupni večkratnik:

```

1 def lcm(a, b):
2     return a * b // gcd(a, b)

```

Red dobimo tako, da po vrsti računamo lcm z dolžinami ciklov:

```

1 def red(self):
2     r = 1
3     for c in self.cikli:
4         r = lcm(r, len(c))
5     return r

```

Zakaj deluje

Permutacija se ponovi šele, ko se vsak cikel obrne celo število krat. To je najmanjši skupni večkratnik dolžin vseh ciklov.

Končna koda.

```

1 def gcd(a, b):
2     if a < b:
3         a, b = b, a
4     while b != 0:
5         a, b = b, a % b
6     return a
7
8 def lcm(a, b):
9     return a * b // gcd(a, b)
10
11 def red(self):
12     r = 1
13     for c in self.cikli:
14         r = lcm(r, len(c))
15     return r

```

23.2 Geometrija

Podani so razredi `Vektor`, `Točka` in `Premica`. Vektor hrani koordinati x in y , točka svoji, premica pa je podana s točko in *normalo* (vektorjem, pravokotnim na premico).

23.2.1 1. in 2. podnalog: `__repr__` in `__str__`

Cilj. `__repr__` naj vrne veljavno kodo, `__str__` pa čitljiv zapis.

```

1 def __repr__(self):
2     return 'Vektor({0}, {1})'.format(self.x, self.y)
3
4 def __str__(self):
5     return '({0}, {1})'.format(self.x, self.y)

```

23.2.2 3. podnalog: `__abs__`

Cilj. `abs(v)` naj vrne dolžino vektorja:

```

1 def __abs__(self):
2     return (self.x**2 + self.y**2)**0.5

```

23.2.3 4. in 5. podnalog: `__sub__` in `__truediv__`

Cilj. Razlika vektorjev in deljenje s skalarjem:

```

1 def __sub__(self, other):
2     return Vektor(self.x - other.x, self.y - other.y)
3
4 def __truediv__(self, skalar):
5     return self * (1 / skalar)

```

23.2.4 6. podnalog: sta pravokotna

Cilj. Vektorja sta pravokotna, ko je njun skalarni produkt 0. Primerjamo z majhno toleranco:

```

1 def sta_pravokotna(self, other):
2     return eq(self.skalarni_produkt(other), 0)

```

Novo v tem razdelku

Toleranca. Zaradi zaokrožitve realnih števil primerjamo z majhno toleranco: `eq(a, b) = abs(a - b) < 1e-12`.

23.2.5 7. podnalog: rotacija

Cilj. Vrtenje vektorja za kot `alpha`:

```

1 def rotacija(self, alpha):
2     return Vektor(self.x * math.cos(alpha) - self.y * math.sin(alpha),
3                   self.x * math.sin(alpha) + self.y * math.cos(alpha))

```

23.2.6 8. podnalog: projekcija

Cilj. Pravokotna projekcija točke na premico.

Vzamemo smerni vektor premice in vektor od izbrane točke premice do točke; projekcijo dobimo s skalarnim produktom:

```

1 def projekcija(self, tocka):
2     s = self.smernik()
3     v = tocka - self.tocka
4     w = s * (s * v) / (s * s)
5     return self.tocka.translacija(w)

```

23.2.7 9. podnalog: presek

Cilj. Presek dveh premic.

Premici zapišemo z normalama in izrazimo koordinate preseka:

```

1  def presek(self, other):
2      a1, b1 = self.normala.x, self.normala.y
3      a2, b2 = other.normala.x, other.normala.y
4      x1, y1 = self.tocka.x, self.tocka.y
5      x2, y2 = other.tocka.x, other.tocka.y
6      c = a1 * b2 - a2 * b1
7      x = (b1 * b2 * (y1 - y2) + a1 * b2 * x1 - a2 * b1 * x2) / c
8      y = (a1 * a2 * (x2 - x1) - a2 * b1 * y1 + a1 * b2 * y2) / c
9      return Tocka(x, y)

```

23.2.8 10. podnalog: zrcaljenje

Cilj. Zrcalna slika točke čez premico.

Projekcijo odštejemo dvakrat:

```

1  def zrcali_cez_premico(self, premica):
2      n = premica.normala
3      v = self - premica.tocka
4      w = n * (n * v) / (n * n)
5      return self.translacija(w * -2)

```

Zakaj deluje

w je vektor od točke premice do naše točke, projiciran na normalo. Če ga prištejemo dvakrat (z negativnim predznakom), točko prezrcalimo na drugo stran premice.

Končna koda (ključne metode).

```

1  class Vektor:
2      def __init__(self, x, y):
3          self.x = x
4          self.y = y
5
6      def __repr__(self):
7          return 'Vektor({0}, {1})'.format(self.x, self.y)
8
9      def __str__(self):
10         return '({0}, {1})'.format(self.x, self.y)
11
12     def __abs__(self):
13         return (self.x**2 + self.y**2)**0.5
14
15     def __sub__(self, other):
16         return Vektor(self.x - other.x, self.y - other.y)
17
18     def __truediv__(self, skalar):
19         return self * (1 / skalar)
20
21     def skalarni_produkt(self, other):
22         return self.x * other.x + self.y * other.y
23
24     def __mul__(self, other):

```

```
25     if type(other) == Vektor:
26         return self.skalarni_produkt(other)
27     else:
28         return Vektor(self.x * other, self.y * other)
29
30     def sta_pravokotna(self, other):
31         return eq(self.skalarni_produkt(other), 0)
32
33     def rotacija(self, alpha):
34         return Vektor(self.x * math.cos(alpha) - self.y * math.sin(alpha),
35                       self.x * math.sin(alpha) + self.y * math.cos(alpha))
```

23.3 Povzetek

- permutacijo normaliziramo (cikle zavrtimo in uredimo);
- inverz obrne vsak cikel, red je najmanjši skupni večkratnik dolžin;
- pri geometriji dunder metode omogočijo +, -, *, /, abs;
- realna števila primerjamo z toleranco.

Preveri se

1. Zakaj cikle normaliziramo?
2. Kako izračunamo red permutacije?
3. Zakaj pri primerjanju vektorjev uporabimo toleranco?

Poglavje 24

Mnogokotniki in grafi

V tem poglavju bomo utrdili objektno programiranje z dvema večjima razredoma: mnogokotnikom in neusmerjenim grafom.

24.1 Mnogokotnik

24.1.1 1. podnaloga: konstruktor

Cilj. `Mnogokotnik(oglisca)` naj shrani oglišča; če se prvo in zadnje ujemata (sklenjen zapis), zadnje odstranimo.

```
1 class Mnogokotnik:
2     def __init__(self, oglisca):
3         self.oglisca = list(oglisca)
4         if self.oglisca[0] == self.oglisca[-1]:
5             self.oglisca.pop()
```

Zakaj deluje

Nekateri mnogokotnike zapišejo tako, da na koncu ponovijo prvo oglišče. Tako podvojeno oglišče odstranimo, da je dolžina seznama enaka številu oglišč.

Končna koda.

```
1 class Mnogokotnik:
2     def __init__(self, oglisca):
3         self.oglisca = list(oglisca)
4         if self.oglisca[0] == self.oglisca[-1]:
5             self.oglisca.pop()
```

24.1.2 2. podnaloga: obseg

Cilj. `obseg(self)` naj vrne obseg.

Seštejemo dolžine vseh stranic. Ker je mnogokotnik sklenjen, gre zadnja stranica od zadnjega oglišča nazaj k prvemu (zato `(i + 1) % n`):

```
1     def obseg(self):
2         ob = 0
3         n = len(self.oglisca)
4         for i in range(n):
5             ob += dolzina_daljice(self.oglisca[i], self.oglisca[(i + 1) % n])
6         return ob
```

Končna koda.

```

1 def dolzina_daljice(a, b):
2     ax, ay = a
3     bx, by = b
4     return ((ax - bx)**2 + (ay - by)**2)**0.5
5
6     def obseg(self):
7         ob = 0
8         n = len(self.oglisca)
9         for i in range(n):
10             ob += dolzina_daljice(self.oglisca[i], self.oglisca[(i + 1) % n])
11         return ob

```

24.1.3 3. podnalog: ploščina

Cilj. `ploscina(self)` naj vrne ploščino.

Uporabimo formulo za ploščino mnogokotnika (shoelace), kjer vsakemu oglišču prištejemo $x*y_2 - x_2*y$, na koncu pa vzamemo absolutno vrednost in delimo z 2:

```

1 def ploscina(self):
2     a = 0
3     n = len(self.oglisca)
4     for i in range(n):
5         x, y = self.oglisca[i]
6         x2, y2 = self.oglisca[(i + 1) % n]
7         a += x * y2 - x2 * y
8     return abs(a) / 2

```

Končna koda.

```

1 def ploscina(self):
2     a = 0
3     n = len(self.oglisca)
4     for i in range(n):
5         x, y = self.oglisca[i]
6         x2, y2 = self.oglisca[(i + 1) % n]
7         a += x * y2 - x2 * y
8     return abs(a) / 2

```

24.1.4 4. podnalog: je konveksen

Cilj. `je_konveksen(self)` naj vrne `True`, če je mnogokotnik konveksen.

Pri obhodu po robu moramo vedno zavijati v isto smer. Smer ugotovimo z **z**-komponento vektorskega produkta zaporednih stranic:

```

1 def je_konveksen(self):
2     n = len(self.oglisca)
3     px, py = self.oglisca[-1]
4     qx, qy = self.oglisca[0]
5     rx, ry = self.oglisca[1]
6     u = (qx - px, qy - py, 0)
7     v = (rx - qx, ry - qy, 0)
8     znak = vektorski_produkt(u, v)[2]

```

Za vsa zaporedna oglišča preverimo, da ima vektorski produkt isti predznak:

```

1     for i in range(1, n):
2         px, py = self.oglisca[i - 1]
3         qx, qy = self.oglisca[i]
4         rx, ry = self.oglisca[(i + 1) % n]
5         u = (qx - px, qy - py, 0)
6         v = (rx - qx, ry - qy, 0)
7         znak2 = vektorski_produkt(u, v)[2]
8         if znak2 * znak < 0:
9             return False
10    return True

```

Zakaj deluje

$\text{znak2} * \text{znak} < 0$ pomeni, da sta si zaporedna ovinka v nasprotnih smereh — mnogokotnik torej ni konveksen.

Končna koda.

```

1 def vektorski_produkt(a, b):
2     ax, ay, az = a
3     bx, by, bz = b
4     return (ay * bz - az * by, az * bx - ax * bz, ax * by - ay * bx)
5
6 def je_konveksen(self):
7     n = len(self.oglisca)
8     px, py = self.oglisca[-1]
9     qx, qy = self.oglisca[0]
10    rx, ry = self.oglisca[1]
11    u = (qx - px, qy - py, 0)
12    v = (rx - qx, ry - qy, 0)
13    znak = vektorski_produkt(u, v)[2]
14    for i in range(1, n):
15        px, py = self.oglisca[i - 1]
16        qx, qy = self.oglisca[i]
17        rx, ry = self.oglisca[(i + 1) % n]
18        u = (qx - px, qy - py, 0)
19        v = (rx - qx, ry - qy, 0)
20        znak2 = vektorski_produkt(u, v)[2]
21        if znak2 * znak < 0:
22            return False
23    return True

```

24.2 Neusmerjen graf

24.2.1 1. podnalog: konstruktor

Cilj. NeusmerjenGraf(seznam_povezav) naj zgradi slovar sosedov.

Pripravimo prazen slovar. Za vsako povezavo (u, v) poskrbimo, da imata oba vozlišča svojo množico, in dodamo povezavo v obe smeri:

```

1 class NeusmerjenGraf:
2     def __init__(self, seznam_povezav):
3         self.sosedje = dict()
4         for u, v in seznam_povezav:
5             if u not in self.sosedje:

```

```

6         self.sosedje[u] = set()
7         self.sosedje[u].add(v)
8         if v not in self.sosedje:
9             self.sosedje[v] = set()
10            self.sosedje[v].add(u)

```

Zakaj deluje

Ker je graf neusmerjen, povezava $u - v$ velja v obe smeri, zato jo zabeležimo pri obeh vozliščih.

Končna koda.

```

1 class NeusmerjenGraf:
2     def __init__(self, seznam_povezav):
3         self.sosedje = dict()
4         for u, v in seznam_povezav:
5             if u not in self.sosedje:
6                 self.sosedje[u] = set()
7             self.sosedje[u].add(v)
8             if v not in self.sosedje:
9                 self.sosedje[v] = set()
10            self.sosedje[v].add(u)

```

24.2.2 2. podnalog: slovar stopenj

Cilj. Stopnja vozlišča je število njegovih sosedov:

```

1     def slovar_stopenj(self):
2         return {k: len(v) for k, v in self.sosedje.items()}

```

Končna koda.

```

1     def slovar_stopenj(self):
2         return {k: len(v) for k, v in self.sosedje.items()}

```

24.2.3 3. in 4. podnalog: dodaj in odstrani vozlišče

Cilj. `dodaj_vozlisce(u)` doda izolirano vozlišče; `odstrani_vozlisce(u)` odstrani vozlišče in vse njegove povezave.

Dodajanje je preprosto, če vozlišča še ni:

```

1     def dodaj_vozlisce(self, u):
2         if u not in self.sosedje:
3             self.sosedje[u] = set()

```

Pri odstranjevanju, če vozlišča ni, sprožimo izjemo `ValueError`:

```

1     def odstrani_vozlisce(self, u):
2         if u not in self.sosedje:
3             raise ValueError('vozlišče z oznako {0} ne obstaja'.format(repr(u)))

```

Nato vozlišče izbrisemo in ga odstranimo iz vseh sosedov:

```

1         del self.sosedje[u]
2         for mn_sos in self.sosedje.values():
3             mn_sos.discard(u)

```

Novo v tem razdelku

raise ValueError(...). Sproži izjemo z danim sporočilom. `set.discard(x)` odstrani `x`, če obstaja (brez napake, če ga ni).

Končna koda.

```

1  def dodaj_vozlisce(self, u):
2      if u not in self.sosedje:
3          self.sosedje[u] = set()
4
5  def odstrani_vozlisce(self, u):
6      if u not in self.sosedje:
7          raise ValueError('vozlišče z oznako {0} ne obstaja'.format(repr(u)))
8      del self.sosedje[u]
9      for mn_sos in self.sosedje.values():
10         mn_sos.discard(u)

```

24.2.4 5. in 6. podnalog: dodaj in odstrani povezavo

Cilj. `dodaj_povezavo(e)` doda povezavo (in manjkajoči vozlišči), `odstrani_povezavo(e)` pa jo odstrani.

Dodajanje najprej poskrbi za vozlišči, nato doda povezavo v obe smeri:

```

1  def dodaj_povezavo(self, e):
2      u, v = e
3      self.dodaj_vozlisce(u)
4      self.dodaj_vozlisce(v)
5      self.sosedje[u].add(v)
6      self.sosedje[v].add(u)

```

Pri odstranjevanju, če povezave ni, sprožimo izjemo, sicer jo odstranimo iz obeh smeri:

```

1  def odstrani_povezavo(self, e):
2      u, v = e
3      if u not in self.sosedje or v not in self.sosedje[u]:
4          raise ValueError('povezave {0} ni v grafu'.format(repr(e)))
5      self.sosedje[u].remove(v)
6      self.sosedje[v].remove(u)

```

Končna koda.

```

1  def dodaj_povezavo(self, e):
2      u, v = e
3      self.dodaj_vozlisce(u)
4      self.dodaj_vozlisce(v)
5      self.sosedje[u].add(v)
6      self.sosedje[v].add(u)
7
8  def odstrani_povezavo(self, e):
9      u, v = e
10     if u not in self.sosedje or v not in self.sosedje[u]:
11         raise ValueError('povezave {0} ni v grafu'.format(repr(e)))
12     self.sosedje[u].remove(v)
13     self.sosedje[v].remove(u)

```

24.2.5 7. podnalog: je povezan

Cilj. `je_povezan(self)` naj vrne `True`, če so vsa vozlišča v isti komponenti.

Najprej poiščemo komponento danega vozlišča z iskanjem v globino. Pripravimo množico in sklad:

```
1 def povezana_komponenta(self, vozlisce):
2     komponenta = {vozlisce}
3     sklad = [vozlisce]
```

Dokler imamo vozlišča, jih obdelujemo; nove sosedje dodajamo:

```
1 while sklad:
2     u = sklad.pop()
3     for v in self.sosedje[u]:
4         if v not in komponenta:
5             komponenta.add(v)
6             sklad.append(v)
7 return komponenta
```

Graf je povezan, če komponenta poljubnega vozlišča vsebuje vsa vozlišča:

```
1 def je_povezan(self):
2     if not self.sosedje:
3         return True
4     u = next(iter(self.sosedje))
5     return len(self.povezana_komponenta(u)) == len(self.sosedje)
```

Končna koda.

```
1 def povezana_komponenta(self, vozlisce):
2     komponenta = {vozlisce}
3     sklad = [vozlisce]
4     while sklad:
5         u = sklad.pop()
6         for v in self.sosedje[u]:
7             if v not in komponenta:
8                 komponenta.add(v)
9                 sklad.append(v)
10    return komponenta
11
12 def je_povezan(self):
13     if not self.sosedje:
14         return True
15     u = next(iter(self.sosedje))
16    return len(self.povezana_komponenta(u)) == len(self.sosedje)
```

24.2.6 8. podnalog: je drevo

Cilj. `je_drevo(self)` naj vrne `True`, če je graf drevo.

Najprej preštejemo vozlišča in povezave. Povezave preštejemo iz vsote stopenj (lema o rokovanju: vsota stopenj je dvakratnik števila povezav):

```
1 def stevilo_vozlisc(self):
2     return len(self.sosedje)
3
4 def stevilo_povezav(self):
5     return sum(self.slovar_stopenj().values()) // 2
```

Graf je drevo natanko tedaj, ko je povezan in velja $|E| + 1 = |V|$:

```

1 def je_drevo(self):
2     return (self.stevilo_vozlisc() == self.stevilo_povezav() + 1
3           and self.je_povezan())

```

Končna koda.

```

1 def stevilo_vozlisc(self):
2     return len(self.sosedje)
3
4 def stevilo_povezav(self):
5     return sum(self.slovar_stopenj().values()) // 2
6
7 def je_drevo(self):
8     return (self.stevilo_vozlisc() == self.stevilo_povezav() + 1
9           and self.je_povezan())

```

24.2.7 9. podnalog: induciran podgraf

Cilj. `induciran_podgraf(mnozica_vozlisc)` naj vrne nov graf na dani podmnožici vozlišč.

Pripravimo prazen graf in se sprehodimo po vozliščih. Če vozlišča ni, sprožimo izjemo; sicer ga dodamo in povežemo z vsemi sosedi v podmnožici:

```

1 def induciran_podgraf(self, mnozica_vozlisc):
2     ret = NeusmerjenGraf([])
3     for u in mnozica_vozlisc:
4         if u not in self.sosedje:
5             raise ValueError('vozlišče z oznako {0} ne obstaja'.format(repr(u)))
6         ret.dodaj_vozlisce(u)
7         for v in self.sosedje[u]:
8             if v in mnozica_vozlisc:
9                 ret.dodaj_povezavo((u, v))
10    return ret

```

Zakaj deluje

Induciran podgraf ohrani natanko tiste povezave, katerih obe krajišči sta v izbrani podmnožici.

Končna koda.

```

1 def induciran_podgraf(self, mnozica_vozlisc):
2     ret = NeusmerjenGraf([])
3     for u in mnozica_vozlisc:
4         if u not in self.sosedje:
5             raise ValueError('vozlišče z oznako {0} ne obstaja'.format(repr(u)))
6         ret.dodaj_vozlisce(u)
7         for v in self.sosedje[u]:
8             if v in mnozica_vozlisc:
9                 ret.dodaj_povezavo((u, v))
10    return ret

```

24.3 Povzetek

- **mnogokotnik:** obseg, ploščina (shoelace) in konveksnost (predznak vektorskega produkta);

- **graf** predstavimo s slovarjem sosedov (množice);
- komponente poiščemo z iskanjem v globino;
- drevo je povezan graf z $|E| + 1 = |V|$;
- `ValueError` uporabimo za neveljavne argumente.

Preveri se

1. Zakaj se pri obsegu uporabi $(i + 1) \% n$?
2. Kako iz stopenj izračunamo število povezav?
3. Kdaj je graf drevo?

Poglavje 25

Drevesa v praksi

V tem poglavju bomo rekurzivno obdelali tri drevesne strukture: mafijsko hierarhijo, vojaško hierarhijo in segmentno drevo za snežno službo.

25.1 Mafija

Drevo hrani ime (`ime`) in oddano količino denarja (`vrednost`). Pomožna metoda vrne vrednost ali 0 pri praznem vozlišču:

```
1 def get_vrednost(self):
2     return self.vrednost if not self.prazno else 0
```

25.1.1 1. podloga: koliko ponikne

Cilj. `koliko_ponikne(self)` naj vrne vsoto utajenega denarja.

Utajeni denar v vozlišču je razlika med tem, kar bi moral oddati (od podrejenih pobrano), in tem, kar je oddal. Če je razlika negativna, je ni:

```
1 def koliko_ponikne(self):
2     if self.prazno:
3         return 0
4     utajil = max(self.levo.get_vrednost() + self.desno.get_vrednost()
5                 - self.get_vrednost(), 0)
```

Utajenemu denarju v tem vozlišču prištejemo še utajeni denar poddreves:

```
1 return utajil + self.levo.koliko_ponikne() + self.desno.koliko_ponikne()
```

Končna koda.

```
1 def koliko_ponikne(self):
2     if self.prazno:
3         return 0
4     utajil = max(self.levo.get_vrednost() + self.desno.get_vrednost()
5                 - self.get_vrednost(), 0)
6     return utajil + self.levo.koliko_ponikne() + self.desno.koliko_ponikne()
```

25.1.2 2. podnaloga: goljufi

Cilj. `goljufi(self)` naj vrne množico naborov (`ime`, `utajil`).

```

1 def goljufi(self):
2     if self.prazno:
3         return set()
4     utajil = max(self.levo.get_vrednost() + self.desno.get_vrednost()
5                 - self.get_vrednost(), 0)

```

Če je utajil kaj, dodamo sebe, nato pa združimo z goljufi iz poddreves:

```

1     return (({self.ime, utajil}) if utajil > 0 else set())
2           | self.levo.goljufi() | self.desno.goljufi())

```

Končna koda.

```

1 def goljufi(self):
2     if self.prazno:
3         return set()
4     utajil = max(self.levo.get_vrednost() + self.desno.get_vrednost()
5                 - self.get_vrednost(), 0)
6     return (({self.ime, utajil}) if utajil > 0 else set())
7           | self.levo.goljufi() | self.desno.goljufi())

```

25.1.3 3. podnalog: zaslužek

Cilj. `zasluzek(self)` naj vrne par: vsota zaslužka majhnih rib in velikih rib (brez pobirkov).
Rekurzivno dobimo rezultate poddreves:

```

1 def zasluzek(self):
2     if self.prazno:
3         return 0, 0
4     l_male, l_velike = self.levo.zasluzek()
5     d_male, d_velike = self.desno.zasluzek()

```

Vozlišče je mala riba, če nima podrejenih. Njegov prispevek je tisto, kar mu ostane po oddaji pobirkov:

```

1     mala = self.levo.prazno and self.desno.prazno
2     prispevek = max(self.get_vrednost()
3                     - self.levo.get_vrednost()
4                     - self.desno.get_vrednost(), 0)

```

Prispevek prištejemo pravemu seznamu in vrnemo:

```

1     m, v = l_male + d_male, l_velike + d_velike
2     if mala:
3         m += prispevek
4     else:
5         v += prispevek
6     return m, v

```

Končna koda.

```

1 def zasluzek(self):
2     if self.prazno:
3         return 0, 0
4     l_male, l_velike = self.levo.zasluzek()
5     d_male, d_velike = self.desno.zasluzek()
6     mala = self.levo.prazno and self.desno.prazno
7     prispevek = max(self.get_vrednost()
8                     - self.levo.get_vrednost()

```

```

9         - self.desno.get_vrednost(), 0)
10    m, v = l_male + d_male, l_velike + d_velike
11    if mala:
12        m += prispevek
13    else:
14        v += prispevek
15    return m, v

```

25.2 Vojska Republike Banana

25.2.1 1. podnalog: izracunaj itm

Cilj. Vsakemu vozlišču pripnemo atribut `itm` (indeks telesne mase), zaokrožen na dve decimali.

Prazno vozlišče preskočimo:

```

1    def izracunaj_itm(self):
2        if self.prazno:
3            return

```

Izračunamo ITM (masa deljena s kvadratom višine v metrih) in nadaljujemo rekurzivno:

```

1        self.itm = round(self.masa / (self.visina / 100)**2, 2)
2        self.levo.izracunaj_itm()
3        self.desno.izracunaj_itm()

```

Končna koda.

```

1    def izracunaj_itm(self):
2        if self.prazno:
3            return
4        self.itm = round(self.masa / (self.visina / 100)**2, 2)
5        self.levo.izracunaj_itm()
6        self.desno.izracunaj_itm()

```

25.2.2 2. podnalog: trije debeluški

Cilj. `trije_debeluski(self)` naj vrne tri najdebelejše.

```

1    def trije_debeluski(self):
2        if self.prazno:
3            return []
4        debeli = [(self.ime, self.itm)]
5                  + self.levo.trije_debeluski()
6                  + self.desno.trije_debeluski()

```

Uredimo padajoče po ITM in vzamemo prvih treh:

```

1        debeli.sort(key=lambda x: -x[1])
2        return debeli[:3]

```

Novo v tem razdelku

`sort(key=...)`. Urejamo po drugem elementu para; negativna vrednost `-x[1]` poskrbi za padajoč vrstni red.

Končna koda.

```

1 def trije_debeluski(self):
2     if self.prazno:
3         return []
4     debeli = [(self.ime, self.itm)]
5               + self.levo.trije_debeluski()
6               + self.desno.trije_debeluski()
7     debeli.sort(key=lambda x: -x[1])
8     return debeli[:3]

```

25.2.3 3. podnalog: mali debeluški

Cilj. Enako, a le med vojaki, ki niso častniki (nimajo podrejenih).

Če je vozlišče list, je vojak in ga upoštevamo; sicer samega sebe izpustimo:

```

1 def mali_debeluski(self):
2     if self.prazno:
3         return []
4     if self.levo.prazno and self.desno.prazno:
5         return [(self.ime, self.itm)]
6     debeli = (self.levo.mali_debeluski()
7               + self.desno.mali_debeluski())
8     debeli.sort(key=lambda x: -x[1])
9     return debeli[:3]

```

Končna koda.

```

1 def mali_debeluski(self):
2     if self.prazno:
3         return []
4     if self.levo.prazno and self.desno.prazno:
5         return [(self.ime, self.itm)]
6     debeli = (self.levo.mali_debeluski()
7               + self.desno.mali_debeluski())
8     debeli.sort(key=lambda x: -x[1])
9     return debeli[:3]

```

25.3 Zasnežena cesta

Razred **Cesta** je segmentno drevo: vsako vozlišče pokriva interval `[zacetek, konec]` in hrani količino snega. Metoda `seznam` sešteje sneg po poti do listov:

```

1 def seznam(self):
2     if self.zacetek == self.konec:
3         return [self.sneg]
4     return [x + self.sneg for x in self.levo.seznam() + self.desno.seznam()]

```

25.3.1 1. podnalog: zasnezi

Cilj. `zasnezi(a, b, k)` naj doda `k` snega na odseke `[a, b]`.

Če je interval v celoti zunaj, ne naredimo nič:

```

1 def zasnezi(self, a, b, k):
2     if self.konec < a or b < self.zacetek:
3         return

```

Če je vozlišče v celoti znotraj intervala, dodamo sneg kar tu in nehamo (to je bistvo segmentnega drevesa):

```

1     if a <= self.zacetek and self.konec <= b:
2         self.sneg += k
3         return

```

Sicer se spustimo v oba otroka:

```

1     self.levo.zasnezi(a, b, k)
2     self.desno.zasnezi(a, b, k)

```

Zakaj deluje

Ker sneg dodamo le vozliščem, ki so v celoti znotraj intervala, je dodajanje učinkovito. Dejansko količino na odseku dobimo tako, da seštejemo **sneg** po poti od korena do lista.

Končna koda.

```

1     def zasnezi(self, a, b, k):
2         if self.konec < a or b < self.zacetek:
3             return
4         if a <= self.zacetek and self.konec <= b:
5             self.sneg += k
6             return
7         self.levo.zasnezi(a, b, k)
8         self.desno.zasnezi(a, b, k)

```

25.3.2 2. podnaloga: kolicina

Cilj. `kolicina(t)` naj vrne količino snega na odseku `t`.

Na listu vrnemo sneg (ali 0, če to ni naš odsek):

```

1     def kolicina(self, t):
2         if self.zacetek == self.konec:
3             return 0 if t != self.zacetek else self.sneg

```

Če `t` ni v intervalu, vrnemo 0; sicer seštejemo sneg vozlišča in količini iz otrok:

```

1     if t < self.zacetek or t > self.konec:
2         return 0
3     return self.sneg + self.levo.kolicina(t) + self.desno.kolicina(t)

```

Končna koda.

```

1     def kolicina(self, t):
2         if self.zacetek == self.konec:
3             return 0 if t != self.zacetek else self.sneg
4         if t < self.zacetek or t > self.konec:
5             return 0
6         return self.sneg + self.levo.kolicina(t) + self.desno.kolicina(t)

```

25.3.3 3. podnaloga: ocisti

Cilj. `ocisti(t)` naj počisti sneg z odseka `t`.

Če `t` ni v intervalu, ne naredimo nič; na listu nastavimo sneg na 0:

```

1 def ocisti(self, t):
2     if t < self.zacetek or t > self.konec:
3         return
4     if self.zacetek == self.konec:
5         self.sneg = 0
6         return

```

Če ima vozlišče nabran sneg, ga moramo najprej “spustiti” na otroke (na vse odseke razen t), preden počistimo t :

```

1     if self.sneg > 0:
2         self.levo.zasnezi(self.zacetek, t - 1, self.sneg)
3         self.levo.zasnezi(t + 1, self.konec, self.sneg)
4         self.desno.zasnezi(self.zacetek, t - 1, self.sneg)
5         self.desno.zasnezi(t + 1, self.konec, self.sneg)
6         self.sneg = 0
7     self.levo.ocisti(t)
8     self.desno.ocisti(t)

```

Zakaj deluje

Sneg je morda shranjen visoko v drevesu. Preden počistimo en odsek, moramo preostali sneg porazdeliti na otroke, da ne izgubimo informacije o drugih odsekih.

Končna koda.

```

1 def ocisti(self, t):
2     if t < self.zacetek or t > self.konec:
3         return
4     if self.zacetek == self.konec:
5         self.sneg = 0
6         return
7     if self.sneg > 0:
8         self.levo.zasnezi(self.zacetek, t - 1, self.sneg)
9         self.levo.zasnezi(t + 1, self.konec, self.sneg)
10        self.desno.zasnezi(self.zacetek, t - 1, self.sneg)
11        self.desno.zasnezi(t + 1, self.konec, self.sneg)
12        self.sneg = 0
13    self.levo.ocisti(t)
14    self.desno.ocisti(t)

```

25.4 Povzetek

- drevesa obdelujemo rekurzivno, prazno vozlišče je osnovni primer;
- pri mafiji vsoto utajenega denarja zberemo iz vseh vozlišč;
- pri vojski urejamo pare po ITM;
- **segmentno drevo** hrani vrednosti po intervalih in omogoča učinkovito posodabljanje in poizvedovanje.

Preveri se

1. Zakaj pri koliko_ponikne uporabimo `max(..., 0)`?

2. Kdaj je vojak “mala riba”?
3. Zakaj mora `ocisti` pred brisanjem spustiti sneg na otroke?

Poglavje 26

Podnapisi, koledarji in razporejanje

V zadnjem poglavju bomo rešili tri zaokrožene praktične naloge. Sklop *Podnapisi* je enak tistemu iz 17. poglavja, zato ga ne ponavljamo.

26.1 Gregorijanski koledar

Koledarja se razlikujeta le v pravilu za prestopna leta: po julijanskem je prestopno vsako leto deljivo s 4, po gregorijanskem pa le tista, ki so deljiva s 4 in ne s 100, ali pa s 400.

26.1.1 1. podnaloge: jul jutri

Cilj. `jul_jutri(d, m, l)` naj vrne naslednji dan po julijanskem koledarju.

Pripravimo seznam dolžin mesecev (na mestu 0 je 0, da indeks ustreza mesecu) in februar podaljšamo, če je leto prestopno:

```
1 def jul_jutri(d, m, l):
2     mes = [0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
3     if l % 4 == 0:
4         mes[2] += 1
```

Dan povečamo; če preseže mesec, ga postavimo na 1 in povečamo mesec:

```
1     d += 1
2     if d > mes[m]:
3         d = 1
4         m += 1
```

Če presežemo december, gremo na januar naslednjega leta:

```
1     if m > 12:
2         m = 1
3         l += 1
4     return (d, m, l)
```

Končna koda.

```
1 def jul_jutri(d, m, l):
2     mes = [0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
3     if l % 4 == 0:
4         mes[2] += 1
5     d += 1
6     if d > mes[m]:
7         d = 1
8         m += 1
```

```

9     if m > 12:
10         m = 1
11         l += 1
12     return (d, m, l)

```

26.1.2 2. podnalog: greg jutri

Cilj. Enako za gregorijanski koledar.

Edina razlika je pravilo za prestopno leto:

```

1     if (l % 4 == 0 and l % 100 != 0) or (l % 400 == 0):
2         mes[2] += 1

```

Končna koda.

```

1 def greg_jutri(d, m, l):
2     mes = [0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
3     if (l % 4 == 0 and l % 100 != 0) or (l % 400 == 0):
4         mes[2] += 1
5     d += 1
6     if d > mes[m]:
7         d = 1
8         m += 1
9     if m > 12:
10        m = 1
11        l += 1
12    return (d, m, l)

```

26.1.3 3. podnalog: jul2greg

Cilj. jul2greg(d, m, l) naj pretvori julijanski datum v gregorijanski.

Uporabimo znano ujemanje: 5. oktober 1582 (julijansko) je 15. oktober 1582 (gregorijansko). Oba datuma hkrati pomikamo naprej, dokler ne pridemo do želenega:

```

1 def jul2greg(d, m, l):
2     jul = (5, 10, 1582)
3     greg = (15, 10, 1582)

```

Zanka se ustavi, ko se julijanski datum izenači z vhodnim; takrat vrnemo ustrezni gregorijanski:

```

1     while jul != (d, m, l):
2         jul = jul_jutri(jul[0], jul[1], jul[2])
3         greg = greg_jutri(greg[0], greg[1], greg[2])
4     return greg

```

Konzola

```

>>> jul2greg(16, 2, 1923)
(1, 3, 1923)

```

Zakaj deluje

Zaradi razlike v prestopnih letih se koledarja razlikujeta za 13 dni (v 20. stoletju). Ker oba datuma premikamo z istim številom dni, je razlika med njima ves čas enaka.

Končna koda.

```

1 def jul2greg(d, m, l):
2     jul = (5, 10, 1582)
3     greg = (15, 10, 1582)
4     while jul != (d, m, l):
5         jul = jul_jutri(jul[0], jul[1], jul[2])
6         greg = greg_jutri(greg[0], greg[1], greg[2])
7     return greg

```

26.2 Izpitno obdobje

26.2.1 1. podnaloga: mozno resiti

Cilj. `mozno_resiti(p, s, d, m, k)` naj pove, ali lahko študenti rešijo vse naloge v k dneh. p je seznam nalog po predmetih, s število slabih in d dobrih študentov, m pa koliko nalog reši dober študent na dan.

Dobri študentje delajo pakete po m nalog iz istega predmeta. Preštujemo, koliko celih paketov zmoremo, in koliko nalog ostane:

```

1 def mozno_resiti(p, s, d, m, k):
2     delta = sum(x // m for x in p)
3     p2 = sorted([x % m for x in p], key=lambda x: -x)

```

Če dobri študentje ne zmorejo vseh paketov, morajo preostanek rešiti slabi:

```

1     if d * k < delta:
2         sigma = m * (delta - d * k) + sum(p2)
3         return sigma <= s * k

```

Sicer dobri poskrbijo za vse pakete in še za nekaj ostankov:

```

1     else:
2         delta2 = d * k - delta
3         sigma2 = sum(p2[delta2:])
4         return sigma2 <= s * k

```

Zakaj deluje

Ostanke uredimo padajoče, da jih dobri študentje poberejo čim več (najprej največje). Preostanek morajo rešiti slabi študentje, ki zmorejo $s * k$ nalog.

Končna koda.

```

1 def mozno_resiti(p, s, d, m, k):
2     delta = sum(x // m for x in p)
3     p2 = sorted([x % m for x in p], key=lambda x: -x)
4     if d * k < delta:
5         sigma = m * (delta - d * k) + sum(p2)
6         return sigma <= s * k
7     else:
8         delta2 = d * k - delta
9         sigma2 = sum(p2[delta2:])
10        return sigma2 <= s * k

```

26.2.2 2. podnalog: koliko dni

Cilj. `koliko_dni(p, s, d, m)` naj izračuna najmanjše število dni.

Ker je `mozno_resiti` monotona po k (več dni ne more škoditi), lahko najmanjše število dni poiščemo z *bisekcijo*:

```
1 def koliko_dni(p, s, d, m):
2     a = 0
3     b = 1 + sum(p) // (d + s)
```

Iščemo najmanjši c , pri katerem je `mozno_resiti` resničen:

```
1 while a < b:
2     c = (a + b) // 2
3     if mozno_resiti(p, s, d, m, c):
4         b = c
5     else:
6         a = c + 1
7 return a
```

Zakaj deluje

b je zagotovo dovolj velik, saj vsak študent reši vsaj eno nalogo na dan. Bisekcija najde najmanjši zadosten c v $O(\log(\text{sum}(p)))$ korakih.

Konzola

```
>>> koliko_dni([4, 2, 2], 2, 2, 3)
2
```

Končna koda.

```
1 def koliko_dni(p, s, d, m):
2     a = 0
3     b = 1 + sum(p) // (d + s)
4     while a < b:
5         c = (a + b) // 2
6         if mozno_resiti(p, s, d, m, c):
7             b = c
8         else:
9             a = c + 1
10    return a
```

26.3 Povzetek

- julijanski in gregorijanski koledar se razlikujeta le v prestopnih letih;
- pretvorbo med koledarjema dobimo s hkratnim pomikanjem obeh datumov;
- razporejanje del med dobre in slabe študente rešimo z monotono funkcijo in bisekcijo;
- **bisekcija** poišče najmanjše zadostno število dni.

Preveri se

1. Zakaj je 1900 prestopno po julijanskem, ne pa po gregorijanskem koledarju?
2. Zakaj je `mozno_resiti` monotona po `k`?
3. Zakaj lahko najmanjše število dni poiščemo z bisekcijo?

Dodatek A

Izpitne naloge

V tem dodatku so rešene izpitne naloge, katerih besedila so v celoti zapisana v gradivu. Pri nekaterih nalogah se podatki (npr. graf ali matrika) nahajajo na izpitnem listu; tam to posebej opozorimo.

A.1 Izpit 45 (1. rok, 2026)

A.1.1 1. naloga: Cenzura

Naloga. Funkcija `cenzura(f, besede, sporocilo)` naj vrne novo funkcijo, ki vrne isti niz kot `f`, razen kadar se v njem pojavi katera od prepovedanih besed — takrat vrne `sporocilo` (velikost črk ni pomembna).

Vrnemo notranjo funkcijo `f_c`, ki pokliče `f` in preveri prepovedane besede. Uporabimo `any` in male črke:

```
1 def cenzura(f, besede, sporocilo):
2     def f_c(arg):
3         niz = f(arg)
4         if any(b.lower() in niz.lower() for b in besede):
5             return sporocilo
6         return niz
7     return f_c
```

Za poljubno število argumentov zamenjamo `arg` z `*arg`:

```
1 def f_c(*arg):
2     niz = f(*arg)
```

Za maske dodamo še njihove preslikave prepovedanih besed:

```
1 def super_cenzura(f, besede, sporocilo, maske=[]):
2     for m in maske:
3         besede += [m(b) for b in besede]
4     return cenzura(f, besede, sporocilo)
```

Končna koda.

```
1 def cenzura(f, besede, sporocilo):
2     def f_c(*arg):
3         niz = f(*arg)
4         if any(b.lower() in niz.lower() for b in besede):
5             return sporocilo
6         return niz
7     return f_c
8
```

```

9 def super_cenzura(f, besede, sporocilo, maske=[]):
10     for m in maske:
11         besede += [m(b) for b in besede]
12     return cenzura(f, besede, sporocilo)

```

A.1.2 2. naloga: Štetje

Naloga. Razred `Stevec` šteje elemente iteratorja. Konstruktor je podan (atributi `iter`, `stevilo`, `tece`).

Enakost primerja preštete elemente in stanje; koliko vrne število pojavitev elementa:

```

1 def __eq__(self, other):
2     return self.stevilo == other.stevilo and self.tece == other.tece
3
4 def koliko(self, x):
5     return self.stevilo.get(x, 0)

```

`prestěj(n)` prešteje `n` elementov; ob izteku iteratorja posodobi stanje:

```

1 def prestej(self, n):
2     if self.tece:
3         while n:
4             try:
5                 x = next(self.iter)
6             except StopIteration:
7                 self.tece = False
8             return
9         self.stevilo[x] = self.stevilo.get(x, 0) + 1
10        n -= 1

```

Seštevanje združi dva iteratorja izmenoma. Uporabimo generator, ki izmenoma jema iz obeh, dokler kateri teče:

```

1 def __add__(self, other):
2     def g_na_roke():
3         prvi_tece = True
4         drugi_tece = True
5         while prvi_tece or drugi_tece:
6             if prvi_tece:
7                 try:
8                     yield next(self.iter)
9                 except StopIteration:
10                    prvi_tece = False
11             if drugi_tece:
12                 try:
13                     yield next(other.iter)
14                 except StopIteration:
15                    drugi_tece = False
16     return Stevec(g_na_roke())

```

Zakaj deluje

Generator `g_na_roke` jemlje izmenoma iz obeh iteratorjev. Ko eden izteče, nastavimo ustrežno zastavico in nadaljujemo z drugim. `StopIteration` je izjema, ki jo `next` sproži ob koncu iteratorja.

Končna koda.

```

1 class Stevec:
2     def __init__(self, iterator):
3         self.iter = iterator
4         self.stevilo = {}
5         self.tece = True
6
7     def __eq__(self, other):
8         return self.stevilo == other.stevilo and self.tece == other.tece
9
10    def koliko(self, x):
11        return self.stevilo.get(x, 0)
12
13    def prestej(self, n):
14        if self.tece:
15            while n:
16                try:
17                    x = next(self.iter)
18                except StopIteration:
19                    self.tece = False
20                return
21                self.stevilo[x] = self.stevilo.get(x, 0) + 1
22                n -= 1
23
24    def __add__(self, other):
25        def g_na_roke():
26            prvi_tece = True
27            drugi_tece = True
28            while prvi_tece or drugi_tece:
29                if prvi_tece:
30                    try:
31                        yield next(self.iter)
32                    except StopIteration:
33                        prvi_tece = False
34                if drugi_tece:
35                    try:
36                        yield next(other.iter)
37                    except StopIteration:
38                        drugi_tece = False
39        return Stevec(g_na_roke())

```

A.1.3 3. naloga: Grafi

Naloga. Graf je podan s slovarjem sosedov. *Opomba:* primer grafa je na izpitnem listu.

Pregled v globino z rekurzijo; sosedje obiskujemo urejeno:

```

1 def pregled_v_globino(graf, x):
2     obiskani = []
3     def obisci(n):
4         obiskani.append(n)
5         for s in sorted(graf[n]):
6             if s not in obiskani:
7                 obisci(s)
8     obisci(x)
9     return obiskani

```

Cikel odkrijemo, če med pregledom naletimo na že obiskano vozlišče (ki ni neposredni predhodnik):

```

1 def dosegljiv_cikel(graf, x):
2     obiskani = []
3     def obisci(n, prejsnji):
4         sosedi = [v for v in graf[n] if v != prejsnji]
5         if any(s in obiskani for s in sosedi):
6             return True
7         obiskani.append(n)
8         for s in sosedi:
9             if obisci(s, n):
10                return True
11        return False
12    return obisci(x, None)

```

Podgraf je vpeto drevo, če doseže vsa vozlišča grafa in nima cikla:

```

1 def vpeto_drevo(graf, podgraf):
2     sprehod = pregled_v_globino(podgraf, 1)
3     cikel = dosegljiv_cikel(podgraf, 1)
4     return sorted(sprehod) == list(range(1, len(graf) + 1)) and not cikel

```

Končna koda.

```

1 def pregled_v_globino(graf, x):
2     obiskani = []
3     def obisci(n):
4         obiskani.append(n)
5         for s in sorted(graf[n]):
6             if s not in obiskani:
7                 obisci(s)
8     obisci(x)
9     return obiskani
10
11 def dosegljiv_cikel(graf, x):
12     obiskani = []
13     def obisci(n, prejsnji):
14         sosedi = [v for v in graf[n] if v != prejsnji]
15         if any(s in obiskani for s in sosedi):
16             return True
17         obiskani.append(n)
18         for s in sosedi:
19             if obisci(s, n):
20                 return True
21        return False
22    return obisci(x, None)
23
24 def vpeto_drevo(graf, podgraf):
25     sprehod = pregled_v_globino(podgraf, 1)
26     cikel = dosegljiv_cikel(podgraf, 1)
27     return sorted(sprehod) == list(range(1, len(graf) + 1)) and not cikel

```

A.2 Izpit 46 (2. rok, 2026)

A.2.1 1. naloga: Sestavljanje

Naloga. `kovanci(n, denominacije)` naj pove, ali lahko z danimi kovanci natanko dosežemo `n`.

Osnovna primera in rekurzivni korak:

```

1 def kovanci(n, denominacije):
2     if n == 0:
3         return True
4     elif n < min(denominacije):
5         return False
6     return any(kovanci(n - i, denominacije) for i in denominacije)

```

razbij(niz, besede) naj vrne vsa razbitja niza na veljavne besede:

```

1 def razbij(niz, besede):
2     razbitja = set()
3     def pomocna(n, trenutna=[]):
4         if n == '':
5             razbitja.add(' '.join(trenutna))
6         else:
7             for b in besede:
8                 if n.startswith(b):
9                     pomocna(n[len(b):], trenutna + [b])
10    pomocna(niz)
11    return razbitja

```

kombiniraj(o, sumandi, prevec) naj vrne vse vsote, ki dajo o:

```

1 def kombiniraj(o, sumandi, prevec):
2     mozne = set()
3     def pomocna(trenutna=None, vsota=None):
4         if trenutna == o:
5             mozne.add(tuple(vsota))
6         else:
7             for s in sumandi:
8                 if trenutna is None:
9                     nova = s
10                    nova_vsota = [s]
11                else:
12                    nova = trenutna + s
13                    nova_vsota = vsota + [s]
14                if not prevec(nova, o):
15                    pomocna(nova, nova_vsota)
16    pomocna()
17    return mozne

```

Zakaj deluje

prevec(vsota, o) vrne True, kadar je vsota že presegla cilj. S tem odrežemo veje, ki ne morejo več pripeljati do rešitve.

Končna koda.

```

1 def kovanci(n, denominacije):
2     if n == 0:
3         return True
4     elif n < min(denominacije):
5         return False
6     return any(kovanci(n - i, denominacije) for i in denominacije)
7
8 def razbij(niz, besede):
9     razbitja = set()
10    def pomocna(n, trenutna=[]):

```

```

11     if n == '':
12         razbitja.add(' '.join(trenutna))
13     else:
14         for b in besede:
15             if n.startswith(b):
16                 pomozna(n[len(b):], trenutna + [b])
17     pomozna(niz)
18     return razbitja

```

A.2.2 2. naloga: Pesem višav

Naloga. Moira potuje po matriki od zgornjega levega do spodnjega desnega kota; premik navzdol stane vrednost polja, premik desno pa 1. *Opomba:* primer matrike je na izpitnem listu.

najhitreje izračuna minimalni čas z dinamičnim programiranjem od cilja nazaj:

```

1 def najhitreje(hribi):
2     n, m = len(hribi), len(hribi[0])
3     s = [[0 for j in range(m)] for i in range(n)]
4     for i in range(n - 1, -1, -1):
5         for j in range(m - 1, -1, -1):
6             if (i, j) == (n - 1, m - 1):
7                 s[i][j] = 0
8             elif i == n - 1:
9                 s[i][j] = s[i][j + 1] + 1
10            elif j == m - 1:
11                s[i][j] = s[i + 1][j] + hribi[i][j]
12            else:
13                s[i][j] = min(s[i][j + 1] + 1, s[i + 1][j] + hribi[i][j])
14    return s[0][0]

```

najhitrejsa_pot hrani še predhodnika vsakega polja, da pot rekonstruiramo:

```

1 prejsnji = [[None for j in range(m)] for i in range(n)]
2 for i in range(n):
3     for j in range(m):
4         if (i, j) == (0, 0):
5             s[i][j] = 0
6         elif i == 0:
7             s[i][j] = s[i][j - 1] + 1
8             prejsnji[i][j] = (i, j - 1)
9         elif j == 0:
10            s[i][j] = s[i - 1][j] + hribi[i - 1][j]
11            prejsnji[i][j] = (i - 1, j)
12        else:
13            s[i][j] = min(s[i][j - 1] + 1, s[i - 1][j] + hribi[i - 1][j])
14            if s[i][j - 1] + 1 < s[i - 1][j] + hribi[i - 1][j]:
15                prejsnji[i][j] = (i, j - 1)
16            else:
17                prejsnji[i][j] = (i - 1, j)

```

Nato pot sestavimo od cilja nazaj:

```

1 pot = [(n - 1, m - 1)]
2 x, y = (n - 1, m - 1)
3 while (x, y) != (0, 0):
4     pot.insert(0, prejsnji[x][y])
5     x, y = prejsnji[x][y]

```

```
6 return pot
```

najhitreje_jame doda možnost skoka čez dve vrstici (skozi jamo), kar stane polovico začetnega in polovico končnega polja plus 5 ur:

```
1         if i >= n - 2:
2             s[i][j] = min(s[i][j + 1] + 1, s[i + 1][j] + hribi[i][j])
3         else:
4             s[i][j] = min(s[i][j + 1] + 1,
5                           s[i + 1][j] + hribi[i][j],
6                           s[i + 2][j] + 5
7                           + hribi[i][j] / 2 + hribi[i + 2][j] / 2)
```

Končna koda (najhitreje).

```
1 def najhitreje(hribi):
2     n, m = len(hribi), len(hribi[0])
3     s = [[0 for j in range(m)] for i in range(n)]
4     for i in range(n - 1, -1, -1):
5         for j in range(m - 1, -1, -1):
6             if (i, j) == (n - 1, m - 1):
7                 s[i][j] = 0
8             elif i == n - 1:
9                 s[i][j] = s[i][j + 1] + 1
10            elif j == m - 1:
11                s[i][j] = s[i + 1][j] + hribi[i][j]
12            else:
13                s[i][j] = min(s[i][j + 1] + 1, s[i + 1][j] + hribi[i][j])
14    return s[0][0]
```

A.2.3 3. naloga: Vrhskala

Naloga. Izvidnica se giblje v spirali navzven; pot se konča, ko pride nad previs ($x \leq 0$).
Opomba: primer poti je na izpitnem listu.

Konstruktor vnaprej zgradi celotno pot (spiralo), premik pa se pomakne na naslednjo koordinato:

```
1 class Izvidnica:
2     def __init__(self, zacetna_x, zacetna_y):
3         self.x = zacetna_x
4         self.y = zacetna_y
5         self.zanimivosti = set()
6         self.pot = []
7         tx, ty = (self.x, self.y)
8         korak = 1
9         smeri = [(0, -1), (1, 0), (0, 1), (-1, 0)]
10        idx_smer = 0
11        done = tx <= 0
12        while not done:
13            for _ in range(korak):
14                dx, dy = smeri[idx_smer]
15                tx += dx
16                ty += dy
17                self.pot.append((tx, ty))
18                if tx <= 0:
19                    done = True
20                    break
21            idx_smer = (idx_smer + 1) % 4
```

```

22         if idx_smer % 2 == 0:
23             korak += 1
24
25     def premik(self):
26         if self.pot:
27             self.x, self.y = self.pot.pop(0)

```

preisci preveri zanimivost na trenutnih koordinatah:

```

1     def preisci(self):
2         if met_kock():
3             self.zanimivosti.add((self.x, self.y))
4
5     def do_sedaj(self):
6         return self.zanimivosti

```

preiskovanje simulira vse izvidnice hkrati, dokler vsaj ena še ni nad previsom:

```

1 def preiskovanje(zacetne):
2     izvidnice = [Izvidnica(x, y) for x, y in zacetne]
3     while any(i.x > 0 for i in izvidnice):
4         for i in izvidnice:
5             if i.x > 0:
6                 i.preisci()
7                 i.premik()
8     return izvidnice

```

Zakaj deluje

Ker vse izvidnice potujejo z isto hitrostjo, v vsakem koraku simulacije obdelamo vse, ki še niso končale. Zanka se ustavi, ko nobena ni več nad previsom.

Končna koda.

```

1 class Izvidnica:
2     def __init__(self, zacetna_x, zacetna_y):
3         self.x = zacetna_x
4         self.y = zacetna_y
5         self.zanimivosti = set()
6         self.pot = []
7         tx, ty = (self.x, self.y)
8         korak = 1
9         smeri = [(0, -1), (1, 0), (0, 1), (-1, 0)]
10        idx_smer = 0
11        done = tx <= 0
12        while not done:
13            for _ in range(korak):
14                dx, dy = smeri[idx_smer]
15                tx += dx
16                ty += dy
17                self.pot.append((tx, ty))
18                if tx <= 0:
19                    done = True
20                    break
21            idx_smer = (idx_smer + 1) % 4
22            if idx_smer % 2 == 0:
23                korak += 1
24

```

```
25     def premik(self):
26         if self.pot:
27             self.x, self.y = self.pot.pop(0)
28
29     def preisci(self):
30         if met_kock():
31             self.zanimivosti.add((self.x, self.y))
32
33     def do_sedaj(self):
34         return self.zanimivosti
35
36 def preiskovanje(zacetne):
37     izvidnice = [Izvidnica(x, y) for x, y in zacetne]
38     while any(i.x > 0 for i in izvidnice):
39         for i in izvidnice:
40             if i.x > 0:
41                 i.preisci()
42                 i.premik()
43     return izvidnice
```